

SiamRAM, From the Ground Up

A Complete, Self-Contained Mathematical & Theoretical Course

A theory course derived directly from the SiamRAM implementation

2026-06-13

Reverse-engineered from the implementation • distractor mode & spike watcher out of scope

Contents

SiamRAM, From the Ground Up — Reading Guide & System Overview	4
0. How to read this series	4
1. The problem: single-object tracking	5
2. Notation used throughout	6
3. The processing pipeline in one breath	6
4. The cast of subsystems	7
5. A note on honesty and on neural networks	8
6. What you should be able to do after each milestone	8
01 — Mathematical Foundations	9
1. Vectors	9
2. Matrices and linear maps	10
3. Functions you will meet constantly	11
4. A little calculus: derivatives, gradients, Jacobians	12
5. Probability you actually need	13
6. The exponential moving average (EMA)	15
7. Geometry helpers: angles, projections, robust averages	16
8. Putting the toolkit to work — a preview	16
02 — Bounding Boxes & Image Geometry	18
1. The image as a coordinate grid	18
2. Two ways to write a box	18
3. Intersection over Union (IoU): the overlap measure	19
4. From a box to a network input: crops and context	20
5. The score grid: where the network “looks”	22
6. The two coordinate spaces of SiamRAM	22
7. Recap	23
03 — Siamese Tracking and the SiamABC Matcher	24
1. The Siamese idea: tracking as matching	24
2. Cross-correlation: the comparison operation	24
3. The dual-template trick (why SiamABC survives change)	25
4. Inside the network: backbone, neck, attention, head	25
5. Anchor-free box decoding (FCOS style)	26
6. Penalties: stopping the tracker from teleporting	28
7. Turning the response into a confidence number	29
8. Test-Time Adaptation via Adaptive BatchNorm (brief)	30
9. The online dual-template update (RAM-lite)	30
10. The per-frame loop, assembled	31
11. Recap and limitations	32
04 — Probability, Bayesian Filtering, and the Kalman Filter	33
1. The problem: estimating a hidden state from noisy data	33
2. The state-space model	33
3. The recursive Bayesian recipe (two half-steps)	34
4. The Kalman filter in 1-D (full derivation)	35
5. The vector Kalman filter	36
6. The constant-velocity model (what SiamRAM uses)	37
7. When the model is nonlinear: the Extended Kalman Filter	37

8. The filtering algorithm in one box	38
9. Recap	38
05 — Homography Estimation and the Camera-Aware EKF	40
Part A — Projective geometry and the homography	40
Part B — Estimating the homography from two frames	42
Part C — The camera-aware EKF (BBoxEKF)	44
Part D — Using and trusting the camera motion	48
Recap	48
06 — Motion Priors: Steering the Matcher with Physics	50
1. Two questions, two priors	50
2. The target-motion Kalman prior (KalmanMotionPrior)	50
3. The camera-motion GMC prior (apply_gmc_search_prior)	53
4. Heavy camera motion: a gate for the whole system	53
5. How a frame uses all three (ordering)	54
6. Recap	55
07 — Appearance Memory: RAM and DRM	56
1. Why a separate appearance memory?	56
2. RAM — the short-term buffer	56
3. DRM — the long-term reference bank	57
4. The composite re-acquisition score (drm_match)	58
5. How the pieces relate	60
6. Recap	60
08 — Occlusion Recovery	61
1. Two worlds, one switch	61
2. Detecting loss	61
3. Coasting: the EKF runs the search	63
4. The phase state machine	63
5. Verification and the confirmation streak	65
6. The whole recovery loop	65
7. Recap	65
09 — Frame Dynamics: Injecting Motion Saliency	67
1. The problem: appearance collapses for tiny targets	67
2. Frame differencing: motion as a temporal gradient	67
3. SiamRAM’s constraint and its workaround	67
4. The tiny-only gate (why it’s not always on)	68
5. State and edge cases	69
6. Where it sits in the frame	69
7. Recap	69
10 — Adaptive Controllers: Self-Tuning Thresholds	70
1. Why fixed thresholds fail, and the general pattern	70
2. AutoConfThreshold — EMA-to-range mapping	71
3. AutoDrmMargin — sit just below the target’s own score	71
4. AutoTemplateRate — motion drives the update cadence	72
5. Other self-adjusting quantities (cross-references)	72
6. Why this design is safe	73
7. Recap	73
11 — Putting It All Together: The Conductor	74
1. The two-state machine	74
2. The per-frame entry (update / _update_impl)	74
3. The normal update, ordered (_normal_update)	75

4. The full wiring: where every subsystem plugs in

76

5. Three worked frames

76

6. The design philosophy, in one view

77

7. Where to go next

78

SiamRAM, From the Ground Up — Reading Guide & System Overview

Who this is for. A student who likes mathematics and wants to understand, not just use, a modern single-object visual tracker. We assume you know high-school algebra and a little calculus (what a derivative is). Everything else — vectors, matrices, probability, Gaussians, Kalman filters, homographies, convolution — is built up from scratch in these notes. Nothing is hand-waved.

Scope. These notes describe the **SiamRAM** tracker in this repository, component by component, theoretically and mathematically. They deliberately **ignore “distractor mode”** (the identity-arbitration subsystem) **and the spike watcher** that feeds it — that line of work is a deprecated/failed experiment and is out of scope here. Everything else — the visual matcher, the motion filter, the camera-motion estimator, the appearance memory, occlusion recovery, frame-dynamics motion injection, the adaptive controllers, and how they connect — is covered in full.

0. How to read this series

The documents are numbered. **Read them in order the first time.** The early ones build tools and core subsystems; the later ones assemble those subsystems into the full tracker.

#	File	What it teaches	Depends on
00	00_READING_GUIDE.md	(this file) the problem, the map, notation	—
01	01_MATHEMATICAL_FOUNDATIONS.md	vectors, matrices, norms, dot/cosine, sigmoid, argmax, expectation, Gaussians, EMA, interpolation	—
02	02_BOUNDING_BOXES_AND_IMAGE_GEOMETRY.md	boxes, IoU, crops, context, coordinate spaces, the score grid	01
03	03_SIAMESE_TRACKING_SIAMABC.md	the visual matcher: correlation, attention, FCOS decoding, penalties, dual templates	01, 02
04	04_PROBABILITY_AND_BAYESIAN_FILTERING.md	Bayes’ rule, recursive estimation, the Kalman filter derived from scratch, the EKF	01
05	05_HOMOGRAPHY_AND_EKF.md	projective geometry, homographies, optical flow, RANSAC, and SiamRAM’s camera-aware EKF	01, 04
06	06_MOTION_PRIORS.md	the constant-velocity Kalman prior, KF-IoU response fusion, the GMC search prior, heavy-motion gating	03, 04, 05
07	07_APPEARANCE_MEMORY_RAM_DRM.md	appearance descriptors, short-term RAM, long-term DRM, the composite re-acquisition score	01, 02

#	File	What it teaches	Depends on
08	08_OCCLUSION_RECOVERY.md	loss detection, EKF-guided search, the phase machine, YOLO re-detection, verification	05, 06, 07
09	09_FRAME_DYNAMICS.md	injecting short-term motion saliency into the search crop for tiny targets	02, 03
10	10_ADAPTIVE_CONTROLLERS.md	exponential moving averages as controllers; the three auto-tuners	01
11	11_PUTTING_IT_ALL_TOGETHER.md	the per-frame conductor and the full state machine	all

If you only have time for one “deep” file, the user-requested centerpiece is **05 (homography + EKF)**, but it leans on **04 (Kalman filtering)**, so read those two together.

1. The problem: single-object tracking

You are given a video — a sequence of images (frames) $I_0, I_1, I_2, \dots$ — and, **on the first frame only**, a rectangle telling you where a target object is:

given: I_0 and a box $b_0 = (x_0, y_0, w_0, h_0)$.

Your job: for every later frame I_t , output a box b_t that bounds the same object. You are never told the answer again. You must follow that one object — and only that object — through everything the world throws at it.

This is **single-object tracking (SOT)**, also called model-free tracking because you are not told what class of thing you are tracking (it might be a fish, a drone, a player, a car). You only know its appearance from one frame.

1.1 Why this is hard

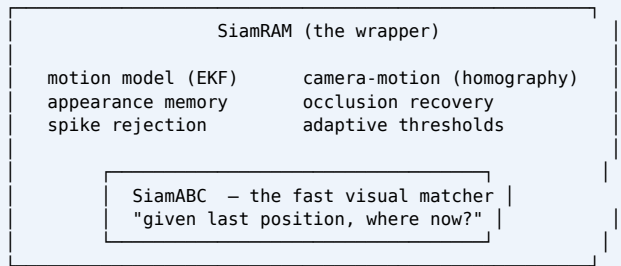
If the target always stayed sharp, fully visible, alone, and still relative to the camera, a simple template-matcher would suffice. Real video breaks each of those assumptions:

- **Appearance change.** The object rotates, deforms, changes lighting, changes scale. The frame-0 crop stops looking like the object.
- **Camera motion.** The camera pans, zooms, shakes. The object can move dozens of pixels in the image without moving at all in the world.
- **Occlusion / leaving frame.** The object disappears behind something, or exits the frame entirely, then comes back — possibly far from where it vanished.
- **Look-alikes (distractors).** Another similar object passes by, and a naive matcher silently jumps onto the impostor. (SiamRAM has a “distractor mode” for this; we set it — and the spike watcher that triggers it — aside by design.)
- **Tiny / low-texture targets.** A small, faint, or distant object barely registers in appearance; its motion is often the strongest cue that it is there at all.

SiamRAM is, essentially, **a fast visual matcher wrapped in a stack of mathematical safety nets**, each one designed to survive one of these failure modes.

1.2 The one mental model

Hold this picture in your head for the whole series:



- The **inner matcher** (SiamABC, doc 03) is fast and precise but myopic. It only looks in a small window near where the target was last, and it has no memory and no notion of physics — only “what looks most like the template, right here, right now.”
- The **outer layer** (everything else) watches the matcher. It predicts where the target should be (motion + camera models), remembers what it looks like (appearance memory), notices when the matcher’s confidence collapses (loss detection), runs a structured search to find it again (occlusion recovery), and rejects physically implausible jumps (spike rejection). It also tunes its own thresholds to the difficulty of the current video (adaptive controllers).

Every later document is one of those safety nets, explained in full.

2. Notation used throughout

We fix conventions now so later math is unambiguous.

Scalars, vectors, matrices. Scalars are italic lowercase: x, w, s . Vectors are bold lowercase: $\mathbf{x} = (x_1, \dots, x_n)$, treated as **column** vectors unless stated. Matrices are uppercase: A, H, P . The transpose is $A^\top$; the inverse is A^{-1} ; the $n \times n$ identity is I_n (or just I).

Frames and time. I_t is the image at discrete time (frame index) $t = 0, 1, 2, \dots$. A quantity “at time t ” gets subscript t ; “predicted before seeing the measurement” gets a minus superscript, e.g. $\hat{\mathbf{x}}_t^-$; “after correction” gets a plus, $\hat{\mathbf{x}}_t^+$.

Bounding boxes. Two formats appear constantly (doc 02 details them):

$$b = (x, y, w, h) \quad (\text{top-left corner + size}), \quad b = (x_1, y_1, x_2, y_2) \quad (\text{two corners}).$$

The **center** of a box is $(c_x, c_y) = (x + \frac{w}{2}, y + \frac{h}{2})$.

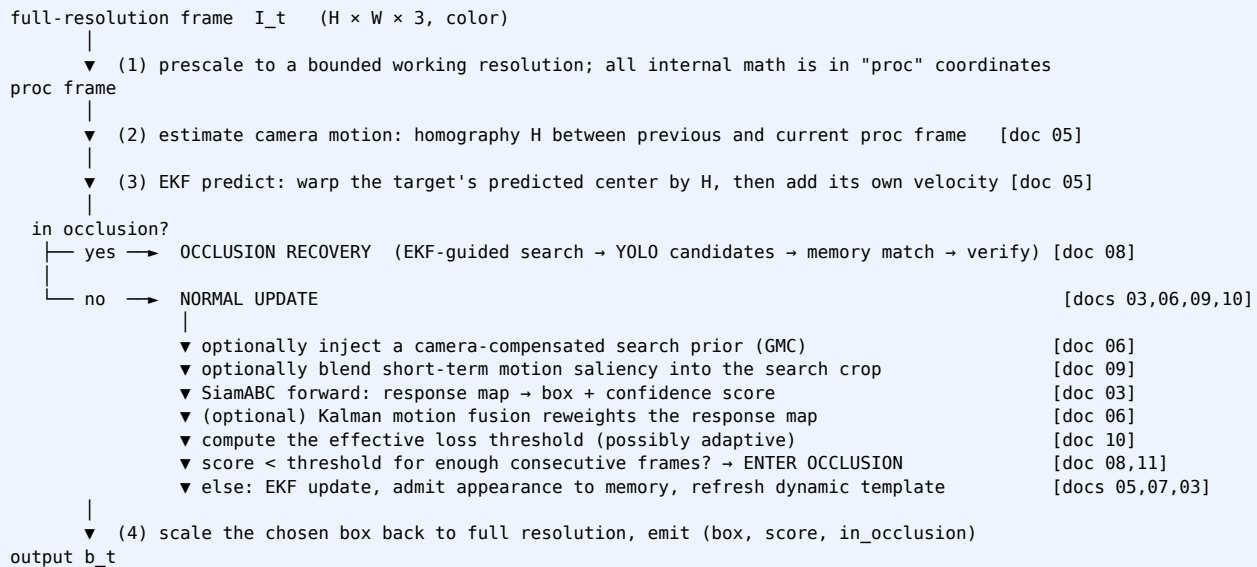
Pixels and coordinates. Image coordinates are (u, v) or (x, y) with x to the right and y **downward** (the standard image convention — the origin is the top-left corner). A point in the image is $\mathbf{p} = (x, y)$.

Probability. $p(\mathbf{x})$ is a probability density; $\mathbb{E}[\cdot]$ is expectation; $\text{Cov}[\cdot]$ is covariance; $\mathcal{N}(\mu, \Sigma)$ is the Gaussian with mean μ and covariance Σ .

Indicator / clamp. $\mathbb{1}[\text{condition}]$ is 1 when the condition holds, else 0. $\text{clip}(z, a, b) = \min(\max(z, a), b)$ clamps z into $[a, b]$.

3. The processing pipeline in one breath

Before diving into subsystems, here is the journey of a single frame through SiamRAM. Every box and arrow below is expanded — with its mathematics — somewhere in docs 03–11. Treat this as the skeleton.



Two structural facts to anchor on now, because they recur:

1. **Exactly one of normal update or occlusion recovery runs per frame.** They are two disjoint “worlds.” The tracker is a state machine with these two macro-states (plus the distractor world we are ignoring).
2. **All internal computation happens in a single, downscaled “proc” coordinate space,** and only the final box is scaled back to the original video resolution. So when later docs talk about pixels, displacements, or box diagonals, they mean proc pixels unless noted. This keeps every subsystem (matcher, motion, camera, YOLO, memory) speaking the same units.

4. The cast of subsystems

A quick glossary so the names in later docs are familiar. Each links to its full treatment.

- **SiamABC** (doc 03) — the inner Siamese neural matcher. Compares a template of the target to a search region and outputs a dense map of “is the target here?” scores plus box geometry. Fast, local, appearance-only.
- **EKF** — Extended Kalman Filter (docs 04, 05) — a recursive estimator of the target’s **center and velocity** (c_x, c_y, v_x, v_y). It predicts where the target will be next frame and corrects itself from the matcher’s output. It can fold in **camera motion** through a homography.
- **GMC** — Global Motion Compensation (doc 06) — uses the estimated camera homography to shift the search region to where the target should appear after the camera moved.
- **KF motion prior** (doc 06) — a second, constant-velocity Kalman filter over the full box ($c_x, c_y, w, h, \dots$) whose predicted box reweights the matcher’s response map toward motion-consistent locations (the “SAMURAI” idea).
- **Appearance memory: RAM + DRM** (doc 07) — RAM is a short-term buffer of recent confident target descriptors; DRM (“Dynamic Reference Memory”) is a curated long-term bank used to re-identify the target after it is lost. Matching uses a composite of appearance, overlap, motion direction, and recency.
- **Occlusion recovery** (doc 08) — the structured re-detection pipeline that runs once the matcher’s confidence has collapsed: it coasts on the EKF, grows a search region, runs

an object detector (YOLO), scores candidates against DRM, then verifies a winner by re-running SiamABC.

- **Frame dynamics** (doc 09) — injects short-term motion saliency (frame differences) into the search crop to help lock onto tiny moving targets.
 - **Adaptive controllers** (doc 10) — small exponential-moving-average estimators that retune the confidence threshold, the re-acquisition margin, and the template-update rate to each video’s difficulty.
 - **The conductor** (doc 11) — the master per-frame routine (update) that wires all the above into a coherent state machine.
-

5. A note on honesty and on neural networks

Two caveats that matter for a careful reader.

We describe the code that runs. Where a research paper’s intent and the implementation differ, these notes follow the implementation, because that is what produces the tracker’s behavior. Citations to the source look like `tracker.py` or name a function.

The neural network is not derived here. SiamABC’s weights come from training, which is its own large topic. These notes explain the **architecture and the math of how the network is used at inference** — what it consumes, what it outputs, how those outputs are turned into a box, and why each operation is shaped the way it is. We treat the trained convolutional backbone as a given function f_θ that maps an image crop to a feature map; doc 03 explains everything built around it. That is the right altitude for understanding the tracker (as opposed to re-deriving deep learning).

6. What you should be able to do after each milestone

- **After doc 03:** explain how a Siamese tracker turns two image crops into a bounding box, and why a cosine window and a scale penalty stop it from teleporting.
- **After doc 05:** derive the Kalman prediction/update equations, explain what a homography is and how it is estimated from optical flow + RANSAC, and read the EKF’s Jacobian line by line.
- **After doc 07:** explain the multi-term re-acquisition score and why each term is there.
- **After doc 11:** trace a single frame through the entire state machine and say, at each branch, why the tracker does what it does.

Turn to **01_MATHEMATICAL_FOUNDATIONS.md** to assemble the toolkit.

01 — Mathematical Foundations

This document builds, from scratch, every piece of mathematics the rest of the series uses. If you already know linear algebra and probability, skim it and move on; if you don't, read it carefully — later documents lean on every section here and assume no further background. Each idea is introduced, defined, given intuition, and connected to where SiamRAM uses it.

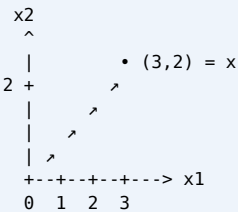
1. Vectors

A **vector** is an ordered list of numbers. We write a vector of n numbers as a column:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

$\mathbb{R}^n$ means “the set of all lists of n real numbers.” In SiamRAM, vectors are everywhere: a target's position is a vector (c_x, c_y) ; the EKF's state is a vector (c_x, c_y, v_x, v_y) ; an appearance “descriptor” is a vector of 512 numbers.

Geometric picture. A vector in $\mathbb{R}^2$ is an arrow from the origin to the point (x_1, x_2) . Addition is tip-to-tail; scaling by α stretches the arrow.



1.1 Length (norm)

The **Euclidean norm** (length) of a vector is

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{\sum_{i=1}^n x_i^2}.$$

This is just the Pythagorean theorem in n dimensions. The distance between two points $\mathbf{a}$ and $\mathbf{b}$ is the length of their difference, $\|\mathbf{a} - \mathbf{b}\|$. In code this is the `hypot` function for 2-D: $\sqrt{\Delta x^2 + \Delta y^2}$. SiamRAM measures, e.g., how far a box center moved between frames as exactly this.

A **unit vector** has length 1. Any nonzero vector can be **normalized** to unit length by dividing by its norm: $\hat{\mathbf{x}} = \mathbf{x} / \|\mathbf{x}\|$.

1.2 The dot product

The **dot product** (inner product) of two vectors of the same size is

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b} = \sum_{i=1}^n a_i b_i.$$

It produces a single number. Two facts make it indispensable:

1. **It relates to length:** $\mathbf{a} \cdot \mathbf{a} = \|\mathbf{a}\|^2$.
2. **It relates to angle:** if θ is the angle between $\mathbf{a}$ and $\mathbf{b}$, then

$$\boxed{\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta}$$

So the dot product is large and positive when two vectors point the same way, zero when they are perpendicular, and negative when they point oppositely. This single identity powers two SiamRAM ideas: **cosine similarity** of appearance descriptors (§3) and **direction consistency** of motion (doc 07).

1.3 Cosine similarity

Rearranging the boxed identity, the cosine of the angle between two vectors is

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \in [-1, 1].$$

This is **cosine similarity**. It ignores the magnitudes of the vectors and compares only their directions. Why is that the right tool for appearance?

A neural network turns an image crop into a feature vector (a “descriptor”). Two crops of the same object produce descriptors pointing in nearly the same direction ($\cos \theta \approx 1$), even if overall brightness scales them differently; two crops of different objects point in different directions ($\cos \theta$ closer to 0). Cosine similarity is brightness-/scale-robust, which is exactly what you want for “do these two crops show the same thing?” SiamRAM’s `_cos_sim` computes precisely

$$\text{cossim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\| + \varepsilon},$$

with a tiny ε (e.g. 10^{-8}) added to the denominator to avoid division by zero when a vector is all zeros. (That ε trick appears everywhere in numerical code; whenever you see “+ 1e-8” in a denominator, it is guarding against divide-by-zero.)

2. Matrices and linear maps

A **matrix** is a rectangular grid of numbers. An $m \times n$ matrix has m rows and n columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

2.1 Matrix-vector multiplication

A matrix multiplies a vector to produce another vector. If A is $m \times n$ and $\mathbf{x} \in \mathbb{R}^n$, then $\mathbf{y} = A\mathbf{x} \in \mathbb{R}^m$ has entries

$$y_i = \sum_{j=1}^n a_{ij} x_j.$$

The key interpretation: **a matrix is a linear transformation**. $A\mathbf{x}$ takes the vector $\mathbf{x}$ and rotates/scales/shears/projects it. “Linear” means it respects addition and scaling: $A(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha A\mathbf{x} + \beta A\mathbf{y}$. Lines through the origin stay lines through the origin; the grid stays evenly spaced. This is why so much of tracking is matrix algebra: camera rotation, the constant-velocity motion model, and the Kalman filter’s state transition are all linear maps.

2.2 Matrix-matrix multiplication

If A is $m \times k$ and B is $k \times n$, their product $C = AB$ is $m \times n$ with

$$c_{ij} = \sum_{\ell=1}^k a_{i\ell} b_{\ell j}.$$

Composing transformations = multiplying matrices: applying B then A to a vector is $(AB)\mathbf{x}$. **Order matters:** in general $AB \neq BA$.

2.3 Identity, transpose, inverse

- The **identity matrix** I_n has 1s on the diagonal, 0s elsewhere; $I\mathbf{x} = \mathbf{x}$ (it does nothing).
- The **transpose** $A^\top$ flips rows and columns: $(A^\top)_{ij} = a_{ji}$.
- The **inverse** A^{-1} (only for square matrices, and only when it exists) “undoes” A : $A^{-1}A = AA^{-1} = I$. So if $\mathbf{y} = A\mathbf{x}$ then $\mathbf{x} = A^{-1}\mathbf{y}$. Inverting is how the Kalman filter “solves” for the optimal correction (doc 04). For a 2×2 matrix there is a closed form; for larger ones the computer uses standard numerical routines (`np.linalg.inv`).

2.4 The diagonal and trace

A **diagonal matrix** has nonzero entries only on the diagonal; `np.diag([a,b,c])` builds one. Multiplying by a diagonal matrix just scales each coordinate independently. SiamRAM initializes covariance matrices as diagonals (doc 04/05), meaning “each state variable has its own uncertainty and they start uncorrelated.”

2.5 Symmetric and positive-(semi)definite matrices

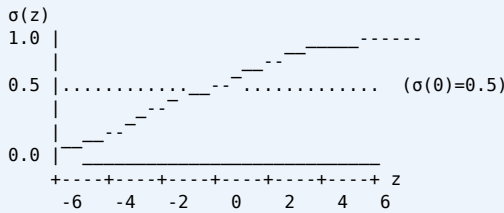
A matrix is **symmetric** if $A = A^\top$. A symmetric matrix is **positive semidefinite (PSD)** if $\mathbf{x}^\top A\mathbf{x} \geq 0$ for all $\mathbf{x}$, and **positive definite** if the inequality is strict for $\mathbf{x} \neq 0$. Covariance matrices (§5.5) are always symmetric PSD. Intuition: $\mathbf{x}^\top A\mathbf{x}$ is a “generalized squared length,” so a covariance can never assign negative variance to any direction.

3. Functions you will meet constantly

3.1 The sigmoid (logistic) function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

It squashes any real number into the open interval $(0, 1)$. It is increasing, with $\sigma(0) = \frac{1}{2}$, $\sigma(-\infty) = 0$, $\sigma(+\infty) = 1$. SiamABC’s classification head outputs raw scores (“logits”) in $(-\infty, \infty)$; applying σ turns each into a probability-like confidence in $(0, 1)$ that “the target is at this location.”



3.2 argmax

Given a list (or grid) of numbers, **argmax** returns the **index of the largest one** (as opposed to **max**, which returns the value). If $s = (0.1, 0.9, 0.3)$, then $\max s = 0.9$ but $\operatorname{argmax} s = 2$ (the second index, 1-based). SiamABC decodes a box by taking the **argmax** over a 2-D grid of confidence scores: “the target is at the grid cell with the highest score.”

For a 2-D grid flattened to 1-D index k , the row and column are recovered by integer division and remainder: $r = \lfloor k/W \rfloor$, $c = k \bmod W$, where W is the grid width. (You will see exactly this in the decoder.)

3.3 Exponential and its uses

e^z (with $e \approx 2.718$) grows fast for positive z and decays toward 0 for negative z , always staying positive. Two recurring uses:

- **Decay weights.** $e^{-\alpha t}$ for $\alpha > 0, t \geq 0$ starts at 1 ($t = 0$) and decays to 0. SiamRAM uses $e^{-\alpha \cdot \text{age}}$ to make old memory entries count less (doc 07), and $e^{-(\dots)}$ to penalize sudden scale changes (doc 03).
- **Gaussian bumps.** $e^{-d^2/(2\sigma^2)}$ is a smooth bump that is 1 at $d = 0$ and falls off with distance d ; it appears in the spatial distance penalty (doc 07) and in soft training labels (doc 03).

3.4 Linear interpolation (lerp)

To blend smoothly between a “slow” value and a “fast” value as a knob t moves from 0 to 1:

$$\operatorname{lerp}(a, b, t) = a + (b - a)t = (1 - t)a + tb, \quad t \in [0, 1].$$

At $t = 0$ you get a ; at $t = 1$ you get b ; in between, a straight-line blend. SiamRAM’s adaptive template-rate controller (doc 10) lerps the update period between a slow setting and a fast setting based on how much motion there is.

3.5 Clamping (clip)

$$\operatorname{clip}(z, a, b) = \min(\max(z, a), b)$$

forces z into $[a, b]$: anything below a becomes a , anything above b becomes b . Used to keep boxes inside the frame, probabilities inside $[0, 1]$, and adaptive thresholds inside their legal range.

4. A little calculus: derivatives, gradients, Jacobians

You need only the idea of a derivative plus how it generalizes to vectors.

4.1 Derivative

For a function $f(x)$ of one variable, the **derivative** $f'(x) = \frac{df}{dx}$ is the slope of f at x — how fast the output changes per unit change in input. Near a point x_0 , f is well approximated by its tangent line:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0).$$

This is the **first-order (linear) approximation**, and it is the entire idea behind the Extended Kalman filter: replace a curved function by its tangent line near the current estimate so the linear Kalman machinery applies (doc 05).

4.2 Partial derivatives and the Jacobian

If a function has several inputs, $f(x_1, \dots, x_n)$, the **partial derivative** $\partial f / \partial x_j$ is its slope with respect to x_j while holding the others fixed.

Now consider a function that maps a vector to a vector, $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, written componentwise as $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))$. Its **Jacobian** is the $m \times n$ matrix of all partial derivatives:

$$J = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}.$$

The Jacobian is the multidimensional generalization of “slope”: near $\mathbf{x}_0$,

$$\mathbf{f}(\mathbf{x}) \approx \mathbf{f}(\mathbf{x}_0) + J(\mathbf{x} - \mathbf{x}_0).$$

When SiamRAM’s EKF warps a point through a camera homography (a nonlinear operation, because of the perspective division), it computes exactly this Jacobian to linearize that warp; doc 05 derives every entry of it.

4.3 The quotient rule (the one calculus rule we actually re-derive)

Because the homography warp involves a division, we will need the derivative of a ratio. If

$$f(x) = \frac{g(x)}{h(x)} \text{ then}$$

$$f'(x) = \frac{g'(x)h(x) - g(x)h'(x)}{h(x)^2}.$$

Keep this handy; it is the only nontrivial differentiation in the whole series, and doc 05 walks through applying it.

5. Probability you actually need

Tracking is estimation under uncertainty, so we need a working vocabulary of probability. Doc 04 turns this into the Kalman filter.

5.1 Random variables and densities

A **continuous random variable** X doesn’t have a single value but a **probability density function (pdf)** $p(x) \geq 0$ describing how likely each value is. Probabilities are areas under the density: $P(a \leq X \leq b) = \int_a^b p(x) dx$, and the total area is 1, $\int_{-\infty}^{\infty} p(x) dx = 1$.

5.2 Expectation and variance

The **expectation** (mean) is the average value, weighting each x by its density:

$$\mu = \mathbb{E}[X] = \int x p(x) dx.$$

The **variance** measures spread — the average squared distance from the mean:

$$\sigma^2 = \text{Var}[X] = \mathbb{E}[(X - \mu)^2].$$

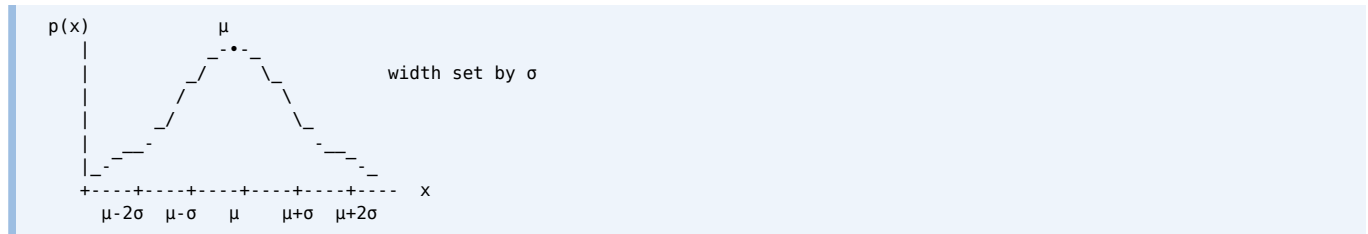
Its square root σ is the **standard deviation**, in the same units as X . Small σ = confident/peaked; large σ = uncertain/spread out.

5.3 The Gaussian (normal) distribution

The single most important distribution for tracking is the **Gaussian**:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

It is the classic symmetric “bell curve,” centered at μ , with width set by σ .



Why Gaussians dominate tracking:

1. **Two numbers describe everything.** A Gaussian is fully specified by its mean μ and variance σ^2 . So “what we believe about the target’s position” can be stored as just a center and an uncertainty.
2. **They stay Gaussian under linear maps.** If X is Gaussian and you apply a linear transformation, the result is still Gaussian. This is why the Kalman filter is exact for linear systems: belief stays Gaussian forever, so you only ever propagate (μ, σ^2) .
3. **The product of two Gaussians is (proportional to) a Gaussian.** Combining a prediction and a measurement — both Gaussian — yields another Gaussian. That product is the Kalman update.

5.4 The multivariate Gaussian

For a vector $\mathbf{x} \in \mathbb{R}^n$, the Gaussian generalizes to

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)\right),$$

written $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$. Here $\mu \in \mathbb{R}^n$ is the mean vector and Σ (an $n \times n$ symmetric PSD matrix) is the **covariance matrix**.

5.5 Covariance

The covariance matrix generalizes variance to vectors:

$$\Sigma = \text{Cov}[\mathbf{x}] = \mathbb{E}[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^\top].$$

- The diagonal entry Σ_{ii} is the variance of coordinate i (its own uncertainty).

- The off-diagonal Σ_{ij} is the **covariance** between coordinates i and j : positive if they tend to be above/below their means together, negative if oppositely, zero if unrelated.

Geometrically, a multivariate Gaussian's " 1σ " contour is an ellipse (in 2-D) or ellipsoid (higher-D); Σ sets the ellipse's size and orientation. In the EKF, Σ (called P) is literally the tracker's belief about how uncertain it is in position and velocity, and how those uncertainties are coupled.

5.6 Bayes' rule (the engine of estimation)

For events/hypotheses A and data B :

$$p(A | B) = \frac{p(B | A) p(A)}{p(B)}$$

Read it as: **posterior** $\propto$ **likelihood** $\times$ **prior**. Your belief about A after seeing data B ("posterior") equals how well A explains B ("likelihood") times what you believed before ("prior"), renormalized. Tracking is one long application of Bayes' rule: the prior is where the motion model predicts the target is; the likelihood is what the measurement (matcher / detector) says; the posterior is the corrected estimate. Doc 04 turns this exact sentence into the Kalman filter.

6. The exponential moving average (EMA)

This deserves its own section because SiamRAM uses it in at least a dozen places (running confidence, adaptive thresholds, blur reference, camera-motion magnitude, the EKF's tracked box size, ...).

You receive a stream of measurements $z_1, z_2, z_3, \dots$ and want a running estimate of their "recent average" that updates cheaply and forgets the distant past. The EMA does this with one line:

$$\hat{s}_t = (1 - \alpha) \hat{s}_{t-1} + \alpha z_t, \quad \alpha \in (0, 1].$$

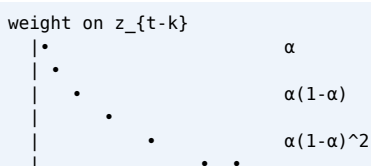
The new estimate is a blend of the old estimate and the new measurement. The single parameter α (the "learning rate" or "smoothing factor") controls memory:

- α near 1: trust the latest measurement, forget quickly (responsive, noisy).
- α near 0: change slowly, heavy smoothing (stable, laggy).

Why it's "exponential." Unrolling the recursion shows each past measurement's weight decays geometrically:

$$\hat{s}_t = \alpha z_t + \alpha(1 - \alpha)z_{t-1} + \alpha(1 - \alpha)^2 z_{t-2} + \dots$$

A measurement k steps ago carries weight $\alpha(1 - \alpha)^k$, which shrinks exponentially with k . The weights sum to 1, so the EMA is a genuine weighted average. Its effective memory length is about $1/\alpha$ samples.



+-----+-----+-----+-----+-----> k (steps into the past)
 0 1 2 3 4 5

A close cousin is the **running confidence with a floor** used by the matcher (doc 03): an EMA of the per-frame score that is never allowed below a floor value, so a hard sequence cannot collapse the tracker’s internal confidence to zero.

7. Geometry helpers: angles, projections, robust averages

7.1 atan2 — recovering an angle

Given a 2-D vector (a, b) , its angle from the positive x -axis is $\theta = \text{atan2}(b, a)$. Unlike plain $\arctan(b/a)$, atan2 gets the **quadrant right** (it knows the signs of a and b separately) and handles $a = 0$. SiamRAM uses it to read the rotation angle out of a camera transform (doc 05): if the top-left 2×2 block of the transform is a scaled rotation $\begin{bmatrix} a & -b \\ b & a \end{bmatrix}$, then the rotation is $\text{atan2}(b, a)$ and the scale is $\sqrt{a^2 + b^2}$.

7.2 Median — a robust average

The **median** of a list is its middle value when sorted. Unlike the mean, a few wild values (“outliers”) barely move it. SiamRAM uses medians where robustness matters: a baseline “typical step size” for spike detection (doc 09), a typical box area for memory admission (doc 07), a representative object size during occlusion (doc 08). Whenever the data may contain a few corrupt entries, the median is the safer summary.

7.3 Weighted average

Given values $v_1, \dots, v_n$ and nonnegative weights $w_1, \dots, w_n$, the **weighted average** is

$$\bar{v} = \frac{\sum_i w_i v_i}{\sum_i w_i}.$$

The EMA (§6) is a weighted average with exponentially decaying weights. SiamRAM also forms an explicit exponentially-weighted average of recent camera displacements, with weights $w_i = e^{(i-n)/(\text{something})}$ that grow toward the present, to estimate “how much is the camera moving lately” (doc 06).

8. Putting the toolkit to work — a preview

Every later document is built from the pieces above. As a sanity check that you have what you need, here is where each tool first becomes essential:

Tool (this doc)	First essential use
Norm / distance (§1.1)	box-center displacement, spike speed (docs 02, 09)
Cosine similarity (§1.3)	appearance matching (docs 03, 07)
Matrix-vector product (§2.1)	constant-velocity model, homography warp (docs 04, 05)
Matrix inverse (§2.3)	Kalman gain (doc 04)
Sigmoid + argmax (§3.1–3.2)	turning the network’s maps into a box (doc 03)
Exponential decay / Gaussian bump (§3.3)	scale penalty, recency & distance penalties (docs 03, 07)
Jacobian + linear approx (§4.2)	the EKF’s camera-warp linearization (doc 05)
Quotient rule (§4.3)	differentiating the perspective division (doc 05)
Gaussian + covariance (§5.3–5.5)	the belief state of the EKF (docs 04, 05)

Tool (this doc)	First essential use
Bayes' rule (§5.6)	the predict/update derivation (doc 04)
EMA (§6)	every adaptive threshold and running statistic (doc 10)
Median / weighted avg (§7)	robust baselines, heavy-motion gating (docs 06, 09)

With the toolkit in hand, turn to **02_BOUNDING_BOXES_AND_IMAGE_GEOMETRY.md**, where we make “a box on an image” precise and set up the coordinate spaces the whole tracker lives in.

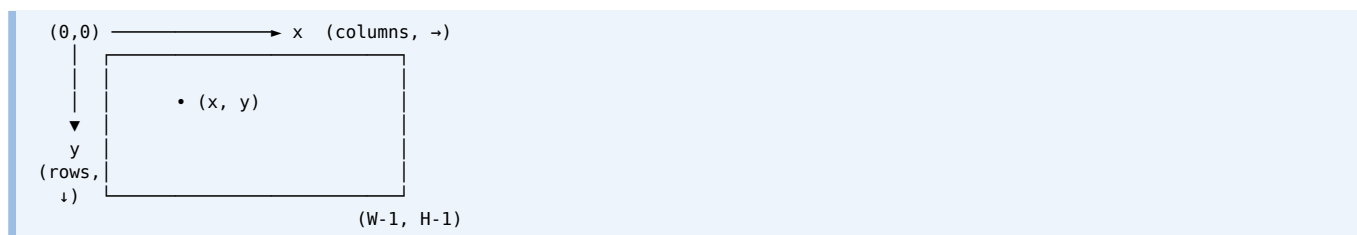
02 — Bounding Boxes & Image Geometry

Prerequisites: doc 01 (norms, the exponential, clamping). This document makes “a box on an image” mathematically precise, defines the overlap measure (IoU) used pervasively, explains how the tracker turns a box into a fixed-size network input (the crop and its context), and lays out the **coordinate spaces** that every later subsystem implicitly lives in. Get this right and the rest of the system stops being mysterious — most of SiamRAM is just careful bookkeeping in these coordinate spaces.

1. The image as a coordinate grid

A digital color image is a 3-D array of numbers with shape $H \times W \times 3$: H rows (height), W columns (width), and 3 color channels. Channel order in this codebase is **BGR** (blue, green, red) because it uses OpenCV; that detail rarely matters for the geometry.

We address a pixel by $(\mathbf{x}, \mathbf{y})$ where $x \in \{0, \dots, W - 1\}$ is the column (horizontal) and $y \in \{0, \dots, H - 1\}$ is the row (vertical). Crucially, in images y **increases downward** and the origin $(0, 0)$ is the **top-left** corner:



This “down-is-positive- y ” convention is just a labeling choice, but it must be consistent everywhere, and it is why a box’s bottom edge has a larger y than its top edge.

2. Two ways to write a box

An **axis-aligned bounding box** is a rectangle with horizontal and vertical sides. SiamRAM uses two equivalent encodings, and converting between them is a constant chore worth internalizing.

(A) Corner-plus-size, $xywh$ — the dominant format in this codebase:

$$b = (x, y, w, h),$$

where (x, y) is the **top-left corner** and w, h are the width and height (both > 0). The box covers columns $[x, x + w)$ and rows $[y, y + h)$.

(B) Two-corner, $xyxy$:

$$b = (x_1, y_1, x_2, y_2),$$

the top-left (x_1, y_1) and bottom-right (x_2, y_2) corners.

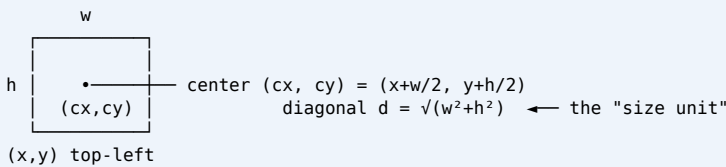
Conversions (memorize these — bugs hide in getting them wrong):

$$\begin{aligned} \text{xywh} \rightarrow \text{xyxy} : \quad x_1 &= x, \quad y_1 = y, \quad x_2 = x + w, \quad y_2 = y + h, \\ \text{xyxy} \rightarrow \text{xywh} : \quad x &= x_1, \quad y = y_1, \quad w = x_2 - x_1, \quad h = y_2 - y_1. \end{aligned}$$

Derived quantities used everywhere:

$$\text{center } (c_x, c_y) = \left(x + \frac{w}{2}, y + \frac{h}{2}\right), \quad \text{area} = w h, \quad \text{diagonal } d = \sqrt{w^2 + h^2}.$$

The **diagonal** d is especially important: it is the natural “size unit” of the box. SiamRAM repeatedly measures displacements in units of the box diagonal so that the same physical event (e.g. “the center moved by half a box”) means the same thing for a tiny target and a huge one. This is called a **scale-normalized** measure, and you’ll see $/d$ all over docs 06–09.



3. Intersection over Union (IoU): the overlap measure

How much do two boxes overlap? The standard answer is **Intersection over Union (IoU)**, also called the Jaccard index. It is used as a gate in many places: admitting a frame to memory (doc 07), checking a re-detection candidate (doc 08), and scoring motion consistency (doc 06).

Take two boxes A and B . Their **intersection** is the overlapping rectangle; its area is

$$\text{inter} = \max(0, \min(x_2^A, x_2^B) - \max(x_1^A, x_1^B)) \times \max(0, \min(y_2^A, y_2^B) - \max(y_1^A, y_1^B)).$$

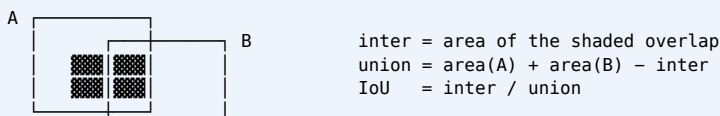
The two $\max(0, \cdot)$ guards are essential: if the boxes don’t overlap, one of the differences is negative, and clamping it to 0 gives zero intersection (rather than a spurious negative “area”). The **union** is the total area both cover, counting the overlap once:

$$\text{union} = \text{area}(A) + \text{area}(B) - \text{inter}.$$

Then

$$\text{IoU}(A, B) = \frac{\text{inter}}{\text{union}} \in [0, 1].$$

$\text{IoU} = 1$ means the boxes are identical; $\text{IoU} = 0$ means they are disjoint. As a guard against the degenerate case where both boxes have zero area ($\text{union} = 0$), the code returns 0.



Why IoU and not center distance? IoU jointly captures position and size agreement: two boxes can have the same center but very different sizes and still get a low IoU. It is also dimensionless (a ratio), so it needs no scale normalization. Its weakness — it is exactly 0 for any pair of non-overlapping boxes, giving no gradient toward “close but not touching” — is why SiamRAM also uses center-distance-in-diagonals in places where it needs to reason about boxes that don’t overlap (e.g. a target that jumped far away).

4. From a box to a network input: crops and context

SiamABC (doc 03) is a neural network. It does not consume the whole frame; it consumes **fixed-size square crops**. Turning a variable-size, variable-shape box into a fixed square crop is a small but important geometric pipeline. Three ideas: context expansion, the equal-context square, and resize + map-back.

4.1 Why context, not a tight crop

If you cropped exactly the target box and showed it to the network, the network would lose all information about the object’s boundary against its surroundings — and that boundary is a strong cue for where the object ends. So the crop always includes a margin of surrounding pixels, called **context**.

The simplest form is a multiplicative expansion. With a context factor ρ (e.g. $\rho \approx 0.5$ means “add 50% of the box size as margin”), `extend_bbox` grows the box outward:

$$\text{expanded box} \approx (x - \rho \frac{w}{2}, y - \rho \frac{h}{2}, (1 + \rho)w, (1 + \rho)h),$$

(the exact per-side arithmetic can be configured, but this is the idea). The expanded box can poke outside the frame; that’s fine — the cropper pads the missing region (§4.4).

4.2 The equal-context square squared_size

Real objects are rectangles, but CNN backbones want **squares** (so the output grid is square and scale is isotropic). We must not stretch a rectangle into a square — that distorts appearance and would change between frames. Instead we compute the side of a square that contains the object plus a consistent amount of context. SiamRAM’s `squared_size` uses the classic SiamFC/SiamRPN rule: pad each dimension by the average half-perimeter and take the geometric mean.

$$\text{pad} = \frac{1}{2}(w + h), \quad \text{side} = \sqrt{(w + \text{pad})(h + \text{pad})}.$$

Let’s sanity-check the design. Substituting $\text{pad} = \frac{1}{2}(w + h)$:

$$\text{side} = \sqrt{\left(w + \frac{w+h}{2}\right)\left(h + \frac{w+h}{2}\right)} = \sqrt{\left(\frac{3w+h}{2}\right)\left(\frac{w+3h}{2}\right)}.$$

For a square object ($w = h$) this gives $\text{side} = 2w$ — the crop is twice the object, i.e. the object occupies the central quarter of the area, with a uniform context ring. For a long thin object the formula still produces a single square side that contains it with proportional context. The key property: **the fraction of the crop occupied by the object is roughly constant** regardless of the object’s aspect ratio, so the network sees a consistent “object-to-context” ratio every frame. That consistency is what lets one trained network handle wildly different object shapes.

4.3 Template vs. search crops

SiamABC takes two kinds of crops, with different context factors (doc 03 uses them):

- **Template crop** — centered on the target, with a small context (a `template_bbox_offset`). It answers “what does the target look like?” Resized to a small square (e.g. 64×64).
- **Search crop** — centered on the predicted location, with a larger context (`search_context`). It answers “where, in this neighborhood, is the target now?” Resized to a larger square (the `instance_size`, e.g. 256×256).

The search crop is bigger (in context) so the target can move between frames and still land inside it. This is the single most important geometric constraint in the whole tracker:

If the target moves outside the search crop, the matcher physically cannot find it. Every motion prior (GMC in doc 06, the Kalman-centered search in doc 06, the EKF-guided occlusion search in doc 08) exists to keep the crop centered where the target actually is.

4.4 Resize, padding, and mapping back

Once the equal-context square is chosen, the cropper (`get_extended_crop`) does three things:

1. **Crop** the square region from the frame. If part of it lies outside the frame (common near edges or during occlusion when the predicted center is near a border), the missing pixels are **padded** with a constant — SiamRAM uses the frame’s mean color, so the padding is visually neutral and doesn’t create a fake edge.
2. **Resize** the (possibly padded) square to the fixed crop size $S \times S$ (e.g. 256×256) the network expects, using interpolation.
3. **Record the mapping** needed to invert step 2 later. The network’s predicted box comes out in crop pixels $[0, S)$; we must convert it back to frame pixels. If the crop came from a region of frame-width W_{ctx} and frame-top-left (x_0, y_0) , then a crop-coordinate x_{crop} maps back to the frame by

$$x_{\text{frame}} = x_0 + x_{\text{crop}} \cdot \frac{W_{\text{ctx}}}{S}, \quad y_{\text{frame}} = y_0 + y_{\text{crop}} \cdot \frac{H_{\text{ctx}}}{S},$$

and a predicted width scales by the same factor W_{ctx}/S . This is exactly what `_rescale_bbox` computes: the scale factor is `padded_box[2] / instance_size` for x /width and `padded_box[3] / instance_size` for y /height, then it shifts by the crop’s top-left. Forgetting this map-back, or getting the scale wrong, is the classic tracker bug where the box drifts or shrinks for no visible reason.

4.5 Preprocessing the pixels

Before the network sees a crop, its pixel values are **normalized**. Each channel is divided by 255 (to bring it into $[0, 1]$) and then standardized with the ImageNet statistics

$$x' = \frac{x/255 - \mu_c}{\sigma_c}, \quad \mu = (0.485, 0.456, 0.406), \quad \sigma = (0.229, 0.224, 0.225),$$

one (μ_c, σ_c) per RGB channel. This is the input distribution the backbone was trained on; matching it at inference is required for the features to be meaningful. It is bookkeeping, but it is load-bearing bookkeeping.

5. The score grid: where the network “looks”

SiamABC’s head does not output a single box; it outputs **dense maps** over a coarse grid (doc 03 explains the maps). If the search crop is $S \times S = 256 \times 256$ pixels and the backbone has total **stride** $r = 16$ (it downsamples by $16\times$), the output grid is

$$G = \frac{S}{r} = \frac{256}{16} = 16,$$

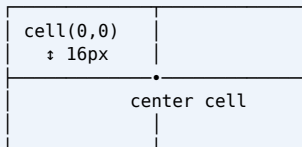
so a 16×16 grid of cells. Each cell corresponds to a 16×16 -pixel patch of the crop. We need a fixed lookup: which crop pixel does grid cell (i, j) correspond to? That is `make_grid`. With cell indices $i, j \in \{0, \dots, G - 1\}$, it places the grid centered on the crop:

$$\text{grid_x}(i, j) = \left(j - \lfloor \frac{G}{2} \rfloor\right) \cdot r + \frac{S}{2}, \quad \text{grid_y}(i, j) = \left(i - \lfloor \frac{G}{2} \rfloor\right) \cdot r + \frac{S}{2}.$$

Read it as: take the cell’s offset from the grid center $\lfloor G/2 \rfloor$, scale by the stride r to convert “cells” into “pixels,” then shift so the grid center sits at the crop center $S/2$. The grid never changes during tracking, so it is computed once and reused — a small but real speedup at 30+ frames per second. Doc 03 uses `grid_x`, `grid_y` to turn the network’s per-cell edge-distances into actual pixel boxes.

16x16 score grid laid over the 256x256 search crop (stride 16):

crop pixel 0 128 (=S/2) 255



$$\begin{aligned} \text{grid_x}(i, j) &= (j - 8) \cdot 16 + 128 \\ \text{grid_y}(i, j) &= (i - 8) \cdot 16 + 128 \end{aligned}$$

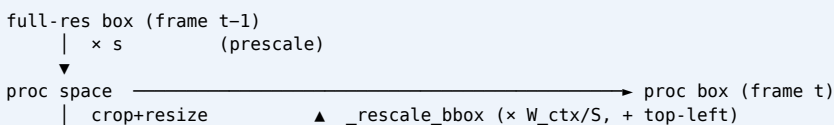
• = crop center maps to grid center

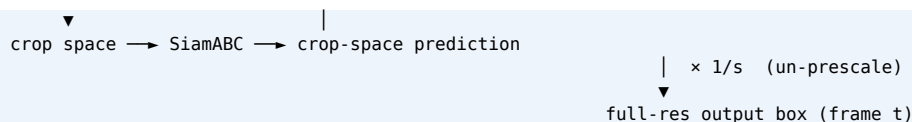
6. The two coordinate spaces of SiamRAM

Finally, the big-picture geometry. There are **three** coordinate frames in play, and every quantity belongs to exactly one of them. Confusing them is the most common source of error in tracker code, so we name them explicitly.

1. **Full-resolution frame space.** The original video pixels, possibly very large (e.g. 1920×1080). The input box on frame 0 and the output box every frame live here.
2. **Proc (processing) space.** At the top of every update, the frame is **prescaled** so its long edge does not exceed a cap (e.g. 1280). Let the scale factor be $s \leq 1$. A full-res point (x, y) becomes a proc point (sx, sy) ; a full-res box scales all four numbers by s . **Every internal subsystem — homography, EKF, SiamABC, YOLO, memory, frame dynamics — works entirely in proc space.** This is deliberate: it bounds the cost of the expensive operations and, more importantly, makes every subsystem speak the same units (a “pixel” means the same thing to all of them). Only at the very end is the chosen box scaled **back** to full-res by $1/s$.
3. **Crop space.** Inside SiamABC, the search crop is its own $S \times S$ coordinate system (§4.4, §5). The network predicts in crop space; `_rescale_bbox` maps the prediction back to proc space.

The data flow of coordinate spaces in one frame:





When later documents say “the center moved 12 pixels,” or “the box diagonal is 80,” they mean **proc pixels**. When they say “scale the box back,” they mean the final $\times 1/s$ step. Holding these three spaces straight makes the entire codebase legible.

7. Recap

- A box is (x, y, w, h) or $(x1, y1, x2, y2)$; center is $(x+w/2, y+h/2)$; the diagonal $\sqrt{w^2 + h^2}$ is the natural size unit, and SiamRAM normalizes displacements by it.
- $\text{IoU} = \frac{\text{intersection}}{\text{union}} \in [0, 1]$ measures position-and-size overlap; it is 0 for disjoint boxes, which is why center-distance-in-diagonals is used as a complement.
- The network consumes fixed square crops with context; `squared_size` makes the object-to-context ratio roughly constant across aspect ratios; the search crop is larger so the target can move and still be found.
- Predictions come out in crop space and must be mapped back with the recorded scale and offset.
- Three coordinate spaces — full-res, proc, crop — and everything internal lives in **proc** space.

Next: **03_SIAMESE_TRACKING_SIAMABC.md**, where these crops become the input to the visual matcher and we follow the math from two images to one box.

03 — Siamese Tracking and the SiamABC Matcher

Prerequisites: doc 01 (dot product, cosine similarity, sigmoid, argmax, exponential) and doc 02 (boxes, crops, the score grid, coordinate map-back). This is the longest core document because the visual matcher is the engine the whole tracker wraps. We go from the idea of “compare two pictures” all the way to “here is a box and a confidence number,” deriving every transformation in between. We will treat the trained convolutional backbone as a known function and explain everything built around it.

1. The Siamese idea: tracking as matching

A **Siamese network** is two (or more) copies of the same network — same architecture, **same weights** — applied to different inputs, whose outputs are then compared. The name is by analogy with Siamese twins: identical, joined.

In tracking, the two inputs are:

- a **template** z — a crop of the target (doc 02 §4.3), “what we are looking for”;
- a **search region** x — a larger crop of the current frame near the last known position, “where we are looking.”

Both are passed through the **same** feature extractor f_θ (the shared backbone), producing feature maps $f_\theta(z)$ and $f_\theta(x)$. We then ask, at every location of the search feature map: how well does the template match here? The answer is a 2-D **response map**; its peak is the predicted location.

```
template z → f_θ → φ_z
search  x → f_θ → φ_x
           (same weights – that's what "Siamese" means)
           |
           |→ compare (correlate) → response map → peak = location
```

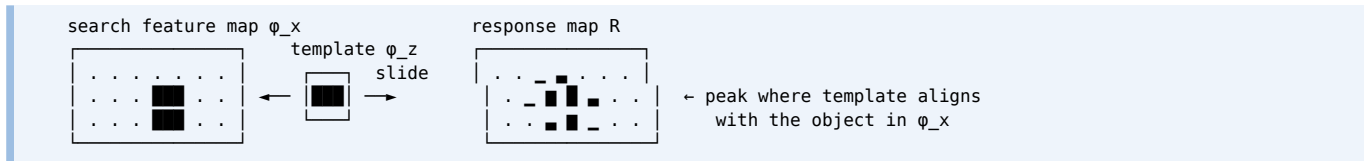
Why share weights? Because we want the same notion of “appearance” applied to both crops, so that “looks like” is a meaningful comparison in a common feature space. Why use a learned f_θ instead of raw pixels? Because raw-pixel matching breaks under lighting, rotation, and small deformation; a network trained on millions of pairs learns features where “the same object” stays close even as it changes.

2. Cross-correlation: the comparison operation

The “compare” step is **cross-correlation**. Take the template feature map φ_z (small, e.g. $C \times 8 \times 8$ channels $\times$ height $\times$ width) and the search feature map φ_x (larger, e.g. $C \times 16 \times 16$). Slide the template over the search map; at each offset, compute the **dot product** of the overlapping features (summed over all C channels):

$$R(u, v) = \sum_c \sum_{i, j} \varphi_z[c, i, j] \varphi_x[c, i + u, j + v].$$

This $R(u, v)$ is the response at offset (u, v) . Recall from doc 01 §1.2 that a dot product is large when two vectors point the same way — so R is large where the search features locally resemble the template features. Cross-correlation is just “the matching score, evaluated at every shift, in one operation.” It is implemented as a convolution where the template plays the role of the filter kernel; on a GPU this is a single fast layer.



SiamABC’s head (the BoxTower/connect_model) is a learned, richer version of this: it cross-correlates the (attention-refined) template against the search features and then runs a few convolutional layers to emit two maps — **classification** and **regression** — described in §5. But the heart is still “correlate the template over the search region.”

3. The dual-template trick (why SiamABC survives change)

A plain Siamese tracker uses one template, cropped from frame 0, forever. That template is a perfect identity anchor (it is, by definition, the true target), but it goes stale: after the object rotates or the lighting shifts, the frame-0 crop no longer matches well.

SiamABC keeps **two** templates:

- **Static template** z — the frame-0 crop. **Never changes.** It is the ground-truth identity anchor, the thing that stops the tracker from slowly drifting onto a similar-looking neighbor.
- **Dynamic template** $\tilde{z}$ — refreshed during tracking from a recent high-confidence frame. It absorbs gradual appearance change (pose, lighting, scale).

Holding both is a deliberate tension: the static template says “this is the real target,” the dynamic one says “this is what it looks like now.” Fusing them gives robustness to change without losing the identity anchor. The “how do we decide when and from which frame to refresh $\tilde{z}$ ” question is the online-update mechanism of §9.

Symmetrically, SiamABC also keeps a **dynamic search** context $\tilde{x}$ from the same recent confident frame, so the network can compare current context against recent context. So the network actually consumes **four** crops per frame: static template z , dynamic template $\tilde{z}$, current search x , dynamic search $\tilde{x}$.

4. Inside the network: backbone, neck, attention, head

We now name the pieces of SiamABCNet and what each does. You do not need to know how the weights were learned; you need to know the shape of the computation.

4.1 Shared backbone f_θ and the neck

Each of the four crops is passed through the **shared backbone** encoder (a lightweight MobileNet-style network for the “S” model, a ResNet for “M”). The backbone is a stack of convolution + normalization + nonlinearity layers that produces a spatial **feature map**: a tensor $C_{\text{enc}} \times H' \times W'$ where each of the $H' \times W'$ spatial positions holds a C_{enc} -dimensional descriptor of the local image content. Spatial resolution is reduced by the backbone’s **stride** (doc 02 §5).

A **convolution** layer, for intuition, slides a small learned filter over the input and computes, at each position, a dot product between the filter and the local patch — the same “local

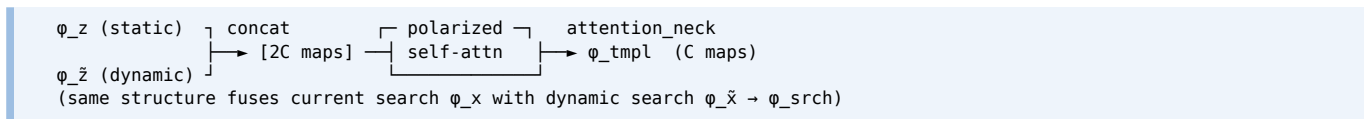
matching” motif as §2, but with learned filters and many of them stacked. Early layers detect edges/textures; deep layers detect object-level patterns. We take that as given.

The **neck** (AdjustLayer) is a 1×1 convolution that projects every position’s C_{enc} -dimensional descriptor down to a common width C (default 256), so all four feature maps share a channel count and can be combined. (A 1×1 convolution is just a learned linear map applied identically at every spatial position — a per-pixel matrix multiply.)

4.2 Polarized self-attention: fusing static + dynamic

The two template feature maps (φ_z static, $\varphi_{\tilde{z}}$ dynamic) are **concatenated along channels** into a $2C$ -channel map and passed through a FastParallelPolarizedSelfAttention block; an attention_neck projects the result back to C channels. The same is done for the two search maps.

You do not need the internal algebra of polarized attention to understand the tracker, but the idea is worth one paragraph because it is the “ABC” (Attention-Based Correlation) in SiamABC. **Attention** is a learned, content-dependent reweighting: the block computes, from the features themselves, a set of weights and uses them to emphasize informative channels and spatial locations and suppress the rest. “Self”-attention means the weights are computed from the same map they reweight. “Polarized” refers to keeping a high-resolution branch in each of the channel and spatial directions (so fine detail is not lost) and combining them. The practical effect: the fused template representation φ_{tmpl} blends the stable identity from z with the current appearance from $\tilde{z}$, weighting whichever is more informative per channel/location. Output shape is unchanged ($C \times H' \times W'$), so the downstream correlation is identical to §2.



4.3 The BoxTower head: two output maps

The fused template φ_{tmpl} is cross-correlated against the fused search φ_{srch} (the §2 operation), and a small stack of convolutions (towernum blocks) turns the result into **two maps** over the $G \times G$ score grid (doc 02 §5, e.g. 16×16):

- **Classification map** $\text{cls} \in \mathbb{R}^{G \times G}$: one raw score (logit) per cell, “is the target centered here?”
- **Regression map** $\text{reg} \in \mathbb{R}^{4 \times G \times G}$: four numbers per cell, describing the box edges (next section).

An optional small **IoU head** predicts, per location, how good the localization is (used as a confidence gate in §8).

5. Anchor-free box decoding (FCOS style)

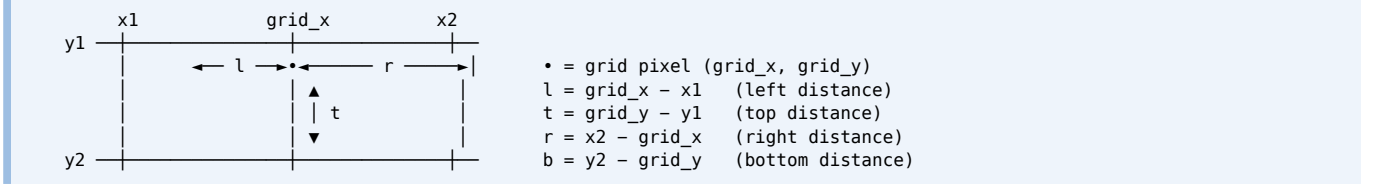
Now the crucial bridge from “maps” to “a box.” SiamABC uses the **FCOS** parameterization: anchor-free, distance-to-edges regression. This is clean and tuning-free, and the encode/decode are exact inverses, so we present both.

5.1 What each cell predicts

Recall (doc 02 §5) that grid cell (i, j) corresponds to a known crop pixel (grid_x, grid_y). The regression map’s four channels at that cell are the **distances from that pixel to the four edges** of the target box:

$$\text{reg}(i, j) = (\ell, t, r, b) = (\underbrace{\text{grid_x} - x_1}_{\text{left}}, \underbrace{\text{grid_y} - y_1}_{\text{top}}, \underbrace{x_2 - \text{grid_x}}_{\text{right}}, \underbrace{y_2 - \text{grid_y}}_{\text{bottom}}).$$

In words: “from me, the left edge is ℓ pixels to the left, the top is t up, the right is r right, the bottom is b down.” A cell is **inside** the box exactly when all four distances are positive. That positivity test is how training labels which cells are “positive” (target) versus “negative” (background).



5.2 Encoding (training targets) — for completeness

Given a ground-truth box (x, y, w, h) , the encoder broadcasts it over the whole grid and computes, per cell,

$$\ell = \text{grid_x} - x, \quad t = \text{grid_y} - y, \quad r = (x + w) - \text{grid_x}, \quad b = (y + h) - \text{grid_y},$$

stacks (ℓ, t, r, b) as the regression target, and sets the classification label to 1 where $\min(\ell, t, r, b) > 0$ (strictly inside) and 0 otherwise. (This is `SiamABCBoxCoder.encode`.) Training nudges the network’s two maps toward these targets; we don’t derive the loss here, but note that some recipes use a soft Gaussian label centered on the box — a 2-D bump $\exp(-d^2/2\sigma^2)$ (doc 01 §3.3) — instead of a hard 0/1 mask, which produces smoother, more peaked score maps and hence a more reliable argmax at decode time.

5.3 Decoding (inference) — turning maps into a box

At inference, `SiamABCBoxCoder.decode` inverts the encoding:

1. **Activate** the classification logits into confidences: $\text{cls_score} = \sigma(\text{cls}) \in (0, 1)^{G \times G}$ (doc 01 §3.1). (After the penalties of §6 it becomes the penalized map used for selection.)
2. **Reconstruct each cell’s box edges** from the grid and the predicted distances:

$$\begin{aligned} x_1(i, j) &= \text{grid_x}(i, j) - \ell(i, j), & x_2(i, j) &= \text{grid_x}(i, j) + r(i, j), \\ y_1(i, j) &= \text{grid_y}(i, j) - t(i, j), & y_2(i, j) &= \text{grid_y}(i, j) + b(i, j). \end{aligned}$$

3. **Pick the winning cell** by argmax over the (penalized) classification map. Flattening the $G \times G$ grid to a 1-D index k , the winner is $k^* = \text{argmax}_k \text{score}_k$, and its row/column are $r_{\max} = \lfloor k^*/G \rfloor$, $c_{\max} = k^* \bmod G$ (doc 01 §3.2).
4. **Read off the box** at the winning cell and convert to xywh:

$$b_{\text{crop}} = (x_1, y_1, x_2 - x_1, y_2 - y_1) \Big|_{(r_{\max}, c_{\max})}.$$

This box is in **crop coordinates**; doc 02 §4.4 maps it back to proc coordinates. The confidence reported for the frame is the score at $(r_{\max}, c_{\max})$ — possibly transformed (§7).

That’s the whole appearance pipeline: four crops $\rightarrow$ features $\rightarrow$ attention fusion $\rightarrow$ correlation $\rightarrow$ two maps $\rightarrow$ penalties $\rightarrow$ argmax $\rightarrow$ one box. Everything else in this document refines which cell wins and how confident we should be in it.

6. Penalties: stopping the tracker from teleporting

A pure argmax over the raw classification map is brittle. Two failure modes:

- A spurious high response near the **edge** of the search crop (clutter, the context ring, a passing object) can beat the true peak.
- The box can suddenly **change size/shape** frame-to-frame in a physically implausible way.

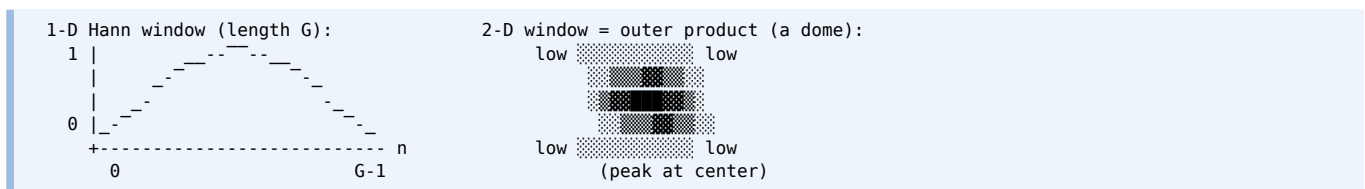
SiamABC multiplies the score map by penalties that encode the priors “the target is probably near the center of the search crop” and “the box probably has a similar size/shape to last frame.” These come from `_confidence_postprocess`.

6.1 The cosine (Hanning) window

The **Hann window** in 1-D of length G is

$$\text{hann}(n) = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{G-1} \right), \quad n = 0, \dots, G-1,$$

a smooth bump that is 0 at the ends and 1 in the middle. The 2-D window is the **outer product** of two 1-D Hann windows, $\text{window}(i, j) = \text{hann}(i) \text{hann}(j)$ — a smooth dome peaking at the grid center and decaying to 0 at the borders.



It is blended into the score map with an **influence** weight $\omega \in [0, 1]$:

$$\text{pscore} = (1 - \omega) \text{pscore} + \omega \text{window}.$$

With $\omega = 0$ the window does nothing; as ω grows, off-center peaks are increasingly suppressed. The prior it encodes is exactly “between consecutive frames the target moves only a little, so it should be near the center of a search crop that was centered on its last position.” This single trick is what keeps Siamese trackers from jumping frame-to-frame.

6.2 The scale/aspect (size) penalty

This penalty demotes candidate boxes whose **size** or **aspect ratio** differ sharply from the previous box. Two ingredients, both built so that any change — growth or shrinkage — is penalized symmetrically.

First, the helper limit, which folds a ratio to be ≥ 1 :

$$\text{limit}(\rho) = \max\left(\rho, \frac{1}{\rho}\right) \geq 1.$$

If $\rho = 2$ (doubled) it returns 2; if $\rho = 0.5$ (halved) it also returns 2. So “twice as big” and “half as big” are treated as equally suspicious — a symmetric penalty curve. (Recall doc 01 §3.5 for clamping intuition; limit is a related symmetry trick.)

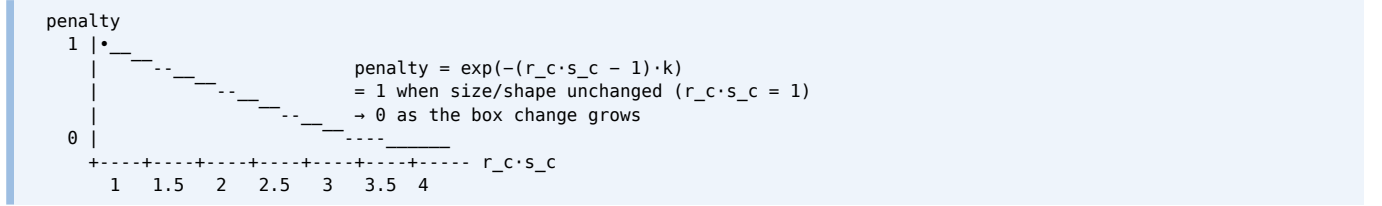
Now define the candidate’s “padded square size” with `squared_size` (doc 02 §4.2) for both the candidate box and the previous box, and form the **size-change** and **aspect-change** ratios:

$$s_c = \text{limit}\left(\frac{\text{squared_size}(w_{\text{cand}}, h_{\text{cand}})}{\text{squared_size}(w_{\text{prev}}, h_{\text{prev}})}\right), \quad r_c = \text{limit}\left(\frac{w_{\text{prev}}/h_{\text{prev}}}{w_{\text{cand}}/h_{\text{cand}}}\right).$$

$s_c \geq 1$ measures overall size change; $r_c \geq 1$ measures aspect-ratio change; both equal 1 for no change. The penalty multiplies the score by

$$\text{penalty} = \exp(-(r_c s_c - 1)k) \quad (0 < \text{penalty} \leq 1),$$

where $k = \text{penalty_k} \geq 0$ is the strength knob. When the candidate matches the previous box, $r_c s_c = 1$, the exponent is 0, and $\text{penalty} = 1$ (no demotion). The more the size/shape changes, the larger $r_c s_c$, the more negative the exponent, the smaller the penalty (toward 0). This is the exponential decay of doc 01 §3.3 used as a soft veto on implausible size jumps. With $k = 0$ the penalty is identically 1 (disabled).



The final selection map is $\text{pscore} = \text{penalty} \odot \text{cls_score}$ (elementwise), then the window of §6.1 is blended in, then argmax (§5.3) selects the winner on this penalized map.

6.3 Size smoothing of the chosen box

Even after selection, the reported box width/height are smoothed toward the previous size so they don't flicker. With a learning rate η modulated by the winning cell's penalty and score,

$$\text{lr} = \text{penalty}[r_{\max}, c_{\max}] \cdot \text{score}[r_{\max}, c_{\max}] \cdot \eta,$$

the new size is an interpolation (doc 01 §3.4) between the previous size and the predicted size. A confident, plausible prediction (high $\text{penalty} \times \text{score}$) moves the size more; an uncertain one barely nudges it. Position is taken from the decoder directly; only size is smoothed.

7. Turning the response into a confidence number

Downstream subsystems (occlusion entry, memory admission, re-acquisition) all threshold a single per-frame **confidence** in $[0, 1]$. SiamABC offers two ways to compute it.

7.1 Peak score (default)

The simplest: the value of the (sigmoid) classification map at the winning cell, $\text{score} = \text{cls_score}[r_{\max}, c_{\max}] \in (0, 1)$. High peak = confident match.

7.2 Peak-to-sidelobe ratio (PSR), optionally

The peak value alone can be misleading: a flat map with a slightly elevated peak is far less trustworthy than a sharp lone spike of the same height. The **peak-to-sidelobe ratio** (from the MOSSE correlation-filter tracker) measures sharpness. Exclude a small square window (half-width ρ) around the peak; call the rest the sidelobe, with mean μ_s and standard deviation σ_s . Then

$$\text{PSR} = \frac{g_{\text{peak}} - \mu_s}{\sigma_s},$$

where g_{peak} is the peak value. PSR is large when the peak stands far above a low, flat sidelobe and small when the map is flat or multi-modal. Since PSR is unbounded (it can be ~ 3 when confused, 40+ when locked on) but the pipeline wants a $[0, 1]$ number, it is squashed monotonically:

$$\text{score} = \frac{\text{PSR}}{\text{PSR} + \kappa} \in [0, 1),$$

with κ a constant (larger κ = stricter). $\text{PSR} \leq 0$ or a degenerate sidelobe maps to 0. Because the output is still in $[0, 1]$, every existing threshold keeps its meaning whether the tracker is in “peak” or “psr” mode.

7.3 The optional IoU-head gate

If the IoU head (§4.3) is active, it predicts a localization-quality value $q \in (0, 1)$ per cell. SiamABC can (a) multiply the confidence by q^p for a power p to fold quality into the score, and/or (b) use q at the peak as a **gate**: if q (smoothed by its own EMA, doc 01 §6) falls below a threshold, the frame’s confidence is penalized (multiplied by 0.25) rather than hard-zeroed — a soft veto, because hard-zeroing caused too many false occlusion entries on noisy IoU maps. This is a small but instructive design point: **soft, reversible penalties are preferred over hard kills** throughout SiamRAM, because a hard kill on a noisy signal is itself a failure mode.

8. Test-Time Adaptation via Adaptive BatchNorm (brief)

Neural networks contain **BatchNorm** layers that standardize activations using running mean/variance statistics learned at training time. If the test video’s lighting/sensor differs, those statistics are slightly off. SiamABC can **adapt** them at test time: an AdaptiveBatchNorm blends the stored statistics with the current crop’s statistics, controlled by a scalar λ (`_tta_lam`). With $\lambda = 0$ it behaves as ordinary frozen BatchNorm (default); setting λ to a small positive value turns adaptation on. This is a lightweight, training-free way to absorb gradual illumination/white-balance drift. It changes the features, not the geometry, so everything above is unaffected.

9. The online dual-template update (RAM-lite)

Section 3 promised a mechanism for refreshing the dynamic template $\tilde{z}$. Here it is. This is a short-term, geometry-and-confidence memory living inside SiamABC — distinct from the appearance memory of doc 07, though related in spirit.

9.1 The memory window

SiamABC keeps a fixed-size deque (`all_memory_imgs`) of the most recent **(frame, box)** pairs, of length M (`memory_window_size`), with their confidence scores in a parallel deque. When a new confident frame arrives it is pushed; the oldest falls off. A running pointer tracks the highest-scoring entry so the best frame can be found cheaply.

9.2 Two admission gates

A frame is admitted to the window only if it passes both:

1. **Score gate.** The score must beat the running confidence EMA, OR clear a hard update threshold. Formally, with running confidence $\bar{c}$ (an EMA, §9.4) and threshold τ_u : admit if $\text{score} > \bar{c}$ or $\text{score} \geq \tau_u$. This keeps low-confidence (possibly wrong) frames out of memory.
2. **IoU gate.** The new box must overlap the previous box: $\text{IoU}(b_t, b_{t-1}) \geq \tau_{\text{iou}}$ (doc 02 §3, default ≈ 0.3). A sudden box jump (low IoU) is refused — it usually signals confusion, and storing it would corrupt the template.

9.3 The blur-admit gate (optional)

A frame that is a transient **motion-blur** smear should not refresh the template, or the smear gets baked in. SiamABC measures crop sharpness with the **variance of the Laplacian**: it resizes the target crop to a fixed 64×64 grayscale patch, applies the Laplacian operator (a discrete second derivative that responds to edges), and takes the variance of the result. A sharp crop has lots of high-frequency edge energy $\rightarrow$ high variance; a blurred crop is low-pass-filtered $\rightarrow$ low variance. The gate is **relative**: the sharpness must clear a fraction (e.g. 0.6) of a running EMA reference of recent confident-frame sharpness. Crucially, the EMA reference is updated even when a frame is vetoed, so a sustained drop in sharpness (the target genuinely became lower-texture) gradually lowers the bar and lets updates resume — only a one-off spike is rejected. (Using a fixed absolute threshold would fail because sharpness depends on content and scale; a relative, self-adjusting bar is the correct design.)

9.4 Selecting the dynamic template, on a cadence

Every N frames (`select_representatives`), the tracker scans the window from newest to oldest for the first entry whose score clears the update threshold τ_u , and refreshes $\tilde{z}$ (and $\tilde{x}$) from that frame. Choosing the newest qualifying frame keeps the dynamic template as current as possible while still requiring it to be confident. If no stored frame clears τ_u , the previous $\tilde{z}$ is kept. (The cadence N and window M can themselves be adapted to motion — doc 10.)

9.5 Running confidence with a floor

The **running confidence** is an EMA of the per-frame score (doc 01 §6),

$$\bar{c} \leftarrow (1 - \lambda) \bar{c} + \lambda \text{score},$$

clamped below by a floor value:

$$\bar{c} \leftarrow \max(\bar{c}, \bar{c}_{\min}).$$

The EMA gives a sequence-adaptive sense of “how confident am I usually,” used as the dynamic admission bar in §9.2. The floor prevents a hard run from driving $\bar{c}$ to 0, which would let any frame into memory and collapse the tracker — a guardrail, again of the “never let an adaptive quantity run away” kind that recurs in doc 10.

10. The per-frame loop, assembled

Putting §§4–9 together, here is one SiamABC frame:

1. extract search crop x around the previous (or motion-predicted) box; preprocess	[doc 02 §4]
2. backbone+neck $\rightarrow \phi_x$; $\phi_z, \phi_{\tilde{z}}, \phi_{\tilde{x}}$ are cached (templates change only on cadence)	[§4.1]
3. attention-fuse $(\phi_z, \phi_{\tilde{z}}) \rightarrow \phi_{\text{tmpl}}$ and $(\phi_x, \phi_{\tilde{x}}) \rightarrow \phi_{\text{srch}}$	[§4.2]
4. BoxTower: correlate $\rightarrow$ classification map (logits) + regression map (l, t, r, b)	[§4.3, §5]
5. $\text{cls_score} = \sigma(\text{cls})$; apply size penalty $\times$ window $\rightarrow$ penalized selection map	[§6]
6. (optional) Kalman-motion fusion reweights the map	[doc 06]
7. $\text{argmax} \rightarrow$ winning cell $\rightarrow$ reconstruct box edges $\rightarrow$ xywh in crop coords	[§5.3]
8. rescale box to proc coords; clamp to frame	[doc 02 §4.4]
9. confidence = peak (or PSR), optionally IoU-gated	[§7]
10. admit frame to memory if score+IoU(+blur) gates pass; every N frames refresh $\tilde{z}$	[§9]
11. update running confidence EMA (with floor)	[§9.5]
$\rightarrow$ returns (box, confidence) to the SiamRAM conductor	[doc 11]

Steps 1–9 are “find the target this frame”; steps 10–11 are “keep the template fresh for next time.” The output (box, confidence) is exactly what the outer SiamRAM layer consumes: the box is a candidate position, and the confidence is the single number the motion filter, memory, and occlusion logic all reason about.

11. Recap and limitations

- A Siamese tracker compares a **template** to a **search region** through a shared learned feature extractor; the comparison is **cross-correlation**, whose peak is the location.
- SiamABC fuses a **static** (identity anchor) and a **dynamic** (current appearance) template via polarized self-attention, then decodes a box with the **FCOS** distance-to-edges parameterization — anchor-free, argmax-selected.
- A **cosine window** and a **scale/aspect penalty** ($\exp(-(r_c s_c - 1)k)$) encode the priors “near center” and “similar size,” preventing frame-to-frame teleports.
- Confidence is the **peak** (or a squashed **PSR** for sharpness-awareness), optionally gated by a learned IoU head — always with soft penalties over hard kills.
- An online **memory window** with score/IoU/blur gates refreshes the dynamic template on a cadence, and a floored **running-confidence EMA** adapts the admission bar.

What SiamABC cannot do alone: it has no physics (it doesn’t know the target’s velocity), no long-term identity memory (only a short window), no notion of camera motion, and no way to recover once the target leaves its search crop. Those are exactly the jobs of the outer SiamRAM layer. The very next thing it needs is a **motion model**, so the search crop can be centered where the target is going, not where it was. That is doc 04 (the theory of filtering) and doc 05 (the EKF + camera motion).

04 — Probability, Bayesian Filtering, and the Kalman Filter

Prerequisites: doc 01 (vectors, matrices, inverse, Gaussian, covariance, Bayes' rule). This document derives the **Kalman filter** from first principles and then the **Extended Kalman Filter (EKF)** that SiamRAM actually uses. We build it slowly: the estimation problem, the state-space model, the recursive Bayesian idea, the 1-D Kalman filter (with full algebra), the vector Kalman filter, and finally linearization for nonlinear models. Doc 05 then plugs SiamRAM's specific motion + camera model into these equations. Nothing here is specific to SiamRAM yet — it is the theory you must own first.

1. The problem: estimating a hidden state from noisy data

Imagine you want to know a target's true position and velocity at each frame. You cannot observe them directly. What you can observe is the matcher's reported box — a noisy, indirect measurement of position (and not of velocity at all). You also know something about physics: between two frames, a moving object roughly continues at its current velocity.

This is the universal **filtering** problem:

- There is a **hidden state** $\mathbf{x}_t$ (here: center position and velocity) that evolves over time.
- You receive **measurements** $\mathbf{z}_t$ that depend on the state but are corrupted by noise.
- You want the **best estimate** of $\mathbf{x}_t$ given all measurements so far, $\mathbf{z}_1, \dots, \mathbf{z}_t$ — computed **recursively** (update the estimate as each new measurement arrives, without re-processing the whole history).

“Recursively” is essential for a real-time tracker: at 30+ fps you cannot reprocess thousands of past frames each step. You want to carry a compact summary of the past and fold in one new measurement per frame in constant time. The Kalman filter is the optimal way to do exactly that — when the model is linear and the noise is Gaussian.

2. The state-space model

We formalize “physics” and “measurement” as two equations.

2.1 The process (motion) model

The state evolves by a linear map plus random disturbance:

$$\mathbf{x}_t = F \mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, Q).$$

- $\mathbf{x}_t \in \mathbb{R}^n$ is the state at time t .
- F (the **state-transition matrix**, $n \times n$) encodes the deterministic dynamics — e.g. “position next = position + velocity.”
- $\mathbf{w}_t$ is **process noise**: a zero-mean Gaussian capturing everything the linear model misses (the target accelerates, turns, etc.). Its covariance Q says how much we distrust the motion model. Big Q = “the target can deviate a lot from constant velocity.”

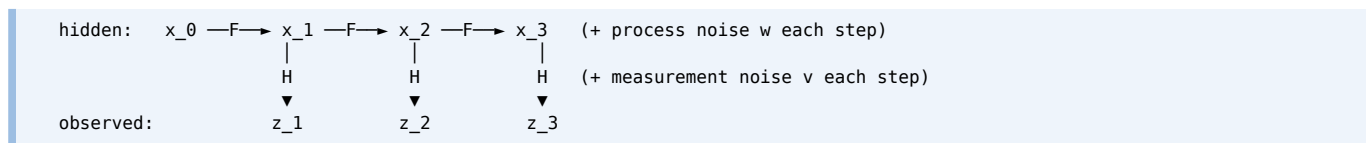
2.2 The measurement model

The measurement is a linear function of the state plus noise:

$$\mathbf{z}_t = H \mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, R).$$

- $\mathbf{z}_t \in \mathbb{R}^m$ is the measurement at time t .
- H (the **measurement matrix**, $m \times n$) maps state to measurement — e.g. “we measure position but not velocity,” so H picks out the position coordinates.
- $\mathbf{v}_t$ is **measurement noise** with covariance R . Big R = “the measurement is unreliable.”

The two noises are assumed independent and white (uncorrelated across time). This pair of linear-Gaussian equations is the **linear-Gaussian state-space model**, and it is the exact setting in which the Kalman filter is optimal.



2.3 The belief: a Gaussian over the state

Our knowledge of the state at any time is itself a **Gaussian** (doc 01 §5.4):

$$\mathbf{x}_t \mid \mathbf{z}_{1:t} \sim \mathcal{N}(\hat{\mathbf{x}}_t, P_t),$$

with mean $\hat{\mathbf{x}}_t$ (our point estimate) and covariance P_t (our uncertainty). The Kalman filter is just the rule for updating this $(\hat{\mathbf{x}}, P)$ pair as time advances and measurements arrive. Because the model is linear-Gaussian, the belief stays Gaussian forever (doc 01 §5.3, fact 2), so $(\hat{\mathbf{x}}, P)$ is a complete summary — that’s why the filter is recursive and cheap.

3. The recursive Bayesian recipe (two half-steps)

Each time step splits into **predict** (advance the belief through the motion model) and **update** (fold in the new measurement via Bayes’ rule). This predict-update rhythm is the skeleton of every filter.

Predict (a.k.a. time update). Push the previous belief through the motion model. Intuitively, applying F shifts the mean and the random kick $\mathbf{w}$ spreads the uncertainty:

$$\hat{\mathbf{x}}_t^- = F \hat{\mathbf{x}}_{t-1}^+, \quad P_t^- = F P_{t-1}^+ F^\top + Q.$$

The superscript $-$ means “before this frame’s measurement” (the prior), $+$ means “after” (the posterior). Notice prediction increases uncertainty (it adds Q and inflates P by $F \cdots F^\top$): with no new data, we become less sure.

Update (a.k.a. measurement update). Use Bayes’ rule (doc 01 §5.6) to combine the prior $\mathcal{N}(\hat{\mathbf{x}}_t^-, P_t^-)$ with the measurement likelihood $\mathcal{N}(\mathbf{z}_t; H\mathbf{x}, R)$. Because both are Gaussian, the posterior is Gaussian, and prediction decreases uncertainty (data makes us surer). The exact formulas are the Kalman update, derived next.

Let’s first do the whole thing in **one dimension** so the algebra is fully visible, then state the vector version.

4. The Kalman filter in 1-D (full derivation)

Strip everything to scalars to see the mechanism. State x (say, position), motion $x_t = x_{t-1} + w$ with $w \sim \mathcal{N}(0, q)$, measurement $z = x + v$ with $v \sim \mathcal{N}(0, r)$.

4.1 Predict

If before the step we believe $x \sim \mathcal{N}(\hat{x}^+, p^+)$, then after applying $x_t = x_{t-1} + w$ (here $F = 1$):

$$\hat{x}^- = \hat{x}^+, \quad p^- = p^+ + q.$$

Mean unchanged (constant model), variance grows by the process noise q .

4.2 Update — combining two Gaussians

Now a measurement z arrives. We have two pieces of information about x :

- the **prior** $\mathcal{N}(\hat{x}^-, p^-)$ from prediction;
- the **likelihood** from the measurement: given $x, z \sim \mathcal{N}(x, r)$, i.e. as a function of x it is $\mathcal{N}(z, r)$.

By Bayes' rule the posterior is proportional to the **product** of the two Gaussians. Multiply the two exponentials and complete the square (this is the one derivation worth seeing once). The product of $\mathcal{N}(\mu_1, \sigma_1^2)$ and $\mathcal{N}(\mu_2, \sigma_2^2)$ (as functions of x) is proportional to a Gaussian with

$$\frac{1}{\sigma^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}, \quad \mu = \sigma^2 \left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2} \right).$$

This is the famous rule: **precisions (inverse variances) add**, and the combined mean is the precision-weighted average of the two means. Plug in $\mu_1 = \hat{x}^-$, $\sigma_1^2 = p^-$ and $\mu_2 = z$, $\sigma_2^2 = r$:

$$\frac{1}{p^+} = \frac{1}{p^-} + \frac{1}{r}, \quad \hat{x}^+ = p^+ \left(\frac{\hat{x}^-}{p^-} + \frac{z}{r} \right).$$

4.3 Rewriting it as “predict + gain $\times$ surprise”

The form above is correct but opaque. Algebra turns it into the canonical Kalman form. Define the **Kalman gain**

$$K = \frac{p^-}{p^- + r}.$$

Then (you can verify by substitution) the update becomes

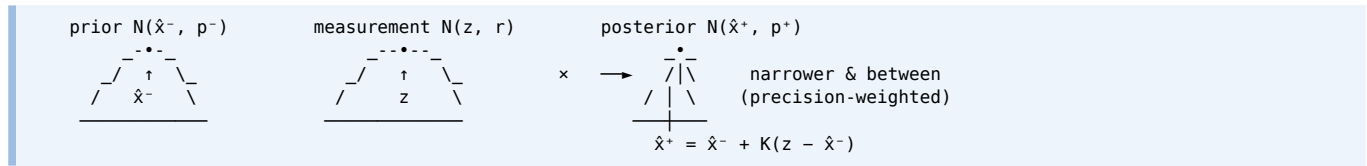
$$\boxed{\hat{x}^+ = \hat{x}^- + K(z - \hat{x}^-), \quad p^+ = (1 - K)p^-}.$$

Read this carefully — it is the soul of the Kalman filter:

- $z - \hat{x}^-$ is the **innovation** (a.k.a. residual or surprise): how much the measurement disagrees with the prediction.
- K decides **how much of the surprise to believe**. Look at the two extremes:
 - If the measurement is very reliable ($r \rightarrow 0$), then $K \rightarrow 1$: trust the measurement fully, $\hat{x}^+ \rightarrow z$.
 - If the measurement is useless ($r \rightarrow \infty$), then $K \rightarrow 0$: ignore it, $\hat{x}^+ \rightarrow \hat{x}^-$.
 - K also grows when the prior is uncertain (p^- large): if we don't trust our own prediction, we lean on the data.

- The variance always **shrinks** on update ($p^+ = (1 - K)p^- \leq p^-$): a measurement, however noisy, can only reduce uncertainty.

So the Kalman filter is a self-tuning weighted average between prediction and measurement, where the weight is set, optimally, by their relative uncertainties. That sentence is the entire intuition; everything else is bookkeeping to make it work for vectors.



5. The vector Kalman filter

The 1-D logic lifts verbatim to vectors; scalars become matrices, division becomes matrix inverse. With state $\mathbf{x} \in \mathbb{R}^n$, measurement $\mathbf{z} \in \mathbb{R}^m$, and the model of §2:

Predict:

$$\hat{\mathbf{x}}_t^- = F \hat{\mathbf{x}}_{t-1}^+, \quad P_t^- = F P_{t-1}^+ F^\top + Q.$$

Update:

$$\begin{aligned} \mathbf{y}_t &= \mathbf{z}_t - H \hat{\mathbf{x}}_t^- && \text{(innovation: measurement - predicted measurement)} \\ S_t &= H P_t^- H^\top + R && \text{(innovation covariance: how surprising can the surprise be?)} \\ K_t &= P_t^- H^\top S_t^{-1} && \text{(Kalman gain: optimal trust in the surprise)} \\ \hat{\mathbf{x}}_t^+ &= \hat{\mathbf{x}}_t^- + K_t \mathbf{y}_t && \text{(corrected state estimate)} \\ P_t^+ &= (I - K_t H) P_t^- && \text{(corrected, shrunk covariance)} \end{aligned}$$

Match each line to the 1-D version: $\mathbf{y}_t$ is $z - \hat{x}^-$; S_t generalizes $p^- + r$ (the total uncertainty in the innovation, mapped through H plus measurement noise R); $K_t = P^- H^\top S^{-1}$ generalizes $\frac{p^-}{p^- + r}$ (note the S^{-1} — that matrix inverse is the vector analogue of dividing by $p^- + r$); $\hat{\mathbf{x}}^+ = \hat{\mathbf{x}}^- + K\mathbf{y}$ is identical in form; $P^+ = (I - KH)P^-$ generalizes $(1 - K)p^-$.

Why $P^- H^\top$ in the gain? $P^- H^\top$ is the **cross-covariance between state error and innovation** — it says which state coordinates the innovation is informative about. This is how a measurement of position only still corrects the velocity estimate: if position and velocity are correlated in P^- (and after a predict step they are, because velocity drove position), then a position surprise updates velocity too, through the off-diagonal of $P^- H^\top$. That coupling is the magic that lets a Kalman filter infer an unmeasured quantity (velocity) from a measured one (position).

5.1 Tuning Q and R

The filter's behavior is governed entirely by the noise covariances:

- R **large** (untrusted measurements) $\rightarrow$ small gain $\rightarrow$ the filter smooths heavily, leaning on its motion model. Good against noisy detections; bad if the model is wrong.
- Q **large** (untrusted model) $\rightarrow P^-$ inflates $\rightarrow$ large gain $\rightarrow$ the filter follows measurements closely. Good for erratic motion; bad against noisy detections.

The ratio Q/R is the single most important tuning quantity: it sets where the filter sits on the smoothing-vs-responsiveness spectrum. SiamRAM picks these by hand for its tracker (doc

05) — e.g. velocity gets more process noise than position, encoding “the object’s speed can change more abruptly than the model assumes.”

6. The constant-velocity model (what SiamRAM uses)

The motion model SiamRAM’s filters use is **constant velocity (CV)**: “between frames, position advances by velocity, and velocity stays the same (up to noise).” For a 1-D position c with velocity v , the state is $\mathbf{x} = (c, v)$ and one frame ($\Delta t = 1$) gives

$$c_t = c_{t-1} + v_{t-1}, \quad v_t = v_{t-1},$$

which in matrix form is $\mathbf{x}_t = F\mathbf{x}_{t-1}$ with

$$F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Check: $F \begin{bmatrix} c \\ v \end{bmatrix} = \begin{bmatrix} c + v \\ v \end{bmatrix}$. ✓ The “1” in the top-right couples velocity into next position; that off-diagonal is what makes velocity observable from position measurements (§5).

For 2-D tracking the state is (c_x, c_y, v_x, v_y) and F is the 4×4 block version: each position gets its velocity added, each velocity persists. We measure position only, so

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad H(c_x, c_y, v_x, v_y)^\top = (c_x, c_y)^\top.$$

This is exactly the structure of SiamRAM’s BBoxEKF (doc 05): a 4-D state, a 2×4 measurement matrix that reads off the center, and constant-velocity F — except that the prediction also folds in camera motion, which makes it nonlinear and forces the “Extended” upgrade we cover now.

7. When the model is nonlinear: the Extended Kalman Filter

The Kalman filter assumed linear F and H . But SiamRAM’s prediction warps the target’s position through a **camera homography** (doc 05), and a homography involves a division (perspective), which is **not linear**. The same happens in countless applications (radar with range/bearing, GPS, robotics). The fix is the **Extended Kalman Filter (EKF)**: keep the exact nonlinear functions for propagating the mean, but **linearize** them (doc 01 §4.2, the Jacobian) for propagating the covariance.

7.1 Nonlinear state-space model

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}) + \mathbf{w}_t, \quad \mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{v}_t,$$

with f and h now general (possibly nonlinear) functions.

7.2 The trick: linearize about the current estimate

Near our current estimate $\hat{\mathbf{x}}$, replace f by its tangent (first-order Taylor expansion, doc 01 §4.2):

$$f(\mathbf{x}) \approx f(\hat{\mathbf{x}}) + F(\mathbf{x} - \hat{\mathbf{x}}), \quad F = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}} \quad (\text{the Jacobian of } f).$$

Likewise $H = \partial h / \partial \mathbf{x}$ at the predicted state. The Jacobian F is the local linear stand-in for the curved map: it tells the covariance how the uncertainty ellipse gets rotated/stretched by the nonlinearity. The EKF then runs the linear Kalman equations with these locally computed F and H :

Predict (EKF):

$$\hat{\mathbf{x}}_t^- = f(\hat{\mathbf{x}}_{t-1}^+), \quad P_t^- = F P_{t-1}^+ F^\top + Q.$$

Note the **mean uses the exact** f (no approximation), while the **covariance uses the Jacobian** F (the linear approximation). That split is the entire idea of the EKF.

Update (EKF): identical to §5 but with the innovation using the exact h ,

$$\mathbf{y}_t = \mathbf{z}_t - h(\hat{\mathbf{x}}_t^-), \quad S_t = H P_t^- H^\top + R, \quad K_t = P_t^- H^\top S_t^{-1}, \quad \hat{\mathbf{x}}_t^+ = \hat{\mathbf{x}}_t^- + K_t \mathbf{y}_t, \quad P_t^+ = (I - K_t H) P_t^-.$$

In SiamRAM the measurement model is already linear (we measure the center, $h(\mathbf{x}) = H\mathbf{x}$ with the constant 2×4 matrix above), so only the **prediction** is nonlinear (because of the camera warp). That means SiamRAM only needs the Jacobian of the prediction function — and doc 05 derives exactly that Jacobian, term by term, including the quotient-rule step (doc 01 §4.3) for the perspective division.

7.3 Caveats of the EKF

The linearization is an approximation, exact only to first order. It can be inaccurate if the function is strongly curved over the uncertainty region, and the filter can diverge if initialized badly or if Q, R are mis-set. For SiamRAM's mild per-frame camera motions the linearization is excellent (the warp is nearly affine over one box), and the engineering guards (covariance resets, "reseed," plausibility gates in later docs) keep it stable. These are normal, expected parts of using an EKF in practice.

8. The filtering algorithm in one box

For reference, the full recursive loop you now understand:

```
initialize  $\hat{\mathbf{x}}_0^+, P_0^+$ 
for each frame  $t = 1, 2, 3, \dots$ :
  — PREDICT —
   $\hat{\mathbf{x}}^- = f(\hat{\mathbf{x}}^+)$  (exact nonlinear move; =  $F \hat{\mathbf{x}}^+$  if linear)
   $F = \partial f / \partial \mathbf{x} |_{\hat{\mathbf{x}}^+}$  (Jacobian; =  $F$  itself if linear)
   $P^- = F P^+ F^\top + Q$  (uncertainty grows)
  — UPDATE (only if a measurement  $\mathbf{z}_t$  arrived) —
   $\mathbf{y} = \mathbf{z}_t - h(\hat{\mathbf{x}}^-)$  (innovation / surprise)
   $S = H P^- H^\top + R$  (innovation covariance)
   $K = P^- H^\top S^{-1}$  (Kalman gain)
   $\hat{\mathbf{x}}^+ = \hat{\mathbf{x}}^- + K \mathbf{y}$  (corrected estimate)
   $P^+ = (I - K H) P^-$  (uncertainty shrinks)
  — (if no measurement:  $\hat{\mathbf{x}}^+ = \hat{\mathbf{x}}^-$ ,  $P^+ = P^-$  — the filter "coasts") —
```

The "coast" case in the last line is exactly what an occluded tracker does (doc 08): when the target is hidden there is no trustworthy measurement, so the filter keeps predicting (and its uncertainty keeps growing), extrapolating where the target probably went until it is re-found.

9. Recap

- Filtering = recursively estimating a hidden, evolving state from noisy measurements, carrying a Gaussian belief $(\hat{\mathbf{x}}, P)$.

- A **state-space model** has a linear **process** ($\mathbf{x}_t = F\mathbf{x}_{t-1} + \mathbf{w}$) and a linear **measurement** ($\mathbf{z}_t = H\mathbf{x}_t + \mathbf{v}$), with Gaussian noises Q, R .
- The **Kalman filter** is a predict (mean shifts, covariance grows by Q) / update (Bayes' product of Gaussians) loop. Its heart is $\hat{\mathbf{x}}^+ = \hat{\mathbf{x}}^- + K(\mathbf{z} - H\hat{\mathbf{x}}^-)$, a precision-weighted blend of prediction and measurement.
- The **Kalman gain** $K = P^-H^\top S^{-1}$ optimally weights measurement against model; off-diagonals of P let a position measurement correct an unmeasured velocity.
- Q/R tunes smoothing vs. responsiveness.
- The **constant-velocity** model (F adds velocity to position, H reads position) is what SiamRAM tracks.
- The **EKF** handles nonlinear f (SiamRAM's camera warp) by using the exact f for the mean and its **Jacobian** for the covariance.

You now have everything to read the centerpiece. Next: **05_HOMOGRAPHY_AND_EKF.md** — what a homography is, how it's estimated from optical flow and RANSAC, and how SiamRAM's BBoxEKF folds it into the prediction step, Jacobian and all.

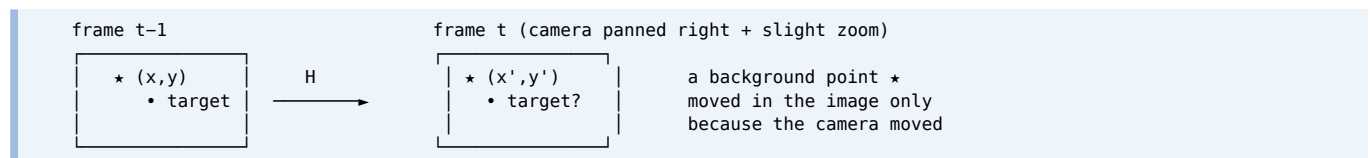
05 — Homography Estimation and the Camera-Aware EKF

Prerequisites: doc 01 (matrices, Jacobian, quotient rule, atan2), doc 02 (boxes, coordinate spaces), doc 04 (the Kalman filter and the EKF). **This is the center-piece.** We build, from scratch: homogeneous coordinates and what a homography is; how SiamRAM estimates the frame-to-frame camera motion from optical flow + RANSAC; how it decomposes and sanity-checks that motion; and finally how its BBoxEKF folds the camera motion into the prediction step — including a line-by-line derivation of the Jacobian the code computes. By the end you should be able to read `models/siamram/motion.py` and the homography functions in `tracker.py` and understand every line.

Part A — Projective geometry and the homography

A.1 The motivating picture

When the camera pans, tilts, rotates, or zooms between two frames, the entire image shifts in a structured way. A static background point at pixel (x, y) in frame $t - 1$ appears at some new pixel (x', y') in frame t . If we can model that map $(x, y) \mapsto (x', y')$ for the background, we can predict where a stationary target would appear after the camera moved — and, more importantly, separate camera-induced image motion from the target's own motion. That map, for a moving camera viewing a roughly planar or distant scene, is a **homography**.



A.2 Homogeneous coordinates

To write perspective maps as matrix multiplications, we use **homogeneous coordinates**. A 2-D point (x, y) is represented by a 3-vector

$$\tilde{\mathbf{p}} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

and — this is the key rule — **any nonzero scalar multiple represents the same 2-D point**:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \lambda x \\ \lambda y \\ \lambda \end{bmatrix} \quad (\lambda \neq 0).$$

To go back to ordinary (“inhomogeneous”) coordinates from a general homogeneous vector (a, b, c) , you **divide by the third coordinate**:

$$(a, b, c) \longrightarrow \left(\frac{a}{c}, \frac{b}{c} \right).$$

That division is the **perspective division** — and it is the nonlinearity at the heart of this whole document. Why bother? Because in this representation, every projective transformation (translation, rotation, scale, shear, and true perspective foreshortening) becomes a single 3×3 matrix multiply. Without homogeneous coordinates, translation can't be written as a matrix multiply at all; with them, everything can.

A.3 The homography matrix

A **homography** is a 3×3 matrix H acting on homogeneous points:

$$\tilde{\mathbf{p}}' = H \tilde{\mathbf{p}}, \quad H = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix}.$$

Writing it out and applying the perspective division (A.2), a point (x, y) maps to

$$x' = \frac{h_{00}x + h_{01}y + h_{02}}{h_{20}x + h_{21}y + h_{22}}, \quad y' = \frac{h_{10}x + h_{11}y + h_{12}}{h_{20}x + h_{21}y + h_{22}}.$$

Stare at this — these two formulas are literally the lines in `BBoxEKF.predict` and `_h_translation_magnitude`. Reading off the structure:

- h_{02}, h_{12} are **translation** (they shift x', y').
- The top-left 2×2 block $\begin{bmatrix} h_{00} & h_{01} \\ h_{10} & h_{11} \end{bmatrix}$ does **rotation, scale, and shear**.
- The bottom row h_{20}, h_{21} produces **perspective**: when nonzero, the denominator depends on position, so the map is not affine — far parts of the image transform differently from near parts. This is exactly what a tilting camera does.
- H is defined **only up to scale** (multiplying all nine entries by the same constant gives the same map, by A.2), so it has 8 degrees of freedom, not 9. Conventionally $h_{22} \approx 1$.

Special case — affine. If the bottom row is $(0, 0, 1)$, the denominator is constant = 1 and the map becomes linear-plus-translation:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} \\ h_{10} & h_{11} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} h_{02} \\ h_{12} \end{bmatrix}.$$

This is an **affine** transform (6 DOF). SiamRAM's fast "classic" estimator (Part B) produces an even more restricted form — a similarity transform: rotation + uniform scale + translation (4 DOF) — and stores it in the top two rows of a 3×3 H with bottom row $(0, 0, 1)$. The "accurate" estimator produces a full 8-DOF homography.

A.4 Decomposing H into interpretable numbers

SiamRAM never uses the raw H for thresholds; it first extracts human-meaningful motion statistics (`_gmc_motion_stats`). Given H (treating the relevant block as a scaled rotation

$\begin{bmatrix} a & * \\ b & * \end{bmatrix}$ with $a = h_{00}$, $b = h_{10}$):

$$\underbrace{dx = h_{02}, \quad dy = h_{12}}_{\text{translation (px)}}, \quad \underbrace{\text{scale} = \sqrt{a^2 + b^2}}_{\text{zoom factor}}, \quad \underbrace{\theta = \text{atan2}(b, a)}_{\text{rotation (rad)}}.$$

The scale and rotation come from recognizing that a rotation-by- θ -and-scale-by- σ matrix is $\sigma \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} a & -b \\ b & a \end{bmatrix}$, so $a = \sigma \cos \theta$, $b = \sigma \sin \theta$, giving $\sigma = \sqrt{a^2 + b^2}$ and $\theta =$

$\text{atan2}(b, a)$ (doc 01 §7.1). The code reports θ in degrees.

It also computes the **maximum corner displacement**: warp the four frame corners through H and take the largest distance moved,

$$\text{max_corner_disp} = \max_{k \in \text{corners}} \|H \cdot \tilde{\mathbf{c}}_k|_{\div \text{persp}} - \mathbf{c}_k\|.$$

This single scalar captures the worst-case image motion implied by H (including perspective), which is a robust summary for the plausibility gates (Part D).

Part B — Estimating the homography from two frames

We have the model; now, how do we find H from two consecutive grayscale frames? The pipeline is: pick points $\rightarrow$ track them with optical flow $\rightarrow$ robustly fit H to the agreeing points $\rightarrow$ flag reliability. Crucially, we estimate from **background** points, masking out the target.

B.1 Why mask the target

We want the camera/background motion, not the target's motion. The target generally moves differently from the background; if it dominated the point set, the fit would learn target motion and the whole compensation would be backwards. So both estimators **exclude points on or near the current target box** (padding the box by $\sim 15\text{--}20\%$ of its size). The fast path simply drops grid points inside the padded box; the accurate path builds a mask image with the box region zeroed so the feature detector won't place points there.

B.2 Working at reduced resolution

Estimation runs on a **downscaled** grayscale image (scale $s = \text{flow_scale}$, e.g. 0.5) for speed. Everything tracked is in downscaled pixels; the resulting transform must be converted back to proc-resolution. For the classic (affine) path the conversion is: keep the linear 2×2 block (a ratio, scale-invariant) and **divide the translation by s** . Why? If a proc point $\mathbf{p}$ becomes a small-image point $s\mathbf{p}$, and the affine map on the small image is $A_{\text{lin}}(s\mathbf{p}) + \mathbf{t}_{\text{small}}$, then converting the result back to proc by dividing by s gives $A_{\text{lin}}\mathbf{p} + \mathbf{t}_{\text{small}}/s$ — the linear part is unchanged and the translation scales by $1/s$. That is exactly $H[0,2] \neq \text{scale}$; $H[1,2] \neq \text{scale}$. (The accurate path applies the equivalent full conjugation $H_{\text{proc}} = D^{-1}H_{\text{small}}D$ with $D = \text{diag}(s, s, 1)$.)

B.3 Optical flow: tracking a point from one frame to the next

Optical flow estimates, for a point in frame $t - 1$, where it went in frame t . SiamRAM uses the **Lucas-Kanade** method. Here is the full idea — it is beautiful and only needs a derivative and a least-squares solve.

Brightness constancy. Assume a small image patch keeps the same brightness as it moves by a small displacement $(\Delta x, \Delta y)$ over one frame:

$$I_t(x + \Delta x, y + \Delta y) = I_{t-1}(x, y).$$

Taylor-expand the left side to first order (doc 01 §4.1) about (x, y, t) :

$$I_{t-1}(x, y) + I_x \Delta x + I_y \Delta y + I'_t \approx I_{t-1}(x, y),$$

where $I_x = \partial I / \partial x$, $I_y = \partial I / \partial y$ are the **spatial image gradients** (how brightness changes across pixels) and $I'_t = I_t - I_{t-1}$ is the **temporal change** (how the pixel's brightness changed between frames). Cancelling $I_{t-1}(x, y)$:

$$I_x \Delta x + I_y \Delta y + I'_t = 0.$$

This is the **optical-flow constraint equation** — one equation, two unknowns $(\Delta x, \Delta y)$. A single pixel is not enough to solve it (this is the famous **aperture problem**: looking through a tiny hole, you can only sense motion across an edge, not along it).

Lucas-Kanade’s fix. Assume the displacement is the same for all pixels in a small window $\mathcal{W}$ around the point. Then every pixel $i \in \mathcal{W}$ gives one equation, and we have an overdetermined linear system:

$$\underbrace{\begin{bmatrix} I_x^{(1)} & I_y^{(1)} \\ \vdots & \vdots \\ I_x^{(n)} & I_y^{(n)} \end{bmatrix}}_A \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \underbrace{\begin{bmatrix} -I_t'^{(1)} \\ \vdots \\ -I_t'^{(n)} \end{bmatrix}}_{\mathbf{b}}.$$

Solve it in the **least-squares** sense (the displacement that best satisfies all the equations) via the normal equations:

$$\begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = (A^\top A)^{-1} A^\top \mathbf{b}.$$

Here $A^\top A$ is the 2×2 “structure tensor” $\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$; it is invertible exactly when the window has gradients in two directions (a corner, not a flat patch or a single edge) — which is why good points to track are **corners** (next paragraph).

Pyramids and the window. A first-order Taylor expansion is only valid for small displacements, but cameras can move fast. The fix is a **pyramid**: build downscaled copies of both frames, estimate the (small, in those coarse pixels) flow at the coarsest level, upscale that estimate, refine at the next level, and so on. This is `maxLevel` in the code; the window size is `winSize`. Iterations stop on a small change or a max count (`TERM_CRITERIA`).

Which points? - The **classic** path tracks a fixed regular **grid** of points (e.g. every 50 px), dropping those inside the padded target box. Simple and fast. - The **accurate** path first runs `goodFeaturesToTrack` (a corner detector, Shi-Tomasi: it scores candidate windows by the smaller eigenvalue of $A^\top A$ — large means “trackable corner”) to find up to `homo_max_corners` strong, well-conditioned points, masked away from the target. Tracking corners (where the structure tensor is well-conditioned) gives far more reliable flow than arbitrary points.

Forward-backward check (accurate path). A tracked point can drift or jump. The accurate path tracks each point forward ($t-1 \rightarrow t$) and then back ($t \rightarrow t-1$); if the round trip doesn’t return near the start ($\|\mathbf{p}_{\text{back}} - \mathbf{p}\| > 1.5 \text{ px}$), the point is discarded. This cheap consistency test removes most bad correspondences before fitting.

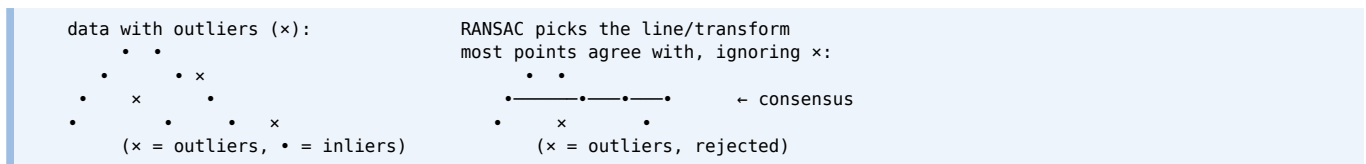
```
forward-backward consistency:
  p  $\xrightarrow{\text{flow } t-1 \rightarrow t}$  p'
  p  $\xleftarrow{\text{flow } t \rightarrow t-1}$  p'      keep iff  $\|\mathbf{p}_{\text{back}} - \mathbf{p}\| \leq 1.5 \text{ px}$ 
  (a reliable track returns to where it started)
```

B.4 RANSAC: fitting a transform when some points lie

Even after the FB check, some correspondences are wrong (a background point that happens to sit on a moving object, repetitive texture, etc.). A plain least-squares fit would be dragged off by these **outliers**. **RANSAC** (RANDOM Sample Consensus) is the standard robust remedy:

1. Randomly pick the minimal number of correspondences needed to define the transform (e.g. 4 point-pairs for a homography).
2. Fit a candidate transform to just those.
3. Count how many all correspondences agree with it — the **inliers** — where “agree” means the point maps to within a reprojection threshold (e.g. 2.5–3 px).
4. Repeat for many random samples; keep the candidate with the most inliers, then refit using all its inliers.

RANSAC’s power: as long as some random sample happens to contain only true inliers, the correct transform will win the vote, no matter how many outliers are present. The classic path uses `cv2.estimateAffinePartial2D` with RANSAC (fits the 4-DOF similarity transform). The accurate path uses `cv2.findHomography` with `USAC_MAGSAC` (MAGSAC++ is a modern RANSAC variant that avoids a hard inlier threshold by marginalizing over it — more robust, slightly slower) to fit the full 8-DOF homography.



B.5 The reliability flag

The estimate is not trusted blindly. RANSAC reports the inlier set; SiamRAM computes the **inlier ratio**

$$\text{inlier_ratio} = \frac{\# \text{ inliers}}{\# \text{ tracked points}}, \quad \text{reliable} = [\text{inlier_ratio} \geq \text{homo_inlier_threshold}].$$

A high ratio means many background points agree on a single global motion — strong evidence the estimate is real. A low ratio means the points disagree (blur, low texture, too many independent movers, a scene cut), so the transform may be garbage and is flagged **unreliable**. Several downstream consumers (the EKF warp, the GMC prior, the spike test we are skipping) require reliable before they will use H . On the first frame there is no previous frame, so the estimate is simply (None, reliable=False) and everyone falls back to “no camera motion.” Handling that “no estimate” case gracefully — rather than forcing a bad warp — is a recurring safety principle.

Part C — The camera-aware EKF (BBoxEKF)

We now have the homography H and its reliability. Time to fold it into a Kalman filter that tracks the target’s center and velocity. This is `models/siamram/motion.py`, and it is a textbook EKF (doc 04 §7) with one nonlinear twist: the prediction warps the center through H .

C.1 State, measurement, and the noise matrices

The state is the 2-D constant-velocity state of doc 04 §6:

$$\mathbf{x} = \begin{bmatrix} c_x \\ c_y \\ v_x \\ v_y \end{bmatrix} \in \mathbb{R}^4.$$

Why center-only, not the box corners or size? Because velocity should be immune to box reshaping. At occlusion onset the matcher’s box often shrinks; if velocity were derived from

corners, that shrink would inject a fake velocity. Tracking only the center decouples motion from size. The width/height are carried separately as a slowly smoothed pair (`_bw`, `_bh`) (an EMA, §C.5), used only to turn the center back into a box on output.

The measurement is the center, so the (linear!) measurement matrix is

$$H_{\text{meas}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad H_{\text{meas}} \mathbf{x} = (c_x, c_y).$$

(We write H_{meas} to avoid clashing with the homography H ; in code it is `_H_mat`.) Initial covariances (diagonal, doc 01 §2.4):

$$P_0 = \text{diag}(10, 10, 100, 100), \quad Q = \text{diag}(q, q, 4q, 4q), \quad R = m I_2,$$

with $q = \text{process_noise}$ and $m = \text{meas_noise}$. Three design choices to read here: - **Velocity starts far more uncertain than position** (100 vs 10 in P_0): we know roughly where the target is at init, but nothing about its velocity. - **Velocity gets $4\times$ the process noise of position** ($4q$ vs q in Q): the constant-velocity assumption is violated mainly through velocity changing (the target accelerates/turns), so we let the filter distrust its own velocity more — exactly the Q -tuning logic of doc 04 §5.1. - $R = mI_2$ treats the two measured coordinates as equally, independently noisy.

C.2 The nonlinear prediction function

One frame's prediction does two things, in order: (1) warp the current center by the camera homography (where would a point sitting at the target go, if it only moved with the camera?), then (2) add the target's own velocity. With (c_x, c_y) the current center, the camera-warped center is the perspective map of Part A.3:

$$g_x(c_x, c_y) = \frac{h_{00}c_x + h_{01}c_y + h_{02}}{D}, \quad g_y(c_x, c_y) = \frac{h_{10}c_x + h_{11}c_y + h_{12}}{D}, \quad D = h_{20}c_x + h_{21}c_y + h_{22}$$

(when H is reliable; otherwise $g_x = c_x$, $g_y = c_y$, i.e. no camera warp). Then the full predicted state is

$$f(\mathbf{x}) = \begin{bmatrix} g_x(c_x, c_y) + v_x \\ g_y(c_x, c_y) + v_y \\ v_x \\ v_y \end{bmatrix}.$$

This is the exact f the EKF uses to move the **mean**: $x[0] = g_x + x[2]$, $x[1] = g_y + x[3]$, velocities unchanged. It is nonlinear precisely because g_x, g_y contain the division by D . The decomposition it encodes is the whole reason camera compensation works:

$$\text{apparent image motion} = \underbrace{\text{camera motion}}_{\text{the } g \text{ warp}} + \underbrace{\text{the target's own motion}}_{\text{the } +v \text{ term}}.$$

Without the warp, a camera pan and a target walk are indistinguishable to the filter; with it, they are separated.

C.3 The Jacobian — derived line by line

To propagate the **covariance** ($P^- = FP^+F^\top + Q$, doc 04 §7.2) the EKF needs $F = \partial f / \partial \mathbf{x}$, the 4×4 Jacobian of f at the current state (doc 01 §4.2). Let's compute all sixteen entries; only four are nontrivial.

The velocity rows are trivial: v_x, v_y don't change, so rows 3 and 4 of F are $(0, 0, 1, 0)$ and $(0, 0, 0, 1)$. The center's dependence on velocity is also trivial: $\partial(g_x + v_x)/\partial v_x = 1$, $\partial(g_y + v_y)/\partial v_y = 1$, giving $F_{0,2} = 1$ and $F_{1,3} = 1$. (These are the constant-velocity couplings — the same $F[0,2]=1$; $F[1,3]=1$ you saw in doc 04 §6.)

The interesting part is how the warped center depends on the current center — the four partial derivatives

$$\frac{\partial g_x}{\partial c_x}, \quad \frac{\partial g_x}{\partial c_y}, \quad \frac{\partial g_y}{\partial c_x}, \quad \frac{\partial g_y}{\partial c_y}.$$

Take $g_x = N_x/D$ with numerator $N_x = h_{00}c_x + h_{01}c_y + h_{02}$ and denominator $D = h_{20}c_x + h_{21}c_y + h_{22}$. Apply the **quotient rule** (doc 01 §4.3), $\frac{d}{dc} \frac{N}{D} = \frac{N'D - ND'}{D^2}$:

$$\frac{\partial g_x}{\partial c_x} = \frac{\frac{\partial N_x}{\partial c_x} D - N_x \frac{\partial D}{\partial c_x}}{D^2} = \frac{h_{00} D - N_x h_{20}}{D^2}.$$

Now the simplifying trick the code uses: since $g_x = N_x/D$, we have $N_x = g_x D$, so

$$\frac{\partial g_x}{\partial c_x} = \frac{h_{00} D - g_x D h_{20}}{D^2} = \frac{h_{00} - g_x h_{20}}{D}.$$

That is the code's $A = (H[0,0] - \text{new_cx_h} * H[2,0]) / \text{denom}$, with $\text{new_cx_h} = g_x$ and $\text{denom} = D$ (plus the 10^{-8} guard, doc 01 §1.3). The other three are identical in form:

$$\boxed{\begin{aligned} A &= \frac{\partial g_x}{\partial c_x} = \frac{h_{00} - g_x h_{20}}{D}, & B &= \frac{\partial g_x}{\partial c_y} = \frac{h_{01} - g_x h_{21}}{D}, \\ C &= \frac{\partial g_y}{\partial c_x} = \frac{h_{10} - g_y h_{20}}{D}, & D_J &= \frac{\partial g_y}{\partial c_y} = \frac{h_{11} - g_y h_{21}}{D}. \end{aligned}}$$

(We write D_J for the fourth entry to avoid clashing with the denominator D ; the code calls it D .) Assembling all sixteen entries:

$$F = \begin{bmatrix} A & B & 1 & 0 \\ C & D_J & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Sanity check. If there is no camera motion (or H unreliable), $g_x = c_x$ and $g_y = c_y$, the denominator is 1, $h_{20} = h_{21} = 0$, $h_{00} = h_{11} = 1$, $h_{01} = h_{10} = 0$, so $A = 1, B = 0, C = 0, D_J = 1$ and

$$F = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

which is exactly the plain constant-velocity transition of doc 04 §6 (F adds velocity to position). So the camera term is a clean generalization: turn the camera off and you recover the ordinary CV Kalman filter. With this F , the covariance prediction is the standard $P^- = FP^+F^\top + Q$.

C.4 The update step

The measurement is the matcher’s box center (or, during recovery, a re-detection). Since the measurement model is **linear** ($h(\mathbf{x}) = H_{\text{meas}}\mathbf{x}$), the update is the ordinary linear Kalman update of doc 04 §5, no Jacobian needed there. With measured center $\mathbf{z} = (c_x^{\text{meas}}, c_y^{\text{meas}})$:

$$\begin{aligned} \mathbf{y} &= \mathbf{z} - H_{\text{meas}} \hat{\mathbf{x}}^- && \text{(innovation)} \\ S &= H_{\text{meas}} P^- H_{\text{meas}}^\top + R \\ K &= P^- H_{\text{meas}}^\top S^{-1} \\ \hat{\mathbf{x}}^+ &= \hat{\mathbf{x}}^- + K \mathbf{y} \\ P^+ &= (I - KH_{\text{meas}}) P^- \end{aligned}$$

Because of the off-diagonal coupling in P^- (built up by the predict step, doc 04 §5), this position-only measurement also corrects the **velocity** estimate — that is how the EKF infers how fast the target is moving from a stream of position measurements. This is exactly the code: `innov = z - H_mat @ x; S = H_mat @ P @ H_mat.T + R; K = P @ H_mat.T @ inv(S); x = x + K@innov; P = (I - K@H_mat)@P.`

C.5 Box size, output, reseed, nudge, and coasting

A few practical methods complete the filter:

- **Size EMA.** On each update the tracked width/height are smoothed toward the measured box (doc 01 §6): $_bw \leftarrow 0.85_bw + 0.15w_{\text{meas}}$ and likewise for height. So size changes gradually and a single bad box can’t resize the output.
- **get_bbox** turns the state back into a box: from center (c_x, c_y) and the smoothed $(_bw, _bh)$, the top-left is $(c_x - _bw/2, c_y - _bh/2)$.
- **get_velocity** returns (v_x, v_y) ; **get_uncertainty** returns $\sqrt{\text{diag}(P_{[1:2]})}$, a scalar standard deviation of the position estimate (how unsure the filter is about where the target is).
- **reseed(box, velocity)** hard-resets the state to a new position and velocity and re-inflates the covariance ($P = \text{diag}(25, 25, 40, 40)$). This is used when an external event teleports the target legitimately — e.g. a successful re-acquisition after occlusion (doc 08). Without reseeding, the filter would slowly drag itself toward the new location and report wrong velocities meanwhile; reseeding says “forget the old trajectory, start fresh here.”
- **nudge_position(box)** moves the center without touching velocity — a gentle correction used during recovery when we want to bias the search center toward a plausible detection but not commit to it (doc 08).
- **Coasting during occlusion.** When the target is out of frame, there is no warp to apply and no measurement to update from, so the code simply inflates uncertainty: $P \leftarrow P + Q$ each frame (this is the “no measurement $\rightarrow$ coast, covariance grows” case of doc 04 §8). The filter keeps extrapolating with growing uncertainty until the target is re-found. When in occlusion but in frame, it still does the normal `predict(H, reliable)` so it keeps

following camera motion while it searches.

Part D — Using and trusting the camera motion

The EKF consumes H in its predict step, but H also drives several outer decisions. Two are worth stating here (the search-prior use is doc 06; loss-cause classification is doc 08).

D.1 Plausibility gates (`gmc_motion_is_valid`)

Even a “reliable” H can be physically absurd (RANSAC can occasionally fit a large bogus rotation in a blurred, low-texture scene). Before any search-prior use, SiamRAM checks the decomposed statistics (A.4) against caps:

$$\begin{aligned} |dx|, |dy| &\leq \text{max_translation_frac} \cdot \max(W, H) && \text{(e.g. 25\% of the long edge)} \\ \text{min_scale} \leq \text{scale} \leq \text{max_scale} &&& \text{(e.g. [0.7, 1.4])} \\ |\theta| &\leq \text{max_rotation_deg} && \text{(e.g. 25}^\circ\text{)} \\ \text{max_corner_disp} &\leq \text{max_corner_frac} \cdot \sqrt{W^2 + H^2} && \text{(fraction of the frame diagonal)} \end{aligned}$$

If any is exceeded, the transform is rejected for that purpose (“a mathematically valid but physically unlikely transform”). This is a belt-and-suspenders design: reliable filters statistically bad fits; the plausibility gates filter statistically fine but physically implausible ones.

D.2 Camera displacement magnitude

A scalar “how far did the camera move this frame” is obtained by warping the **frame center** $(W/2, H/2)$ through H and measuring how far it moved (`_h_translation_magnitude`):

$$\text{cam_disp} = \left\| (g_x, g_y) \Big|_{(W/2, H/2)} - (W/2, H/2) \right\|.$$

This single number, accumulated into a short history, feeds the “heavy camera motion” detector (doc 06) and the loss-cause vote at occlusion entry (doc 08). It is the same perspective formula as everywhere else, evaluated at the frame center.

Recap

- **Homogeneous coordinates** let every camera transform be a 3×3 matrix H ; the **perspective division** $(x', y') = (\frac{h_{00}x + h_{01}y + h_{02}}{D}, \frac{h_{10}x + h_{11}y + h_{12}}{D})$ is the lone nonlinearity, and it is literally the EKF’s predict math.
- H decomposes into translation (dx, dy) , scale $\sqrt{a^2 + b^2}$, rotation $\text{atan2}(b, a)$, and a worst-case corner displacement.
- SiamRAM estimates H from **background** points (target masked) via **Lucas-Kanade optical flow** (brightness constancy + least squares over a window + pyramids, with a forward-backward consistency check on the accurate path) and a **RANSAC** robust fit (4-DOF similarity for “classic”, 8-DOF MAGSAC homography for “accurate”), flagged **reliable** by the inlier ratio.
- The **BBoxEKF** is a constant-velocity EKF over (c_x, c_y, v_x, v_y) whose **prediction warps the center through H then adds velocity**, separating camera motion from target motion. Its Jacobian’s four nontrivial entries are $\frac{h_{00} - g_x h_{20}}{D}$ and $\frac{h_{10} - g_y h_{20}}{D}$, derived by the quotient rule; turning the camera off recovers the plain CV filter.
- The **update** is the ordinary linear Kalman update (measuring the center), which also

corrects velocity through the covariance coupling; size is carried as a separate EMA; reseed/nudge/coasting handle teleports and occlusion.

- Outer code **trusts** H only after a reliability flag and physical-plausibility gates, and reads a scalar camera-displacement from it.

Next: **06_MOTION_PRIORS.md**, where the camera motion and a second Kalman filter become priors that steer the visual matcher's search and reweight its response — the bridge from “we know where the target should be” to “so look there.”

06 — Motion Priors: Steering the Matcher with Physics

Prerequisites: doc 03 (the response map, the search crop, why the target must stay inside it), doc 04 (Kalman filter), doc 05 (homography, the EKF, camera displacement). A **prior** is a belief held before looking at the current image. This document covers the three ways SiamRAM injects motion knowledge into the otherwise appearance-only matcher: a **target-motion Kalman prior** (where the object is going), the **camera-motion GMC prior** (where the background went), and the **heavy-camera-motion** gate that tells the rest of the system when image motion is dominated by the camera. The recurring theme: appearance says where the target looks like it is; motion says where it should plausibly be; the strongest decision is where they agree.

1. Two questions, two priors

A moving image can change for two independent reasons (doc 05 Part C): the **target** moved, or the **camera** moved. SiamRAM has a separate prior for each:

Prior	Question it answers	Mechanism	Source
Target-motion Kalman	“Where will the object go, given its own recent velocity?”	a constant-velocity KF over the box; reweights the response map / recenters the crop	KalmanMotionPrior
Camera-motion GMC	“Where did the whole image move because the camera moved?”	warp the previous box by the homography H ; use it as the search center	apply_gmc_search_prior

They are complementary. If the camera is fixed and the object walks right, the target prior helps and the camera prior has nothing to do. If the camera pans and the object is stationary, the camera prior helps and the target prior alone would mistake the pan for object motion. If both move, the camera prior handles the global shift and the target prior handles the residual.

A note to prevent confusion: there are now **two** Kalman filters in SiamRAM. The **BBoxEKF** of doc 05 (center + velocity, camera-aware) is the outer motion model — it drives the occlusion search and the EKF predict at the top of every frame. The **KalmanMotionPrior** of this document is a second, inner filter (full box, no camera term) used only to reweight the matcher’s response map. They serve different roles and do not share state.

2. The target-motion Kalman prior (KalmanMotionPrior)

This is a small constant-velocity Kalman filter (doc 04 §6) over the **full box**, in the SAMURAI style. Its job is not to be the answer but to score the matcher’s candidate locations by motion consistency.

2.1 State and model

The state is the box and its velocity — eight numbers:

$$\mathbf{x} = (c_x, c_y, w, h, v_x, v_y, v_w, v_h).$$

So it tracks not only how the center moves (v_x, v_y) but also how the box grows/shrinks (v_w, v_h) . The constant-velocity transition F (an 8×8 matrix) adds each velocity to its position-like coordinate:

$$c_x \leftarrow c_x + v_x, \quad c_y \leftarrow c_y + v_y, \quad w \leftarrow w + v_w, \quad h \leftarrow h + v_h, \quad (\text{velocities unchanged}),$$

i.e. $F = \begin{bmatrix} I_4 & I_4 \\ 0 & I_4 \end{bmatrix}$ (the block form of doc 04 §6). The measurement is the full box, so $H = [I_4 \mid 0]$ reads off (c_x, c_y, w, h) from the state.

2.2 Scale-adaptive noise (the SAMURAI detail)

A clever, tuning-free choice: the process and measurement noise standard deviations are made **proportional to the target’s height** h . Using constants $\sigma_{\text{pos}} = 1/20$ and $\sigma_{\text{vel}} = 1/160$ (the standard MOT/ByteTrack convention), the per-frame process noise standard deviations are

$$\sigma_Q = (\underbrace{\sigma_{\text{pos}}h, \sigma_{\text{pos}}h, \sigma_{\text{pos}}h, \sigma_{\text{pos}}h}_{\text{position-like}}, \underbrace{\sigma_{\text{vel}}h, \sigma_{\text{vel}}h, \sigma_{\text{vel}}h, \sigma_{\text{vel}}h}_{\text{velocity-like}}),$$

and $Q = \text{diag}(\sigma_Q^2)$ (variance is standard deviation squared, doc 01 §5.2); the measurement noise R uses the position-like block. **Why scale by h ?** Because a “big” motion for a tiny far-away target (a few pixels) is a “small” motion for a large near target (tens of pixels). Tying the noise to the target’s size makes the filter behave the same way relative to the target regardless of how big it appears — no per-video retuning. (At seeding, the initial covariance uses inflated versions of these, e.g. $2\sigma_{\text{pos}}h$ for position and $10\sigma_{\text{vel}}h$ for velocity, encoding “unsure at start,” echoing doc 05 §C.1.)

2.3 The three operations

- **seed(box)** — initialize the state at a box with **zero velocity** and the inflated covariance. Called at init and whenever the filter must restart.
- **predict()** — advance one frame by F and grow the covariance by Q (doc 04 §3 predict).
- **update(box)** — the standard linear Kalman update (doc 04 §5) toward the matcher’s chosen box.

It also exposes **predicted_bbox()** (the current state as an xywh box, used for the fusion below) and **center_distance(box)** (Euclidean distance between the state center and a box center, used for the reseed test).

2.4 Stability gating: never fight a fresh start

The prior is only allowed to influence selection after the tracker has been **stable** for several consecutive confident frames (`kf_motion_stable_frames`). The reason is fundamental: right after (re)initialization the filter’s velocity is meaningless (seeded at zero), so trusting it would hurt. The gate is bookkept by a streak counter:

- On a **confident** frame (score $\geq$ a threshold) that is not a big jump, the filter updates and the streak increments.

- If the new box is a big jump from the prediction — center distance $> \text{reseed_dist} \times \text{box diagonal}$ — the filter **reseeds** at the new box and the streak **resets to 0**. This covers legitimate teleports (a reacquisition writing the box directly): the prior should restart there, not slowly drag toward it.
- On a **low-confidence** frame the filter neither updates nor counts; the streak resets. (It “coasts,” doc 04 §8.)

The prior is “active” only when the streak $\geq \text{kf_motion_stable_frames}$ (and not explicitly suppressed). During recovery candidate-verification the prior is suppressed outright (`_kf_suppress`): a re-acquired target is, by definition, far from the stale pre-occlusion trajectory, so the motion prior would wrongly demote exactly the right answer.

2.5 KF-IoU response fusion (the core idea)

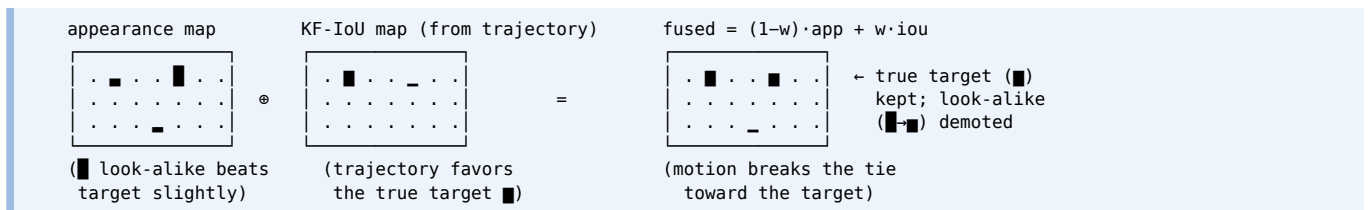
When the prior is active, SiamRAM blends a **motion-consistency score** into the matcher’s response map before the argmax (doc 03 §10, step 6). For every grid cell, the matcher already has its decoded candidate box (doc 03 §5.3). Map the KF’s predicted box into the same crop coordinates (using the crop→frame mapping of doc 02 §4.4, inverted), then compute, per cell, the **IoU** (doc 02 §3) between that cell’s candidate box and the KF box:

$$\text{KF_IoU}(i, j) = \text{IoU}(\text{candidate box}(i, j), \text{KF predicted box}) \in [0, 1].$$

High where the candidate sits where the trajectory expects the target; low where it contradicts the motion. The fused selection map is the convex blend

$$\text{fused}(i, j) = (1 - w) \cdot \text{appearance}(i, j) + w \cdot \text{KF_IoU}(i, j), \quad w = \text{kf_motion_weight} \in [0, 1].$$

This is a **weighted average** (doc 01 §7.3) of “looks like the target” and “is where the target should be.” With $w = 0$ it is pure appearance (legacy behavior); as w grows, motion gets more say. The classic payoff: when a crossing look-alike briefly produces a slightly higher appearance peak, its low KF-IoU (it is off the trajectory) demotes it, and the true target — consistent with motion — wins. The danger, symmetrically, is that too large a w makes the tracker follow a predicted trajectory through a genuine sudden turn; hence w is kept modest (e.g. 0.15-0.2).



2.6 KF-centered search (keeping fast movers in the crop)

The fusion above reweights within a crop, but it can’t help if the target already left the crop. A second, optional use of the same prior addresses that: center the **search crop** at the KF-predicted center instead of the previous box’s center (`kf_motion_center_search`). Only the center moves; the crop size still comes from the tracker’s own box estimate, so the crop geometry and the size-penalty’s `prev_size` are unaffected. For a fast mover, placing the crop ahead of the target (where motion predicts it is going) keeps it inside the search region — recall (doc 03 §3) that a target outside the crop is simply unfindable. This is “predict-then-associate,” the MOT pattern: use the motion model to place the search, then let appearance find the exact location.

3. The camera-motion GMC prior (apply_gmc_search_prior)

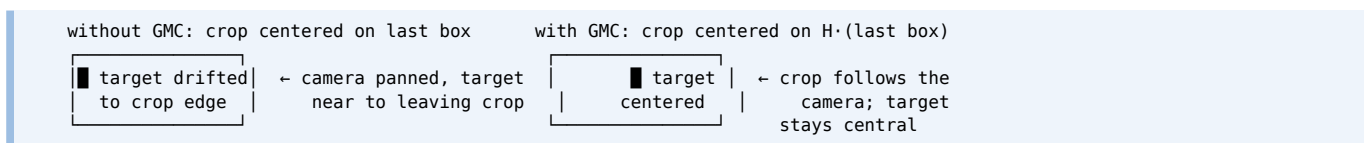
GMC = **Global Motion Compensation**. Where the target prior asks “where is the object going,” GMC asks “where did the image move because the camera moved,” and uses the answer to recenter the search before the matcher runs.

3.1 What it does

Given the reliable homography H from doc 05, GMC warps the **previous target box** into the current frame (doc 05 A.3): take the four corners of the previous box, push each through H (perspective divide), and build the axis-aligned box around the warped corners (`_warp_bbox_with_homography`):

$$b_{\text{prior}} = \text{AABB}(\{H \cdot \tilde{\mathbf{c}}_k\}_{k=1}^4),$$

clipped to the frame. Warping all four corners (rather than just shifting by (dx, dy)) means the prior also reflects mild zoom and rotation, not only translation. This warped box is written into SiamABC’s internal search state (`_set_tracker_search_prior`), so the matcher’s next search crop is centered on the **camera-compensated** location. As the GMC note in the repo puts it: “Before visual matching, start looking from the camera-compensated location.” It does **not** force the answer — the matcher still does the visual match and can land anywhere in the (now better-placed) crop.



3.2 When GMC is skipped

GMC is intentionally conservative — a wrong prior is worse than none (doc 05 B.5). It is skipped when:

- there is no current box, or no homography H (e.g. first frame);
- H is **unreliable** and reliability is required (`gmc_prior_require_reliable_h`);
- H fails the **plausibility gates** of doc 05 §D.1 (translation/scale/rotation/corner caps);
- the warp produces an invalid box.

(There is also a skip during distractor mode, which is out of scope here.) In any skip case the matcher just uses its ordinary last-box-centered crop. The robustness payoff of GMC is exactly in the cases real video throws constantly — fast pans, shake, small zoom — where the target would otherwise drift to the crop edge.

3.3 GMC vs. KF-centered search — do they fight?

Both want to set the search center. They are not contradictory, but they interact: GMC writes a camera-compensated box into the tracker state, and the KF prior may then see that as an external move and reseed (resetting its stability streak, §2.4). This is by design — after another subsystem moves the search, the stale object-velocity prediction shouldn’t be trusted until motion is re-established. The practical upshot: with GMC on, the KF prior tends to be less continuously active. Neither is “wrong”; they encode different, occasionally overlapping, knowledge, and the conservative reseed keeps them from compounding errors.

4. Heavy camera motion: a gate for the whole system

The third use of camera motion is not a search prior at all — it is a **boolean signal**, “is the image motion right now dominated by the camera?”, consumed by several other subsystems (occlusion entry in doc 08; template-rate adaptation in doc 10). Getting this right prevents

a large class of false alarms: during a fast pan, the matcher’s confidence dips and the box lags — but the target is not lost, the camera is just moving faster than the search can follow. Treating that dip as “target gone” would trigger needless occlusion recovery.

4.1 The displacement signals

Two scalars are combined (`is_heavy_camera_motion`):

- **Instantaneous displacement** — this frame’s camera-center displacement from H (doc 05 §D.2): warp the frame center through H , measure how far it moved.
- **Weighted-history displacement** — an exponentially weighted average (doc 01 §7.3) of recent per-frame camera displacements, with weights growing toward the present:

$$\text{weighted_disp} = \frac{\sum_i w_i d_i}{\sum_i w_i}, \quad w_i = e^{(i-n)/\tau} \text{ (larger for recent } i\text{)},$$

where d_i are the stored displacements. Using a history (not just this frame) smooths out single-frame estimate noise so a sustained pan is detected even if one frame’s H was momentarily weak. The code takes the **max** of the instantaneous and weighted values, `max_disp = max(inst, weighted)`, so either a sudden lurch or a sustained pan can trip the gate.

4.2 Scale-normalizing the displacement

A raw pixel displacement means different things for a tiny vs. a large target — the same recurring scale issue (doc 02 §2, doc 06 §2.2). So the displacement is also normalized by a **reference box diagonal** d_{ref} (the current box’s diagonal, falling back to the frame diagonal if no box):

$$\text{max_norm} = \frac{\text{max_disp}}{d_{\text{ref}}}.$$

This bbox-relative number (“the camera moved by X box-diagonals this frame”) is the scale-independent measure, and it is also published for the adaptive template-rate controller (doc 10).

4.3 The verdict

Heavy camera motion is declared if either an absolute pixel gate or a normalized gate is exceeded (each can be disabled by setting its threshold to 0):

$$\text{heavy} = [\text{max_disp} \geq \tau_{\text{px}}] \vee [\text{max_norm} \geq \tau_{\text{norm}}].$$

This boolean then blocks premature occlusion entry (doc 08), extends the patience before declaring loss, and can adapt the template-update rate (doc 10). It is one of the most leveraged signals in the system, which is why it draws on both an absolute and a scale-relative criterion and on both instantaneous and smoothed history — robustness layered on robustness.

5. How a frame uses all three (ordering)

In the normal (non-occlusion) update, the priors are applied in this order, before and around the SiamABC forward of doc 03:

1. heavy-camera-motion verdict refreshed (used later for gating) [§4]
2. GMC: if H reliable & plausible, recenter the search crop on H ·(prev box) [§3]
3. (frame-dynamics motion saliency may be blended into the crop – doc 09)
4. SiamABC forward → appearance response map [doc 03]

```

5. if the KF prior is active: fuse KF-IoU into the response map, then argmax [$2.5]
   (and/or the crop was KF-centered in step 2's place — §2.6)
6. ... downstream: thresholding, EKF update, memory, occlusion entry ...

```

So GMC acts before the matcher (it changes where we look); the KF fusion acts inside the matcher's decoding (it changes which peak wins); the heavy-motion verdict acts after (it changes how we interpret a low score). Three different insertion points for three different kinds of motion knowledge.

6. Recap

- SiamRAM holds **two** motion priors: a **target-motion** Kalman prior (object's own velocity) and a **camera-motion** GMC prior (global image shift). They answer different questions and combine cleanly.
- The **KalmanMotionPrior** is an 8-state constant-velocity KF over the full box with **height-scaled noise** (tuning-free across target sizes). It is gated to act only after a **stability streak**, reseeds on big jumps, and is suppressed during recovery verification.
- **KF-IoU fusion** blends a motion-consistency IoU into the response map, fused = $(1 - w)$ appearance + w KF_IoU, demoting motion-inconsistent look-alikes; **KF-centered search** places the crop ahead of fast movers.
- The **GMC prior** warps the previous box by the homography to recenter the search on the camera-compensated location, guarded by reliability and physical-plausibility gates; it steers, never forces.
- **Heavy-camera-motion** is a leveraged boolean built from instantaneous + exponentially-weighted, absolute + scale-normalized displacement; it prevents the system from mistaking pan-induced confidence dips for target loss.

Next: **07_APPEARANCE_MEMORY_RAM_DRM.md** — the long-term identity memory that lets the tracker re-recognize the target after it has been lost, and the composite score that ranks re-acquisition candidates.

07 — Appearance Memory: RAM and DRM

Prerequisites: doc 01 (cosine similarity, exponential decay, Gaussian bump, weighted average), doc 02 (IoU, box center). This document explains how SiamRAM remembers what the target looks like so it can **re-recognize** the target after losing it. There are two memories — a short-term **RAM** buffer and a long-term **DRM** (Dynamic Reference Memory) bank — and a multi-term **composite score** used to rank re-acquisition candidates during occlusion recovery (doc 08). The matcher (doc 03) handles short-horizon “where did it move”; this memory handles long-horizon “is this the same object.” Implemented in `models/siamram/memory.py`.

1. Why a separate appearance memory?

The matcher’s dynamic template (doc 03 §9) is a short-window appearance model, refreshed continuously. That is perfect for following gradual change, but it has two weaknesses for the recovery problem:

1. It drifts. If the matcher briefly latched onto the wrong thing, the dynamic template absorbs that error.
2. It is a single fused template, not a bank of verified exemplars you can match a fresh detection against.

When the target has been **lost** (occlusion, doc 08) and an object detector proposes several candidate boxes, we need to ask “which of these, if any, is the same object we were tracking?” Answering that robustly needs (a) a stable set of known-good appearance exemplars, and (b) a scoring rule that combines appearance with overlap, motion, and recency. That is exactly RAM + DRM + the composite score.

1.1 Appearance descriptors

A **descriptor** is a fixed-length feature vector summarizing the look of a box’s contents — produced by a small re-identification network (OSNet) applied to the cropped region (`_extract_descriptor`). Two crops of the same object give descriptors pointing in nearly the same direction; different objects point differently. We compare descriptors with **cosine similarity** (doc 01 §1.3):

$$\text{cossim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\| + \varepsilon} \in [-1, 1],$$

brightness-/scale-robust, 1 for “same look,” near 0 for “unrelated.” This single number is the appearance signal throughout this document.

2. RAM — the short-term buffer

RAM is a fixed-capacity deque of recent (**box, descriptor**) pairs from confidently tracked frames. It is the raw material from which the long-term DRM is curated, and it provides the “what did the target look like most recently” reference for simple matching.

2.1 Admission gates (try_admit)

A new (box, descriptor) is admitted only if it passes **two geometric coherence gates** — appearance is not checked at admission (a confidently tracked frame is assumed to be the target; the gates guard against the box having jumped or resized suspiciously):

1. **IoU gate.** The new box must overlap the previous box: $\text{IoU}(b_t, b_{t-1}) \geq \tau_{\text{iou}}$ (doc 02 §3, e.g. $\tau_{\text{iou}} = 0.4$). A sudden positional jump (low IoU) suggests confusion, not smooth tracking, so it is refused.
2. **Area gate.** The new box’s area must be close to the **median** area of the buffer (doc 01 §7.2 — median for robustness to a few odd entries):

$$\frac{|w_t h_t - \text{median}(\text{areas})|}{\text{median}(\text{areas}) + \varepsilon} \leq \tau_{\text{area}} \quad (\text{e.g. } 0.25).$$

This rejects frames where the box suddenly ballooned or collapsed (a partial occlusion or a bad match) — those would pollute the memory of “the target’s size.”

If both pass, the pair is appended (oldest evicted when full), and the candidate is immediately tested for **promotion** to DRM (§3). Tying admission to geometry rather than appearance is deliberate: we trust that confident, smoothly-moving frames are the target, and we keep out the geometrically suspicious ones.

2.2 The simple match (match)

The most basic query: given candidate boxes in the current frame, extract their descriptors and return the one most similar (cosine) to the **most recent** RAM descriptor, if it clears a threshold. This is the fallback used when the long-term DRM is still empty (early in a sequence). It answers “which candidate looks most like the target did a moment ago.”

3. DRM — the long-term reference bank

The matcher’s dynamic template and RAM both track recent appearance, which is good for drift-following but bad for re-identification after a long gap: by the time the target reappears, “recent” may be corrupted or stale. **DRM** is a small, curated bank of **anchors** — verified, mutually-consistent exemplars — meant to remain valid across an occlusion. Each anchor is a triple (box, descriptor, ρ) where ρ is the time index it was stored (its age stamp).

3.1 Promotion by agreement (_try_promote_to_drm)

A RAM entry is promoted to DRM only if it is **corroborated** by its neighbors. When a new descriptor $\mathbf{d}$ is admitted to RAM, look at the last W RAM entries (a window, window_W) and count how many agree with $\mathbf{d}$ in appearance:

$$\text{agreements} = \#\{\text{recent entry } \mathbf{d}_i : \text{cossim}(\mathbf{d}, \mathbf{d}_i) \geq \tau_{\text{sim}}\}.$$

If $\text{agreements} \geq m_{\text{min}}$ (e.g. $\tau_{\text{sim}} = 0.85$, $m_{\text{min}} = 3$), the entry is appended to DRM. The logic: an exemplar worth keeping long-term should be appearance-consistent with several of its temporal neighbors — that filters out one-off lucky matches and keeps only stable, repeatedly-confirmed looks of the target. (There is a second write path, `add_drm_anchor`, used by the introspection feature; the agreement path is the primary one.) DRM has a small capacity (e.g. 10), so it holds a compact, high-quality set rather than a long history.

RAM (recent, by time $\rightarrow$)
 $[d_1][d_2][d_3][d_4][d_5][d_6 \text{ new}]$
└─ window W ─┘

DRM (curated anchors)
┌──────────────────┐
| (box, desc, ρ) $\times \leq 10$ |

new d_6 agrees ($\text{cossim} \geq \tau_{\text{sim}}$) with
 $\geq m_{\text{min}}$ of the window $\rightarrow$ promote $\rightarrow$

verified, stable

4. The composite re-acquisition score (drm_match)

This is the heart of recovery scoring: given a set of candidate boxes (object-detector proposals during occlusion, doc 08), score each by **how much it looks and behaves like the lost target**, and return the best (or a short list). Appearance alone is not enough — a look-alike can score high on appearance — so the score combines appearance with overlap, motion direction, recency, and spatial plausibility. We build it term by term.

Setup: let the **reference box** b_{ref} (center $(c_x^{\text{ref}}, c_y^{\text{ref}})$) be the last known target box, and $\mathbf{v}$ the target’s velocity (from the EKF, doc 05). For each **DRM anchor** $k = (b_k, \mathbf{d}_k, \rho_k)$ and each **candidate** $c = (b_c, \mathbf{d}_c)$, the score combines the following.

4.1 Overlap term

$$s_{\text{iou}} = \lambda_{\text{iou}} \cdot \text{IoU}(b_k, b_c).$$

Rewards a candidate that geometrically overlaps the anchor’s box. (Most relevant when anchors are spatially near the candidate.)

4.2 Appearance term

$$s_{\text{app}} = \lambda_{\text{app}} \cdot \text{cossim}(\mathbf{d}_k, \mathbf{d}_c).$$

Rewards a candidate whose descriptor matches the anchor’s — the core “same object” signal.

4.3 Motion-direction term

Does the direction from the reference to the anchor agree with where the target was heading? Form the displacement vector from reference center to anchor center, $\mathbf{m}_k = (c_x^k - c_x^{\text{ref}}, c_y^k - c_y^{\text{ref}})$, and take the cosine of the angle between the velocity and that displacement (doc 01 §1.2–1.3), clamped to be non-negative:

$$\pi_k = \max\left(0, \frac{\mathbf{v} \cdot \mathbf{m}_k}{\|\mathbf{v}\| \|\mathbf{m}_k\| + \varepsilon}\right) \in [0, 1].$$

$\pi_k = 1$ when the anchor lies exactly in the direction of motion, 0 when it is sideways or behind. The $\max(0, \cdot)$ means “no bonus for being behind the motion,” but no penalty either (other terms handle that).

4.4 Recency term

Older anchors should count a little less. With age $= t - \rho_k$ (how many time steps ago the anchor was stored) and a decay rate α (doc 01 §3.3):

$$s_{\text{time}} = \lambda_{\text{time}} \cdot e^{-\alpha \text{age}}.$$

$e^{-\alpha \text{age}}$ is 1 for a fresh anchor and decays toward 0 for old ones — a soft preference for recent evidence without discarding the old.

4.5 (Optional) negative-bank penalty

There is a subtractive term $-\gamma \cdot \max_j \text{cossim}(\text{distractor}_j, \mathbf{d}_k)$ that penalizes anchors resembling a bank of known non-target appearances. How that “distractor bank” is populated is tied to the distractor/spike machinery that is **out of scope** for these notes; for our purposes it is simply a small negative term that can demote an anchor that looks like a known impostor. With an empty bank (or $\gamma = 0$) it vanishes.

4.6 Per-anchor raw score and aggregation

The motion + recency (− penalty) part is computed per anchor once; the per-anchor raw score for a candidate is

$$\text{raw}_k = s_{\text{iou}} + s_{\text{app}} + \underbrace{(\lambda_{\text{mot}} \pi_k + \lambda_{\text{time}} e^{-\alpha \text{age}} - \gamma (\text{penalty}))}_{s_{\text{mot+time}}}.$$

A candidate is scored against every anchor, then those per-anchor scores are **aggregated** into one candidate score (`_aggregate_drm_anchor_scores`):

- **max** — the best matching anchor wins (“does this candidate match any good exemplar?”). Most permissive.
- **mean** (optionally **top- k mean**) — average over all anchors, or just the k best (“does this candidate match the target’s exemplars on the whole?”). More conservative; top- k is a middle ground that ignores the worst anchors.

4.7 Two candidate-level adjustments

After aggregation, two terms that depend on the candidate (not per-anchor) are added:

Spatial distance penalty. Candidates far from the current search center (s_{cx}, s_{cy}) are penalized with a Gaussian-shaped falloff (doc 01 §3.3):

$$\text{cand_score} -= \lambda_{\text{dist}} \left(1 - e^{-\frac{1}{2}(d/\sigma)^2} \right), \quad d = \|(c_x^c, c_y^c) - (s_{cx}, s_{cy})\|.$$

The bracket is 0 at the search center (no penalty) and rises toward λ_{dist} far away — a soft “the target should be near where we’re looking” prior, with σ setting how forgiving it is.

Candidate-direction term. Like §4.3 but for the candidate itself: reward candidates lying in the direction of the target’s velocity from the reference,

$$\text{cand_score} += \lambda_{\text{cand_dir}} \cdot \frac{\mathbf{v} \cdot \mathbf{m}_c}{\|\mathbf{v}\| \|\mathbf{m}_c\| + \varepsilon}, \quad \mathbf{m}_c = (c_x^c - c_x^{\text{ref}}, c_y^c - c_y^{\text{ref}}).$$

(Here the cosine is not clamped to ≥ 0 : a candidate behind the motion is actively penalized, since the target was moving forward.)

4.8 Acceptance, ranking, and the skip shortcut

A candidate is kept only if its final composite score exceeds the **margin** μ (a threshold; can be adapted per-video, doc 10). Survivors are sorted descending. Two outputs:

- If the **best** candidate’s score is at least a high **skip threshold** (e.g. 0.80), return only it — it is so good there is no need to verify alternatives.
- Otherwise, return the **top- k** candidates for the verification stage (doc 08 will re-run the matcher on each).

So `drm_match` is a ranked shortlist generator: appearance + overlap + motion + recency + spatial plausibility, summarized into one number per candidate, thresholded by a margin, with a fast path when one candidate is obviously the target.

```

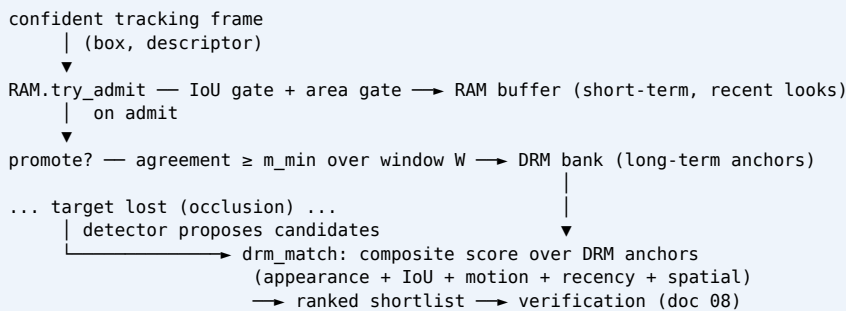
candidates → for each: combine over DRM anchors
    s_iou + s_app + (λ_mot·π + λ_time·e^{−α·age} − γ·pen)
    → aggregate(max / top-k mean) → base score
    − λ_dist·(1 − e^{−½(d/σ)²})          (far from search center?)
    + λ_cand_dir·cos(v, m_c)           (along the velocity?)
    → keep if > margin → sort → (best ≥ skip? return it) else top-k

```


4.9 Scoring the live target (for auto-tuning)

There is a sibling method, `score_target_against_drm`, that runs the same composite formula on the **current, genuine target** as if it were a re-acquisition candidate. Why score something you already have? Because the value of that score tells you how high the true target typically scores against its own DRM — which is exactly the information needed to set the acceptance **margin** automatically (doc 10’s `AutoDrmMargin`). It skips the spatial-distance and candidate-direction terms (degenerate for the live target, whose search center coincides with itself) and returns the composite, so the number is on the same scale as `margin`. This is a nice design motif: measure how the real target scores under your own rule, and set the threshold just below that.

5. How the pieces relate



- **RAM** = short-term, geometry-gated, raw recent exemplars.
- **DRM** = long-term, agreement-curated, stable anchors that survive an occlusion.
- **drm_match** = the composite ranker that turns “here are some candidate boxes” into “here is the one that is most plausibly the same target.”

6. Recap

- Appearance is summarized by an OSNet **descriptor**; comparisons use **cosine similarity**.
- **RAM** admits confident frames through an **IoU gate** and a **median-area gate** (geometry, not appearance), keeping a short window of recent (box, descriptor) pairs.
- **DRM** promotes a RAM entry only when it **agrees** ($\cosine \geq \tau_{sim}$) with at least m_{min} of its recent neighbors — a curated, stable, long-term anchor bank.
- The **composite re-acquisition score** sums an **overlap** term, an **appearance** term, a **motion-direction** term ($\max(0, \cos)$ of velocity vs. displacement), and an exponential **recency** term per anchor; aggregates ($\max$ / top- k mean); then applies a Gaussian **distance** penalty and a **candidate-direction** term; accepts above a **margin**; and returns the best (skip shortcut) or a top- k shortlist.
- `score_target_against_drm` runs the same formula on the live target to drive the **adaptive margin** (doc 10).

Next: **08_OCCLUSION_RECOVERY.md** — how loss is detected, how the EKF coasts and steers a growing search, how the detector + DRM + matcher cooperate across phases to find the target again, and how a winner is verified before tracking resumes.

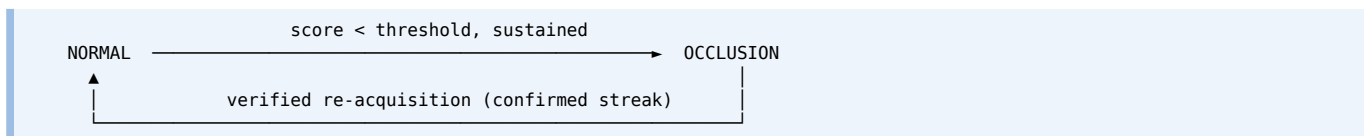
08 — Occlusion Recovery

Prerequisites: doc 03 (matcher confidence), doc 05 (EKF: predict, coast, reseed, velocity), doc 06 (heavy camera motion), doc 07 (RAM/DRM and the composite re-acquisition score). This document explains what happens when the target is **lost** — hidden, off-frame, or otherwise unmatchable. We cover how loss is detected (with hysteresis and guards), how the tracker coasts on the EKF while searching, the multi-**phase** re-detection state machine (fast re-try → detector candidates → memory scoring → verification), the geometry of the growing search region, and how a winner is verified before normal tracking resumes. Implemented across `occlusion_recovery.py` and the conductor in `tracker.py`.

1. Two worlds, one switch

Recall the structural fact from doc 00: **per frame, exactly one of normal update or occlusion update runs**. A single boolean `in_occlusion` selects the world. The normal world (docs 03/06) assumes the target is visible and trusts the matcher; the occlusion world assumes it is not visible and runs a structured search instead.

While `in_occlusion` is true, the public output is special: the tracker emits either zeros or the held box (the EKF's best guess), with score 0.0 and the `in_occlusion` flag set. The “real” internal position is carried in `held_box`; only the **first frame after successful re-acquisition** emits genuine coordinates again. So from the outside, occlusion looks like “no confident answer,” while inside, the machinery is busy coasting and searching.



2. Detecting loss

A tracker must not panic at a single bad frame, nor dawdle while the target is truly gone. SiamRAM uses **hysteresis** plus several guards.

2.1 The effective threshold

Each frame computes an **effective loss threshold** (`_compute_effective_threshold`). It is the configured confidence threshold, switched for far/tiny targets: if the object is “long-distance” (its median area is a tiny fraction of the frame), a separate, lower `long_distance_conf_threshold` is used (such targets genuinely score lower even when tracked correctly — the same difficulty-awareness that motivates the adaptive thresholds of doc 10), and the matcher’s search context is widened (offset +0.5) to keep the small target in view. The threshold itself can be a fixed number or **adaptive** (doc 10).

2.2 Entry hysteresis

A loss is declared only after the score stays below threshold for **several consecutive frames** (`entry_patience`). A streak counter `_entry_streak` increments on each below-threshold frame and resets to 0 on any healthy frame; occlusion is entered when

$$\text{_entry_streak} \geq \text{effective_entry_patience}.$$

This **hysteresis** (requiring persistence before switching states) is the standard cure for chattering: a one-frame dip from blur or a passing shadow won't flip the state, but a real disappearance will. Under heavy camera motion the patience can be raised (`entry_patience_high_motion`), giving the matcher extra frames to catch up to a fast pan before crying loss.

2.3 Guards against false entry

Two more guards before committing:

- **Grace window.** During the first N frames of a sequence (`no_occlusion_first_n_frames`), occlusion entry is forbidden and the streak held at 0. Early on, the dynamic template is still settling (doc 03 §9), so low scores are expected and must not be mistaken for loss.
- **Heavy-camera-motion block.** If heavy camera motion is detected (doc 06 §4) and the target is not exiting the frame, the streak is blocked from accumulating and entry is refused — a confidence dip during a fast pan is “search lag,” not loss. (Exception: if the target is genuinely exiting an edge, §4.2, entry is allowed even under camera motion.)

2.4 Exit-direction detection

At the moment of entry, the tracker checks whether the target was leaving the frame and, if so, which **edge** (`_detect_exit_direction`). This sets `_out_of_frame` and `_exit_edge`, which change the search geometry (§4.2): an off-frame target should be searched for at the edge it left, not in the middle.

2.5 Loss-cause classification (clean history for the EKF)

When entering occlusion, the EKF must coast on a clean velocity. But the last few frames before loss may be corrupted (the box was already drifting/shrinking). `_classify_loss_cause` decides how many recent history frames to discard, by casting **two votes**:

Area-trend vote. Fit a straight line to the recent box areas $a_0, \dots, a_{n-1}$ over time indices $t_i = 0, \dots, n-1$, by least squares. The slope of the best-fit line (doc 01 §4.1 idea; the closed form is)

$$\text{slope} = \frac{\sum_i (t_i - \bar{t})(a_i - \bar{a})}{\sum_i (t_i - \bar{t})^2}$$

measures whether the box was shrinking (negative slope) or steady/growing. Normalize by the median area, `norm_slope = slope / median(a)`. If `norm_slope` $\geq$ a small negative threshold (i.e. not meaningfully shrinking), vote **camera_motion** (the target was full-size right up to loss $\rightarrow$ it didn't dissolve behind something, the camera lurched).

Camera vote. If the exponentially-weighted camera displacement (doc 06 §4.1) exceeds a threshold, vote **camera_motion**.

Only if **both** votes say `camera_motion` is the cause classified as camera motion (and the recent history is kept clean, `skip = 0`). Otherwise it is occlusion, and a number of recent frames are skipped so the EKF rebuilds its velocity from pre-corruption history. Requiring both votes to agree makes the “trust the history” decision conservative — the cost of wrongly keeping corrupted history (a bad coast direction) is high.

3. Coasting: the EKF runs the search

Once in occlusion, there is no trustworthy measurement, so the EKF **coasts** (doc 04 §8, doc 05 §C.5): it keeps predicting (with camera compensation while in frame; pure covariance inflation $P \leftarrow P + Q$ while out of frame), its uncertainty growing each frame. Every occlusion frame (occlusion_update):

1. reads the EKF’s predicted box and sets the **held box** to it (clamped to the frame);
2. sets the **search center** (s_{cx}, s_{cy}) to the EKF center;
3. reads the EKF **velocity** (used by the DRM motion terms, doc 07 §4.3/4.7);
4. increments the occlusion frame counter `_occ_frames`;
5. handles **out-of-frame** edge pinning and re-entry (§4.2);
6. routes to the current **phase** (§4).

So the EKF is the navigator during occlusion: it says “the target is probably here and heading this way,” and the search machinery looks there. The growing uncertainty is reflected in a growing search region (§4.1) — the longer the target is gone, the wider we look.

4. The phase state machine

Recovery proceeds through phases keyed off `_occ_frames` (and a separate `reacq_confirm` sub-state). The dispatcher routes:

occlusion frame			
reacq_confirm active?	→ PHASE: reacq-confirm	(tentative lock-on streak)	§5
_occ_phase == 0	→ PHASE: siam	(fast SiamABC re-try)	§4.3
1 ≤ _occ_phase ≤ N	→ PHASE: collect	(gather YOLO candidates)	§4.4
_occ_phase > N	→ PHASE: final-DRM	(score + verify)	§4.5

The intuition is cheap-first: try the fast appearance matcher at the predicted spot; if that fails, spend a few frames collecting object-detector candidates; then score them against long-term memory and verify the best.

4.1 The growing search ROI (`_get_yolo_search_roi`)

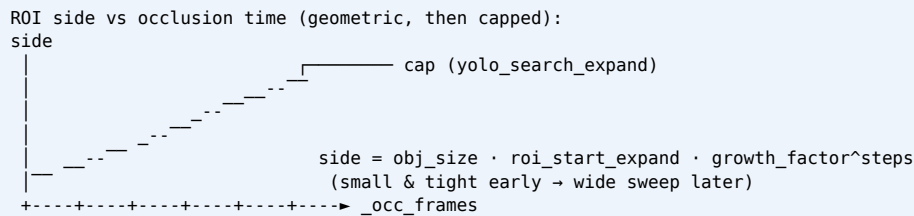
The detector (YOLO) is run not on the whole frame but on a **region of interest (ROI)** that grows geometrically with time, centered on the EKF search center. This is the spatial expression of the EKF’s growing uncertainty. With the object’s representative size `obj_size` (median of recent sizes, doc 01 §7.2) and time-step count

$$\text{steps} = \left\lfloor \frac{_occ_frames}{\text{growth_every}} \right\rfloor,$$

the **expansion factor** grows by a constant ratio per step, capped:

$$\text{effective_expand} = \min \left(\underbrace{\text{roi_start_expand} \cdot \text{growth_factor}^{\text{steps}}}_{\text{geometric growth}}, \text{yolo_search_expand} \right),$$

and the (square) ROI side is $\text{side} = \text{obj_size} \cdot \text{effective_expand}$, centered on (s_{cx}, s_{cy}) and clamped to the frame. **Geometric** growth ($\times$ ratio per step) means the search widens slowly at first, then accelerates — so a quickly-returning target is found in a tight, low-false-positive window, while a long absence eventually triggers a broad sweep. The cap stops it from growing without bound. Running YOLO on a crop (not the full frame) also means fewer distractors in view and faster inference.



A separate, looser parameter set is used for **tiny/far** targets (they need proportionally wider search), and during the warm-up window or after a failed recovery the ROI is simply centered on the frame to give the tracker a stable area to re-lock.

4.2 Out-of-frame handling

If the EKF center leaves the frame by more than a margin, the target is marked `_out_of_frame` with an `_exit_edge` (left/right/top/bottom). Then:

- the **search center is pinned to that edge** (e.g. exited right → $s_{cx} = W - 1$), and the ROI becomes a **strip** along the edge rather than a centered square — because the target will re-enter from the edge it left;
- the EKF **coasts by pure covariance inflation** (no camera warp), since there is no in-frame content to compensate against;
- **re-entry** is declared when the EKF center comes back inside the frame and its velocity points **inward** (e.g. exited right → $v_x < 0$). Requiring inward velocity (not just an inside position) prevents a noisy coast from prematurely declaring the target back.

4.3 Phase: fast SiamABC re-try (`occ_phase_siam`)

The cheapest attempt first: run the matcher (doc 03) seeded at the EKF-predicted location. The EKF has been coasting on the target’s last good velocity, so for a brief occlusion (the target slips behind a thin pole for a few frames) the predicted spot is often right, and the matcher re-locks immediately — no detector needed. If the matcher’s score clears the re-acquisition bar, recovery jumps to verification (§5). If not, advance to the collect phase.

4.4 Phase: candidate collection (`occ_phase_collect`)

For a handful of frames (`_cand_collection_frames`), run YOLO on the growing ROI (§4.1) and **accumulate** the detected boxes as candidates, also recording the camera velocities seen during these frames. Collecting over several frames (rather than one) gives more chances to catch the target the moment it reappears, and builds a richer candidate set for scoring. YOLO is **class-agnostic** or class-filtered depending on configuration — but it provides generic object proposals, not identity; identity is decided next, by memory.

4.5 Phase: memory scoring + verification (`occ_phase_final_drm`)

Now the decisive step. Score the collected candidates against the long-term appearance memory using the **composite DRM score** of doc 07 §4 (`drm_match / _occlusion_memory_match`), passing the EKF velocity (for the motion-direction terms) and the search center (for the distance penalty). This yields a ranked shortlist. Then **verify**: for each top candidate, re-run the matcher as if the target were there (`run_track_for_candidate`, doc 03 §.) — it injects the candidate as a hypothesis, computes the matcher’s score there, and restores state. (The KF motion prior is suppressed during this, doc 06 §2.4, because a genuinely re-acquired target is far from the stale trajectory and would be unfairly demoted.) A candidate is accepted only if its verified matcher score clears the **re-acquisition threshold** `reacq_threshold` (e.g. ≥ 0.70 ; itself possibly adaptive, doc 10).

If no candidate is verified, the tracker keeps coasting (the ROI grows further next frame). If detections exist but none survive scoring, a gentle **nudge** (`_nudge_toward_nearest`) biases the held box a small step toward the most plausible nearby detection (closest, weighted by appearance similarity) — so the EKF search center is pulled in roughly the right direction without committing to an unverified box. This is a soft correction, in keeping with the “prefer

soft over hard” theme.

Using DRM (long-term, occlusion-surviving anchors) here — rather than the short-term RAM or the matcher’s drifting dynamic template — is the whole point: after a long absence, only a stable, curated identity memory can tell the target from a look-alike that the detector also proposed. (Early in a sequence, before DRM has filled, recovery falls back to RAM matching, doc 07 §2.2.)

5. Verification and the confirmation streak

A single high verification score could be a fluke, so re-acquisition is **confirmed over multiple frames** before exiting occlusion. On a successful verification, the tracker enters a tentative lock-on (begin_reacq_confirmation): it seeds the matcher at the candidate box and, for the next `_reacq_confirm_frames` frames, requires the matcher to stay above `reacq_threshold`. A consecutive-frame streak counter tracks this; only when it is satisfied does the tracker **commit**:

- set `in_occlusion = False` (return to the normal world);
- **reseed** the EKF at the re-acquired box with a fresh velocity (doc 05 §C.5) — not nudge, because this is a legitimate teleport and the old trajectory is stale;
- reset the occlusion counters and emit the real coordinates.

This confirmation streak is, again, hysteresis (§2.2) — but on the exit side: just as we required persistence before declaring loss, we require persistence before declaring recovery. It trades a frame or two of latency for a large drop in false re-acquisitions onto clutter or impostors.

6. The whole recovery loop

```

ENTER OCCLUSION (entry_streak ≥ patience, guards passed)          $2
| classify loss cause → keep/skip recent history → rebuild EKF velocity $2.5
↓
each occlusion frame:
  EKF coast → held_box, search center, velocity; grow ROI; edge logic  $3,$4.1,$4.2
  |
  | phase siam:    matcher at EKF spot – re-lock if confident          $4.3
  | phase collect: run YOLO on ROI, accumulate candidates             $4.4
  | phase final-DRM: composite-score candidates (doc 07) → verify top-k $4.5
  |                   (verified ≥ reacq_threshold?)
  |                   | yes
  |                   ↓
  |                   reacq-confirm: stay confident for K frames      $5
  |                   | confirmed
  |                   ↓
  |                   reseed EKF, in_occlusion=False, emit real box → NORMAL $5

```

The design philosophy in one line: **coast on physics, propose with a detector, decide with long-term memory, and confirm before you trust**. Every stage is cheap-before-expensive and conservative-before-committal, so the tracker neither wastes compute nor snaps onto the wrong object.

7. Recap

- A single `in_occlusion` flag selects between the normal and occlusion worlds; during occlusion the public output is the held/EKF box (or zeros) with score 0.
- **Loss detection** uses an **effective threshold** (lowered+widened for tiny targets), **entry hysteresis** (`entry_patience` consecutive low-score frames), a **grace window**, and a **heavy-camera-motion block**; a **loss-cause** vote (area-slope + camera displacement) decides how much recent history to trust.

- The **EKF coasts** (camera-compensated in frame, covariance-inflating out of frame), providing the search center, held box, and velocity; out-of-frame targets are searched for at the edge they left and require inward velocity to re-enter.
- The **phase machine** goes fast-matcher → **detector candidate collection** → **DRM composite scoring + matcher verification**, with a **geometrically growing ROI** mirroring the EKF's growing uncertainty.
- A winner must clear `reacq_threshold` and survive a **confirmation streak** (exit hysteresis); commitment **reseeds** the EKF and returns to normal tracking.

Next: **09_FRAME_DYNAMICS.md** — a small, self-contained input augmentation that surfaces motion in the search crop to help the matcher lock onto tiny moving targets.

09 — Frame Dynamics: Injecting Motion Saliency

Prerequisites: doc 01 (absolute value, weighted blend, clamping), doc 02 (the search crop, coordinate windows), doc 03 (what the matcher consumes). This is a short, self-contained document about an **input augmentation**: for tiny/faint targets, SiamRAM can blend a short-term motion saliency signal into the search crop before the matcher sees it, so a moving speck pops out against static clutter. Implemented in `models/siamram/frame_dynamics.py`. It is independent of the motion filters (docs 05-06): those reason about where the target is; this changes what the matcher sees.

1. The problem: appearance collapses for tiny targets

A drone 20 px wide in a 1080p frame, or a distant low-contrast object, barely registers as appearance — its descriptor is weak and easily confused with background texture. But such a target is usually **moving relative to the static background**, and motion is a powerful, almost orthogonal cue. The idea (from the Anti-UAV literature: “A Simple Detector with Frame Dynamics is a Strong Tracker”) is to feed the network not just “what the scene looks like now” but also “what changed” — because what changed is, mostly, what moved.

2. Frame differencing: motion as a temporal gradient

The simplest motion signal is the **frame difference**: subtract consecutive frames. A static pixel barely changes between frames, so its difference is near 0; a pixel a moving object just entered (or left) changes a lot, so its difference is large. Taking absolute values, the difference map is a **motion saliency** image — bright where motion happened, dark where the scene was still. It is, in effect, a discrete temporal gradient $|\partial I / \partial t|$ (compare the temporal term I'_t in the optical-flow derivation, doc 05 §B.3).

The source paper concatenates the current frame with two such differences into a 6-channel input,

$$x_{\text{fd}} = [x_t, x_t - x_{t-1}, x_t - x_{t-2}],$$

and trains a detector from scratch on those 6 channels. Two differences (one- and two-frame gaps) capture motion at two short time scales, which helps with both fast and slow movers.

3. SiamRAM’s constraint and its workaround

SiamRAM’s matcher (doc 03) is a **frozen, pretrained, 3-channel** network. We cannot change its input to 6 channels, and we will not retrain it. So instead of concatenating the difference maps, SiamRAM **blends them additively** into the existing 3-channel search crop — an “additive motion emphasis.” Per frame:

1. Compute the two absolute short-term differences over the registered search window (oldest→newest in the buffer give x_{t-2} and x_{t-1}):

$$\text{diff}_1 = |x_t - x_{t-1}|, \quad \text{diff}_2 = |x_t - x_{t-2}|.$$

2. Average the available differences into one saliency map (one map if only x_{t-1} is buffered, two if both):

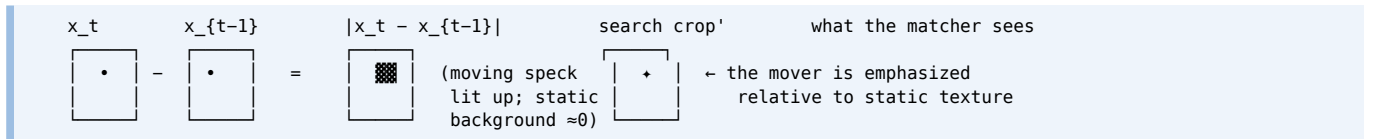
$$\text{saliency} = \frac{1}{n} \sum_k \text{diff}_k, \quad n \in \{1, 2\}.$$

Averaging the two mirrors the paper’s two-difference triplet while staying single-(per-channel) so it can be added to a 3-channel crop.

3. **Register and resize** the saliency to the search crop’s pixel grid (doc 02 §4.4): the differences are computed over the same window of the full frame the crop came from, then resized to the crop’s $S \times S$ size so the saliency lines up pixel-for-pixel with what the backbone consumes.
4. **Scale, clip, blend, clamp.** Multiply the saliency by a gain scale, optionally **clip** it from above (to suppress outlier difference pixels from compression noise or a sudden global illumination change), then add it into the crop with a small weight w (blend_weight), and re-clamp to the valid 8-bit range so the result is still a legal image:

$$\text{crop}' = \text{clip}\left(\text{crop} + w \cdot \min(\text{scale} \cdot \text{saliency}, \text{clip}), 0, 255\right).$$

The effect: pixels that moved are brightened relative to static clutter, nudging the matcher’s response toward the moving target without overwriting appearance. With $w = 0$ this is an exact no-op (the crop is unchanged), so the augmentation can be disabled cleanly. The default w is deliberately small (e.g. 0.06) — the frozen backbone still relies mainly on appearance; the motion is a gentle emphasis, not a takeover.



4. The tiny-only gate (why it’s not always on)

Blending motion into every crop would corrupt appearance on the majority of frames where the target is large and clearly visible — exactly where the frozen backbone is already strong. So by default the blend is **restricted to tiny targets**: it is applied only when the box’s area fraction of the frame is at or below a small threshold (tiny_area_fraction, e.g. 0.001):

$$\text{apply blend} \iff \frac{w_{\text{box}} h_{\text{box}}}{H \cdot W} \leq \text{tiny_area_fraction}.$$

This confines the augmentation to the regime where appearance collapses and motion genuinely helps, leaving normal/large-target crops untouched. (Setting `tiny_only = False` blends on every frame — the unrestricted variant.) Note the gate uses the target’s size, the same scale-awareness that recurs throughout SiamRAM (docs 02, 06, 08).

5. State and edge cases

The processor **owns a tiny rolling buffer** of the previous two full frames (so the host carries no extra state). Two consequences:

- On the **first two frames** of a clip there are not yet two previous frames, so there is no motion signal — the crop is returned unchanged. (The buffer is still advanced so motion is available next frame.)
- The buffer stores **copies** of the frames, so a later in-place edit of the caller's frame cannot corrupt the motion history.

When the feature is disabled the host never even constructs the processor, so the crop is byte-for-byte the legacy crop and no frame-difference work runs — zero cost when off.

6. Where it sits in the frame

Frame dynamics acts in the normal update, after the camera/Kalman search-placement priors (doc 06) and before the matcher extracts its features (doc 03):

```

... GMC recenters the search (doc 06) ...
    ↓
frame-dynamics: blend motion saliency into the search crop (this doc) ← tiny targets only
    ↓
SiamABC forward on the (possibly emphasized) crop (doc 03)

```

So it is purely a perceptual aid: it does not move the search or change any threshold; it makes the moving target slightly more salient in the pixels the matcher reads.

7. Recap

- Tiny/faint targets have weak appearance but usually **move** against a static background; **frame differencing** $|x_t - x_{t-k}|$ is a cheap motion-saliency signal (a discrete temporal gradient).
- SiamRAM cannot extend its frozen 3-channel backbone, so it **additively blends** the averaged absolute differences into the search crop: $\text{crop}' = \text{clip}(\text{crop} + w \cdot \min(\text{scale} \cdot \text{saliency}, \text{clip}), 0, 255)$, with a small weight w .
- A **tiny-only gate** restricts the blend to small targets so normal-target appearance is never corrupted; $w = 0$ or a non-tiny target is an exact no-op.
- It owns a 2-frame buffer (no-op for the first two frames), and is a perceptual aid only — it changes what the matcher sees, not where it looks.

Next: **10_ADAPTIVE_CONTROLLERS.md** — the small EMA-based estimators that retune the tracker's key thresholds and rates to each video's difficulty, so no single fixed number has to fit every sequence.

10 — Adaptive Controllers: Self-Tuning Thresholds

Prerequisites: doc 01 (EMA, linear interpolation, clamping), doc 03 (the confidence score, the dynamic-template admission bar), doc 06 (heavy-camera-motion magnitude, target speed), doc 07 (the DRM composite score and `score_target_against_drm`), doc 08 (occlusion entry / re-acquisition thresholds). This document explains the small **EMA-based estimators** that retune SiamRAM’s key thresholds and rates to each video’s difficulty, so that no single fixed number has to work for an easy, distinctive target and a hard, tiny, low-texture one at the same time. Implemented in `auto_conf_threshold.py`, `auto_margin.py`, and `auto_template_rate.py`.

1. Why fixed thresholds fail, and the general pattern

Consider the occlusion-entry threshold (doc 08 §2). A distinctive target scores ~ 0.9 when correctly tracked, so a 0.5 entry bar means “a real, large confidence drop is needed before we declare loss.” But a genuinely hard target may score ~ 0.55 – 0.65 even when perfectly tracked — for it, that same 0.5 bar would fire a false loss on nearly every frame. One number cannot serve both. The same tension afflicts the dynamic-template admission bar, the re-acquisition margin, and the template update rate.

The fix is a **controller** that observes how the genuine target behaves during healthy tracking and sets the threshold relative to that. All three controllers share one design pattern, worth stating once because it recurs:

Measure a signal from the real target during confident tracking → smooth it with an EMA → map that smoothed value to a threshold → guard with warm-up, minimum-samples, and confident-only sampling.

Two guards make this safe:

- **Warm-up + minimum samples.** Until enough samples are collected, the controller reports a fixed warm-up value, so an unwarmed controller behaves exactly like the legacy fixed threshold. (Construction also keeps the warm-up value inside the legal output range, so value is always valid.)
- **Confident-only sampling (avoid feedback loops).** The controllers fold in a sample only on healthy frames (score above the loss threshold, not in distractor mode), on a cadence (`due/n_frames`). This keeps genuine pre-loss declines out of the statistic — otherwise the threshold would drift down as the target starts to disappear and delay the very loss it should detect, a dangerous positive-feedback loop. Sampling only confident frames breaks that loop: the bar reflects how the target scores when actually locked on.

The EMA itself is doc 01 §6: $\hat{s} \leftarrow (1 - \alpha)\hat{s} + \alpha z$, a one-line recency-weighted average with memory length $\sim 1/\alpha$.

2. AutoConfThreshold — EMA-to-range mapping

Used for the **occlusion-entry** confidence threshold and (with its own knobs) the **dynamic-template admission** bar (doc 03 §9.2). It maps an EMA of the target's own tracking score onto an output threshold range by a **linear map**.

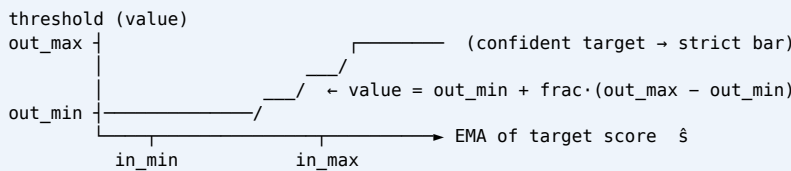
2.1 The mechanism

Maintain $\hat{s}$ = EMA of the per-frame confident score. Map it from an input range $[\text{in_min}, \text{in_max}]$ onto an output range $[\text{out_min}, \text{out_max}]$ (doc 01 §3.4-3.5):

$$\text{frac} = \text{clip}\left(\frac{\hat{s} - \text{in_min}}{\text{in_max} - \text{in_min}}, 0, 1\right), \quad \boxed{\text{value} = \text{out_min} + \text{frac} \cdot (\text{out_max} - \text{out_min}).}$$

Read it: locate $\hat{s}$ as a fraction of the way through the input band (clamped to $[0, 1]$ so values outside the band saturate), then place the threshold the same fraction of the way through the output band. So:

- A target the tracker is consistently **very confident** on (high $\hat{s}$, near in_max) gets a **high** entry bar (near out_max): a real, large confidence drop is required before declaring loss.
- A target it is only **weakly confident** on (low $\hat{s}$, near in_min) gets a **low** entry bar (near out_min): its normally-low scores are not mistaken for occlusion.



It is a monotonic, saturating remap: outside $[\text{in_min}, \text{in_max}]$ it clamps to the nearest output bound. Degenerate input band ($\text{in_max} \approx \text{in_min}$) is handled by snapping to 0 or 1.

2.2 The lifecycle

`due(frame_idx)` returns true once `n_frames` have elapsed since the last sample (a cadence). `observe(score, frame_idx)` folds the score into the EMA and, once `samples ≥ min_samples`, refreshes `value` via the map. `reset()` (on re-init) forgets everything and returns to warmup. Because the threshold the next frames compare against is refreshed after this frame, there is a one-frame lag — harmless, and the same convention across all the controllers.

3. AutoDrmMargin — sit just below the target's own score

Used for the occlusion **re-acquisition margin** μ (doc 07 §4.8, doc 08 §4.5): the bar a candidate's composite DRM score must clear. The insight (doc 07 §4.9): we can measure how high the genuine target scores against its own DRM anchors via `score_target_against_drm`, using the **same composite formula** the matcher uses for candidates — so the number is on the same scale as μ . Set the margin **just below** where the true target lives, so real re-acquisitions clear it but impostors (which score lower) don't.

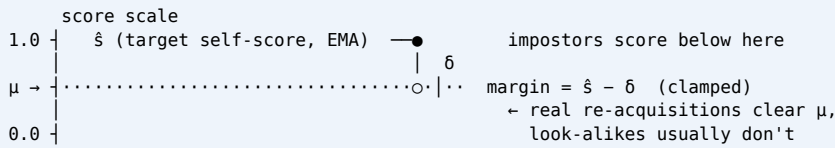
3.1 The mechanism

Maintain $\hat{s}$ = EMA of the target's self-score against DRM (sampled on confident frames). The margin is that EMA minus a small offset, clamped to a legal range:

$$\boxed{\mu = \text{clip}(\hat{s} - \delta, \mu_{\min}, \mu_{\max}).}$$

The offset δ (delta) is the safety gap: the true target typically scores around $\hat{s}$, so a bar at

$\hat{s} - \delta$ admits it with margin to spare while staying above the lower scores a look-alike produces. The clamp keeps μ inside $[\mu_{\min}, \mu_{\max}]$ regardless. For a low-texture/small/fast target whose self-score is genuinely lower, the margin follows it down (so valid re-acquisitions aren't rejected); for a distinctive target it rides higher (so distractors are excluded). Same due/observe/reset/warm-up lifecycle as §2.2 (with its own cadence n_frames and $min_samples$).



4. AutoTemplateRate — motion drives the update cadence

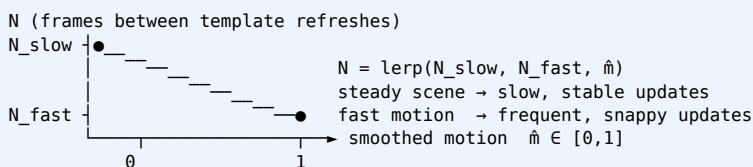
Used to set the dynamic-template refresh cadence N and the memory-window size (doc 03 §9). The trade-off: when the scene is **steady**, appearance is stable, so the template should update slowly (large N / large window = lots of averaging, very stable); when the **camera or target moves fast**, appearance changes quickly, so it should update fast (small N / small window = snap to the latest look). So the cadence should track motion.

4.1 The mechanism

Each frame, form a single “**fastness**” scalar $m \in [0, 1]$ by combining the camera-motion magnitude (doc 06 §4: the bbox-normalized displacement) and the target’s own speed, taking the **max** (so either one being fast drives fast updating). Smooth it with an EMA, $\hat{m} \leftarrow (1 - \alpha)\hat{m} + \alpha m$, then **linearly interpolate** (doc 01 §3.4) the cadence and window between their slow-end and fast-end values as $\hat{m}$ goes $0 \rightarrow 1$:

$$N = \text{round}(\text{lerp}(N_{\text{slow}}, N_{\text{fast}}, \hat{m})), \quad \text{window} = \text{round}(\text{lerp}(\text{win}_{\text{slow}}, \text{win}_{\text{fast}}, \hat{m})),$$

with $N_{\text{fast}} < N_{\text{slow}}$ and $\text{win}_{\text{fast}} < \text{win}_{\text{slow}}$ (both clamped ≥ 1). At $\hat{m} = 0$ (steady) you get the slow, stable rate; at $\hat{m} = 1$ (fast) the fast, snappy rate; in between, a straight-line blend. Before any observation it sits at the slow end, so an unobserved controller reproduces a fixed slow rate.



This controller takes over from the older binary “camera-motion template adapt” / “target-motion template adapt” switches when enabled (they would otherwise fight over N /window); when it is off, the legacy fixed-rate or binary paths are untouched.

5. Other self-adjusting quantities (cross-references)

The same “never let a fixed number fail; never let an adaptive number run away” philosophy appears in a few places already covered:

- **Running confidence with a floor** (doc 03 §9.5): an EMA of the per-frame score is the dynamic-template admission bar, but it is clamped from below by a floor so a hard sequence can’t collapse it to 0 and let any frame into memory.

- **Blur-admit reference** (doc 03 §9.3): a relative sharpness gate whose reference EMA updates even on a veto, so a sustained texture drop lowers the bar and lets updates resume — only a one-off blur spike is rejected.
- **Effective threshold switching** (doc 08 §2.1): a non-EMA but related idea — swap to a lower threshold (and wider search) for tiny targets, because their honest scores are lower.

These are not separate “controllers” with the §1 lifecycle, but they embody the same principle: thresholds should reflect the observed difficulty of the current target, and adaptive quantities need guardrails.

6. Why this design is safe

It is worth making explicit why letting the tracker move its own thresholds doesn’t spiral:

1. **Sampling is gated to confident, healthy frames.** The statistics describe the target when locked on, not when failing — so the bars don’t drift to mask a real loss (§1).
2. **EMAs are slow.** With a modest α , a single odd frame barely moves any threshold; it takes a sustained change to shift the bar, which is exactly when shifting is warranted.
3. **Everything is clamped.** Each controller’s output lives in a fixed legal range ($[\text{out_min}, \text{out_max}]$, $[\mu_{\min}, \mu_{\max}]$, ≥ 1 for N/window), so no value can run off to an absurd extreme.
4. **Unwarmed = legacy.** Before `min_samples` are collected, each controller returns its fixed warm-up value, so behavior degrades gracefully to the hand-tuned constant rather than to something undefined. A numeric (non-“auto”) config never even constructs the controller.

7. Recap

- A single fixed threshold can’t fit both easy and hard targets; SiamRAM uses **EMA-based controllers** that set thresholds relative to how the genuine target behaves during confident tracking.
- **AutoConfThreshold**: EMA of the target score $\rightarrow$ **linear map** $[\text{in_min}, \text{in_max}] \rightarrow [\text{out_min}, \text{out_max}] \rightarrow$ occlusion-entry / dynamic-admission bar; confident targets get a strict bar, weak ones a lenient bar.
- **AutoDrmMargin**: EMA of the target’s self-score against its own DRM, minus a safety offset δ , clamped $\rightarrow$ re-acquisition margin that sits just below where the true target lives.
- **AutoTemplateRate**: a $[0, 1]$ motion “fastness” (max of camera and target motion), EMA-smoothed, **lerped** into the template cadence N and window — slow when steady, fast when moving.
- All share guards — **warm-up, minimum samples, confident-only sampling, clamping** — so adaptation is responsive but cannot feed back on itself or run away.

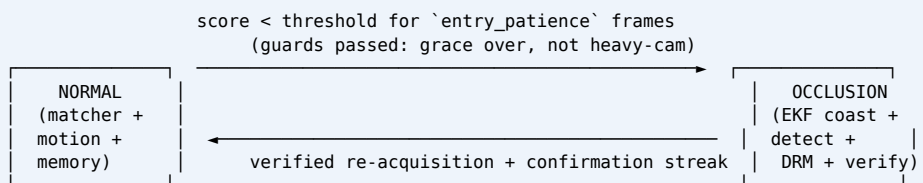
Next, the finale: **11_PUTTING_IT_ALL_TOGETHER.md** — the per-frame conductor that wires every subsystem in this series into one coherent state machine.

11 — Putting It All Together: The Conductor

Prerequisites: ideally all prior documents, but especially doc 00 (the pipeline skeleton), doc 03 (matcher), doc 05 (EKF), doc 06 (motion priors), doc 08 (occlusion). This final document shows the **conductor** — the master per-frame routine `update()` in `tracker.py` — that wires every subsystem of this series into one coherent state machine. We trace a frame end-to-end, explain each branch and why it is ordered as it is, and walk through three concrete scenarios. (The real code also contains `distractor-mode` and `spike-watcher` branches; per the scope of these notes those are deprecated/out of scope and are omitted here — wherever they would sit, we say so.)

1. The two-state machine

At the top level SiamRAM is a state machine with two macro-states, selected each frame by the boolean `in_occlusion` (doc 08 §1):



The invariant: per frame, exactly one of `_normal_update` or `_occlusion_update` runs. They are disjoint worlds with separate logic; `in_occlusion` is the single switch between them. Everything below is the choreography around that switch.

2. The per-frame entry (`update / _update_impl`)

Every frame, before either world runs, the conductor does a fixed preamble — the steps shared by both states:

```
1. proc_frame = prescale(frame)                full-res → proc space (doc 02 §6)
2. frame_idx += 1 ; reset per-frame scratch (yolo cache, ...)
3. H, H_reliable, gray = estimate_homography(proc) camera motion (doc 05 Part B)
   store_last_H, last_H_reliable                (consumed by EKF, GMC, heavy-motion)
4. is_heavy_camera_motion(proc)                  refresh the heavy-motion verdict (doc 06 §4)
5. EKF step:
   if in_occlusion and out_of_frame: P ← P + Q    coast, no warp (doc 05 §C.5)
   else:                                       EKF.predict(H, H_reliable) camera-aware predict (doc 05 §C)
6. branch:
   if in_occlusion: (bbox, score) = occlusion_update(proc)    doc 08
   else:           (bbox, score) = normal_update(proc)       §3 below
7. prev_gray = gray                                set up next frame's optical flow
8. scale bbox back to full-res (× 1/s); emit (bbox, score, in_occlusion, yolo)
```

Three things to notice in the ordering:

- **Camera motion is estimated first**, because both worlds need it: the EKF predict uses H , GMC uses H , the heavy-motion gate uses H , and the loss-cause classifier uses the camera-displacement history derived from H . Estimate once, share everywhere.

- **The EKF predicts before the branch.** Whether we are tracking or occluded, the motion belief advances every frame — so when occlusion begins, the EKF is already coasting on a fresh, camera-compensated prediction; and when tracking, the predicted center is available to place the search.
- **Coordinate discipline.** The frame is prescaled to proc space at step 1 and the answer scaled back at step 8 (doc 02 §6). Everything in between — every subsystem in this series — is proc-space. This is what lets the matcher, EKF, YOLO, and memory all speak the same pixel units.

3. The normal update, ordered (`_normal_update`)

When `in_occlusion` is false, this is the hot path that runs every visible frame. Its order is load-bearing — each step depends on the previous ones — so we annotate why each is where it is.

```
(a) template-freeze / memory-freeze bookkeeping          (distractor-mode lifecycle – out of scope)
(b) GMC search prior: recenter search on H·(prev box)    doc 06 §3 — before the matcher looks
(c) template-rate adaptation: set N / window from motion doc 10 §4 (or legacy binary adapt)
(d) frame-dynamics: blend motion saliency into the crop   doc 09 — tiny targets only
(e) pred_bbox, score = SiamABC.update(frame)              doc 03 — the visual match
    (inside: optional KF-centered search + KF-IoU fusion doc 06 §2)
(f) class warm-up / detectability probe (early frames)    (YOLO sanity checks; see §3.1)
(g) effective_threshold = compute_effective_threshold()   doc 08 §2.1 (tiny-switch; may be adaptive)
    [ here the real code branches into distractor-mode / spike handling – OUT OF SCOPE ]
(h) heavy_cam_motion, disp = is_heavy_camera_motion(pred) doc 06 §4
(i) sample adaptive controllers on confident frames:      doc 10
    reacq_threshold ← AutoDrmMargin / reacq EMA
    conf_threshold ← AutoConfThreshold EMA
(j) occlusion-entry decision:                              doc 08 §2
    grace window? heavy-cam block? → hold streak at 0
    else if score < effective_threshold: entry_streak += 1
    else: entry_streak = 0
    if entry_streak ≥ effective_entry_patience (and not in grace):
        classify loss cause, rebuild EKF, set in_occlusion = True; hand off to occlusion next frame
(k) HEALTHY frame (no entry): commit
    EKF.update(pred_bbox)                                  doc 05 §C.4
    record camera-motion history; update stable references
    admit appearance descriptor to RAM (→ maybe promote to DRM) doc 07
    (SiamABC also admits to its memory window + refreshes Z)   doc 03 §9
→ return (pred_bbox, score)
```

3.1 The early-frame YOLO probes (brief)

Steps (f) run only in the first frames. **Class warm-up** runs the detector to learn which object class the target most likely is, so later occlusion-phase YOLO can optionally filter to that class (doc 08 §4.4). The **detectability probe** checks whether the detector can even see the target at all; if not, the system leans more on the appearance matcher and less on detector-based recovery. These are configuration of the recovery machinery, not part of the per-frame match, which is why they sit off to the side.

3.2 Why entry is gated so heavily

Step (j) is where NORMAL can flip to OCCLUSION, and it is deliberately reluctant (doc 08 §2): a **grace window** early in the sequence, a **heavy-camera-motion block**, **hysteresis** (entry_patience consecutive sub-threshold frames), and patience that increases under camera motion. The reason: entering occlusion needlessly is expensive (the matcher stops trusting itself, the detector spins up, the output goes to “no answer”), and most low-score frames are not true losses (blur, a fast pan, a hard but visible target). The gates cost a frame or two of latency on a true loss in exchange for far fewer false losses — the same latency-for-reliability trade as the exit-side confirmation streak (doc 08 §5).

3.3 Why the EKF updates only on healthy frames

Step (k) calls `EKF.update` only when no occlusion entry happened — i.e., when we have a trustworthy measurement. On a frame that triggers entry, the matcher’s box is suspect, so we do **not** feed it to the EKF; instead the loss-cause classifier (doc 08 §2.5) decides how much

recent history to discard so the EKF coasts on a clean velocity. Feeding a bad box to the filter would corrupt exactly the velocity the recovery search depends on.

4. The full wiring: where every subsystem plugs in

A single table mapping the series to the conductor:

Subsystem (doc)	Runs in	When in the frame	Role
Homography / camera motion (05)	both	preamble step 3	shared camera estimate H + reliability
BBoxEKF predict (04, 05)	both	preamble step 5	advance motion belief (camera-aware)
Heavy-camera-motion gate (06)	both	preamble 4 / normal (h)	block false entry, drive adaptation
GMC search prior (06)	normal	(b) before matcher	recenter search on camera-comp. location
Template-rate controller (10)	normal	(c)	set dynamic-template cadence from motion
Frame dynamics (09)	normal	(d)	motion-saliency blend (tiny targets)
SiamABC matcher (03)	normal	(e)	the visual match → box + score
KF motion prior + fusion (06)	normal	inside (e)	reweight response toward motion
Effective threshold (08, 10)	normal	(g)	the loss bar (tiny-switch / adaptive)
Adaptive controllers (10)	normal	(i)	retune thresholds to this video
Occlusion entry (08)	normal	(j)	NORMAL → OCCLUSION switch
BBoxEKF update (05)	normal	(k)	correct motion from a good box
RAM/DRM admission (07)	normal	(k)	grow appearance memory
EKF coast + search nav (05, 08)	occlusion	step 5 / occlusion	extrapolate, steer the search
Phase machine: siam/collect/DRM (08)	occlusion	occlusion	re-detect the target
DRM composite scoring (07)	occlusion	occlusion phase	rank re-acquisition candidates
Re-acquisition verify + confirm (08)	occlusion	occlusion phase	OCCLUSION → NORMAL switch

Read top to bottom, it is the whole tracker: estimate camera motion → advance the motion belief → (if tracking) steer & run the matcher, fuse motion, decide if lost, else update motion & memory → (if lost) coast, detect, score against memory, verify, recover.

5. Three worked frames

To make the choreography concrete, here are three representative frames narrated through the conductor.

5.1 A healthy frame (target visible, calm scene)

```

preamble: H estimated (reliable, tiny motion); EKF.predict warps center ~0 px, adds velocity.
normal:   (b) GMC prior ~no shift (camera still).
          (e) SiamABC matches; sharp peak; score 0.88.
          (g) effective_threshold 0.55; (h) not heavy-cam.
          (i) controllers fold 0.88 into their EMAs (confident frame).
          (j) 0.88 > 0.55 → entry_streak = 0 (no loss).
          (k) EKF.update toward the box (velocity refined); descriptor admitted to RAM
              (and, if it agrees with neighbors, promoted to DRM); Z refreshed on cadence.
emit:     box scaled back to full-res, score 0.88, in_occlusion = False.

```

Everything agrees; the safety nets are quietly maintaining state for the day things go wrong.

5.2 Entering occlusion (target slips behind a wall)

```

frames t..t+2: score falls to 0.30, 0.28, 0.31 (< 0.55). Scene calm (not heavy-cam, grace over).
              (j) entry_streak climbs 1 → 2 → 3. At streak ≥ entry_patience (say 3):
                  loss-cause vote: box was full-size up to loss BUT camera barely moved
                      → not both camera votes → cause = OCCLUSION → skip a few corrupted history frames,
                      rebuild EKF velocity from clean history.
                  in_occlusion = True.
next frame: OCCLUSION world. EKF coasts (predict with whatever camera motion); held_box = EKF box;
            phase 0 (siam) re-tries at the predicted spot – still hidden, score low; advance to collect.
emit during occlusion: held/EKF box (or zeros), score 0.0, in_occlusion = True.

```

The hysteresis ensured a few low frames didn't trigger this prematurely; the clean-history step ensured the coast direction is right.

5.3 Re-acquiring (target reappears across the frame)

```

occlusion, _occ_frames large: ROI has grown (geometric); YOLO proposes 3 boxes in the wide ROI.
phase final-DRM: composite-score each candidate against DRM anchors
                  (appearance + IoU + motion-direction along EKF velocity + recency – distance penalty).
                  Best candidate scores 0.83 ≥ skip_threshold → shortlist = [best].
verify: run_track_for_candidate seeds the matcher there (KF prior suppressed); matcher score 0.78
        ≥ reacq_threshold (0.70) → begin reacq-confirm.
confirm: matcher stays ≥ 0.70 for K consecutive frames → COMMIT:
        reseed EKF at the new box with fresh velocity; in_occlusion = False; emit real coords.
back to NORMAL: next frame is an ordinary healthy frame ($5.1), now locked onto the true target.

```

Long-term memory (DRM) told the true target from the other detected objects; verification + confirmation made sure before trusting it; the EKF reseed (not nudge) acknowledged the legitimate teleport.

6. The design philosophy, in one view

Step back from the code and the whole system is one idea applied repeatedly: **a fast, myopic appearance matcher, surrounded by layered, mostly-soft safety nets, each handling one failure mode, each conservative about taking control and conservative about giving it up.**

- Appearance (doc 03) is fast but myopic and identity-blind.
- Motion (docs 04–06) supplies physics: where the target should be (EKF), where the camera moved it (homography/GMC), and which response peak is trajectory-consistent (KF fusion).
- Memory (doc 07) supplies long-term identity, so the target can be told from look-alikes after a gap.
- Occlusion recovery (doc 08) supplies the structured fallback when appearance fails: coast on physics, propose with a detector, decide with memory, confirm before trusting.
- Perception aids (doc 09) make hard (tiny) targets more visible.
- Adaptive controllers (doc 10) make every threshold fit the difficulty of the current video instead of a global guess.

And throughout, recurring principles you can now name on sight:

1. **Hysteresis** before switching states (enter and exit occlusion only after persistence).
2. **Soft penalties over hard kills** (IoU-gate scales the score by 0.25, doesn't zero it; nudges over commits).

3. **Scale normalization** (displacements in box diagonals; noise scaled by target size) so one rule fits all target sizes.
4. **Measure the genuine target, threshold relative to it** (the adaptive controllers).
5. **Estimate once, share everywhere** (one homography feeds five consumers).
6. **Graceful fallback when a signal is missing** (no homography → no warp; empty DRM → RAM match; feature off → exact no-op).

That is SiamRAM: not one clever trick, but a disciplined stack of small, individually understandable, mathematically grounded mechanisms — every one of which you have now derived from first principles.

7. Where to go next

- Re-read **doc 05** with the conductor in mind: you will see exactly when predict, update, reseed, and the coast path fire.
- Trace a real video with debug logging and watch `entry_streak`, the heavy-motion verdict, and the phase counter move through the transitions of §5.
- The two subsystems we deliberately skipped — **distractor mode** and the **spike watcher** — build on the same primitives (camera-compensated displacement, appearance similarity, hysteresis); the foundations in docs 01-07 are exactly what you'd need to read them, should you choose to.

You now understand the whole tracker, end to end, mathematically. That was the goal.